



Data Warehouses

Lecture 12

Lecture Goals

Data flows in DW

Metadata

SSoT, SVoT, MDM

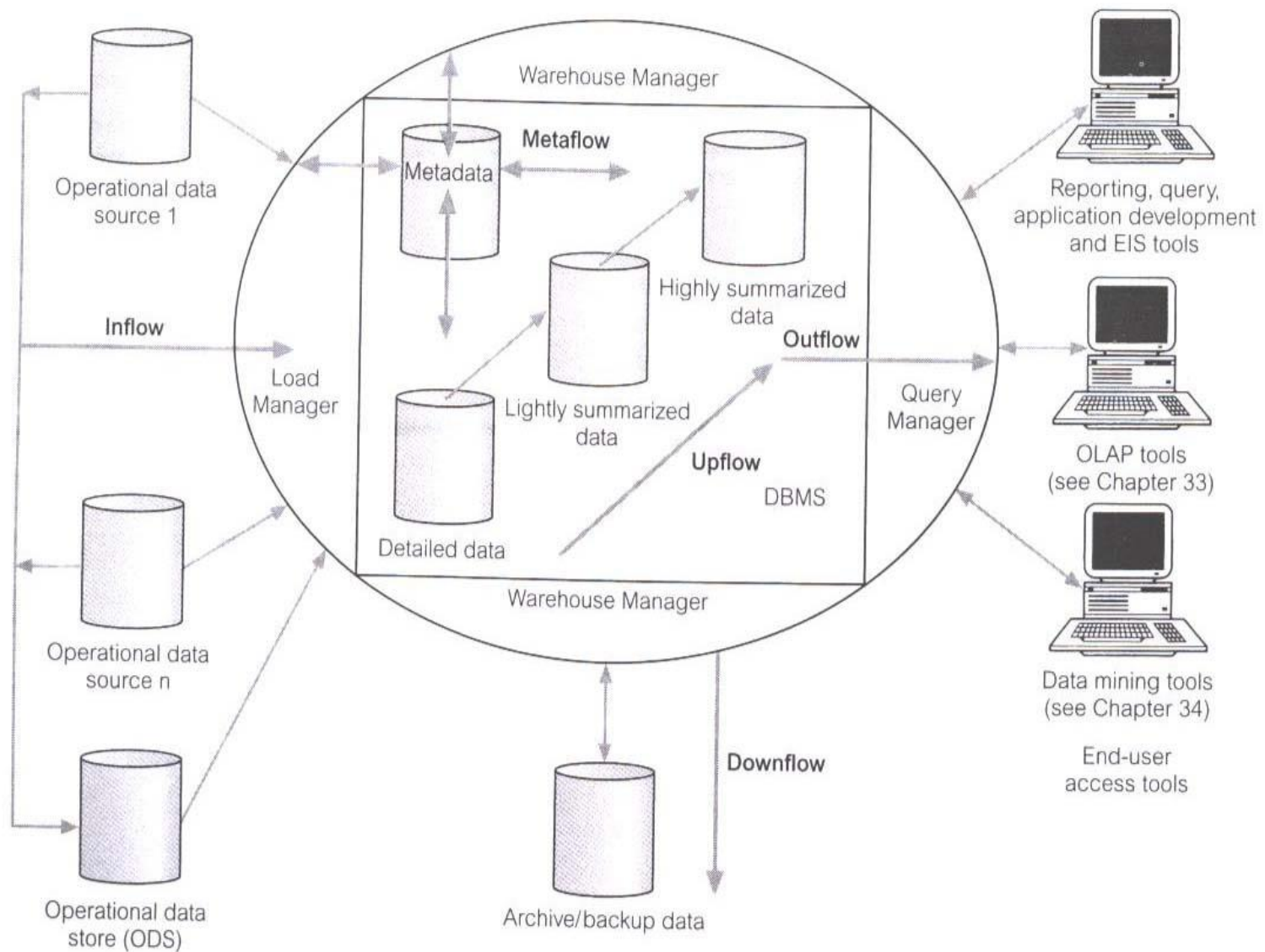
Data Warehouse Architectures

Data Vault

Modernization

Data Flows in DW





upstream

downstream

Data Flows

Inflow

The processes associated with the extraction, cleansing, and loading of the data from the source systems into the data warehouse .

Upflow

The process associated with adding value to the data in the warehouse through summarizing, packaging and distribution of the data

Downflow

The processes associated with archiving and backing - up of data in the warehouse

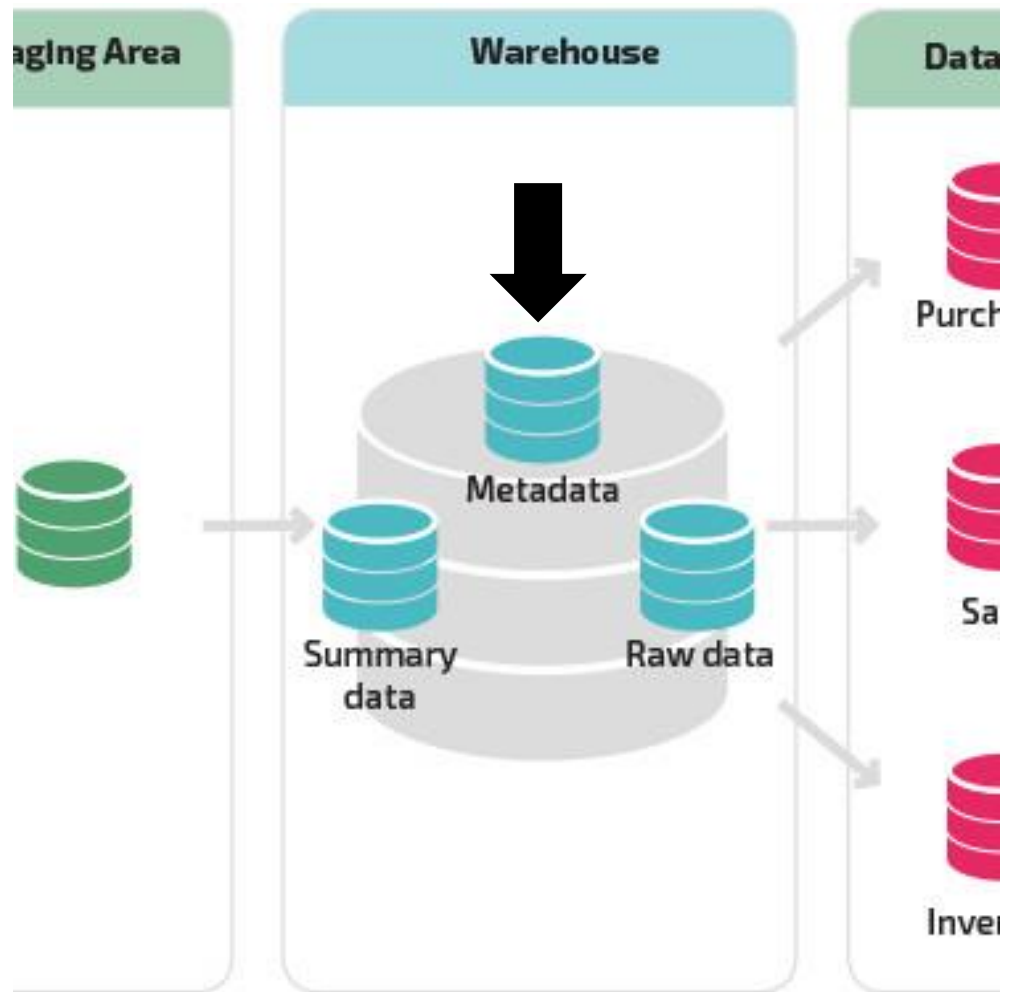
Outflow

The process associated with making the data available to the end-users

Meta-flow

The processes associated with the management of the metadata

METADATA





METADATA

Metadata is data about data

Metadata in data warehouses

provides the necessary information to effectively manage, understand, and utilize data,

specifies the source, usage, values, and features of data warehouse data

defines how data can be changed and processed.

Metadata is the backbone of a well-managed data warehouse

data discovery and understanding

data quality and governance

Integration and transformation

developing effective business intelligence and analytics solutions.

METADATA

Data Discovery and Understanding

"Where did this data come from?" ; "What does this data represent?" ; "Is this data reliable?"

Data Governance and Quality

"Who owns this data?" ; "Who has access to this data?" ; "What are the rules and constraints for this data?"

Data Integration and Transformation

"How is this data related to other data?" ; "What transformations have been applied to this data?"

Business Intelligence and Analytics

"What reports and dashboards use this data?" ; "How can I find the data I need for my analysis?"

Data Operations and Maintenance

"When was this data last updated?" ; "What are the performance characteristics of this data?"

Metadata

Taxonomy of Metadata

Business Metadata (The Semantic Layer)

Business glossary, KPI logic, ownership

Technical Metadata (The Structural Layer)

Schema and tables, data lineage, transformation logic

Operational Metadata (The Runtime Layer)

Job statistics, data quality scores, usage patterns

Metadata

Administrative - focus:

describes the technical characteristics of the data and its storage.

*definition of sources, tools, ...
schemas, dimension hierarchies, ...
rules for extraction, cleaning, ...
refresh, purging policies
user profiles, access control, ...*

Business - focus:

describes the business context of the data.

*business terms & definition
data ownership, charging*

Usage - focus:

records how data is used and accessed.

*User access logs
Report and dashboard usage statistics,
Query performance metrics,
Data usage trends and patterns*

Operational - focus:

describes the operational aspects of data management.

*data lineage
data currency (e.g., active, archived, purged)
use stats, error reports, audit trails*

Metadata

The "5 W's" of Metadata

Who

Identifies data creators, owners (Data Stewards), and users.

What

Defines the entity, its format (integer, string), and its business definition.

When

Tracks creation dates, update frequency (SLA), and last access times.

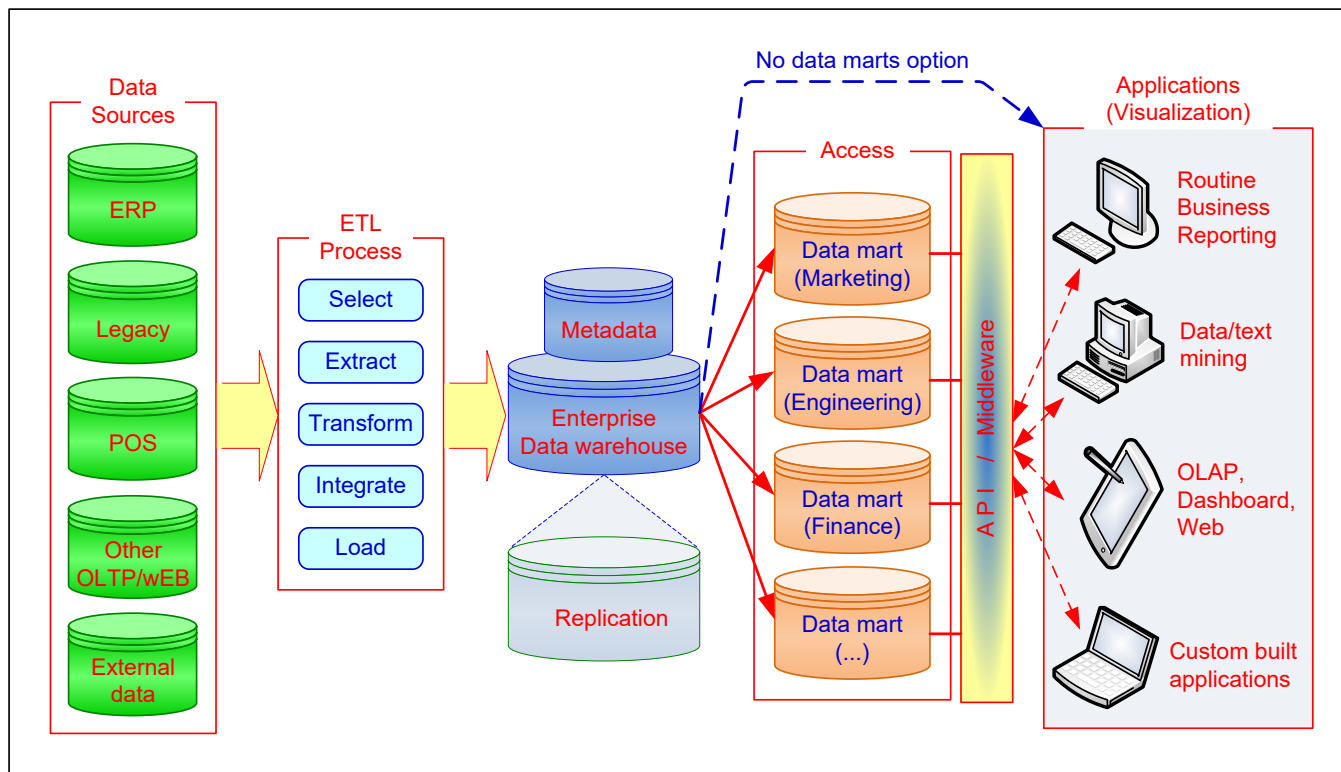
Where

Points to the physical location (cloud bucket, database cluster) and its origin (Source System).

Why

Explains the business justification for collecting the data and its intended use (compliance, marketing, etc.).

DW



Metadata

Metadata in the Data Lifecycle

In modern architectures, metadata is not just a record
it is the driver of automation.

Metadata-Driven Pipelines

Instead of writing custom code for every new table, engineers create generic templates.

The pipeline reads metadata (table name, source URL, data types) and dynamically creates the ingestion code.

Data Lineage & Impact Analysis

Upstream Lineage

"Where did this number come from?" (Critical for debugging).

Downstream Impact

"If I change the length of the 'ZipCode' column, which 50 reports will break?"
(Critical for change management).



Data Warehouse system

<https://www.youtube.com/watch?v=rvURMymCpJM>

Data warehouse - purpose

Data warehouse is all about creating a single version of truth for historical data from various sources and making it easily accessible, in a usable format and quick to query for business insights.

In such a way, that business analysts and management can quickly uncover trends, patterns or anomalies about business performance and decide on what's the best thing to do.

SSoT and SVoT

In the realm of Data Warehousing (DW) and Business Intelligence (BI), **Single Source of Truth (SSOT)** and **Single Version of Truth (SVOT)** are frequently used interchangeably.

Consider this classic corporate scenario:

Setting: The CEO sits down in a board meeting.

The VP of Sales stands up and says, *"We made \$10 Million in revenue this quarter!"*

The VP of Finance stands up and says, *"Actually, we made \$8.5 Million in revenue this quarter."*

Both executives are pulling data from the exact same data warehouse table - so, why are the numbers different?

Sales defines "Revenue" as Total Booked Contracts signed this quarter (including deals that haven't paid yet).

Finance defines "Revenue" as Recognized Revenue based on cash received and services rendered.

In this scenario, the organization

has a Single Source of Truth

(one table holding all transaction records)

lacks a Single Version of Truth

(no shared semantic agreement on how to calculate "Revenue").

SSoT

Single Source of Truth (SSOT)

is a data storage principle where every data element is mastered (or edited) in only one place within an organization's information architecture

This means:

- Data is stored exactly once in a centralized and authoritative platform

- Any linkages to this data element in other systems are by reference only

- Updates to the data in the primary location propagate throughout the entire system

- It prevents data duplication and redundancy across system

is not a specific technology, tool, or strategy

rather a state where all organizational data is accessible through a single reference point

SVoT

Single Version of Truth (SVOT)

refers to the practice of delivering clear and accurate data to decision-makers in a form that everyone in the organization agrees is the real, trusted information

Key aspects include:

- It's about presenting the correct data to decision-makers, even if it comes from multiple sources

- It represents one view of data that everyone in a company agrees is the real, trusted number for operating data

- is more focused on the business perspective and decision-making process

- It's the ideal of having either a single centralized database or a distributed synchronized database that stores all organizational data in a consistent form

SSoT / SVoT

Single Source of Truth (SSoT)

SSoT refers to a data storage principle to always source a particular piece of information from one place.

is the practice of structuring information models and associated schemata, such that every data element is stored exactly once.

from a business perspective, at an organisational level, it means that data is only created at source, in the relevant master system, following a specific process or set of processes.

enables greater data transparency, relevant storage system, traceability, clear ownership, cost effective re-use, etc.

Single Version of Truth (SVoT)

SVoT refers to one view [of data] that everyone in a company agrees is the real, trusted number for some operating data.

is the practice of delivering clear and accurate data to decision-makers in the form of answers to highly strategic questions.

effective decision-making assumes that accurate and verified data, serving a clear and controlled purpose, and that everyone is trusting and recognising against that purpose.

enables greater data accuracy, uniqueness, timeliness, alignment, etc.

SSoT / SVoT

SSoT is typically realized in the Staging Area, the Operational Data Store (ODS), or the Raw/Normalized Enterprise Data Warehouse (EDW) layer (such as a 3NF relational model or a Data Vault architecture).

Driven heavily by ETL/ELT pipelines and Master Data Management.

If System A says a customer's phone number is X and System B says it is Y, the SSOT infrastructure (via MDM) resolves this conflict so the DW holds exactly one authoritative record.

SVoT is realized in the Semantic Layer, Data Marts, OLAP Cubes, or Centralized Metrics Stores

e.g., dbt semantic layers, Looker LookML

Driven by metadata management, data governance, and standardized business transformation rules.

It takes the raw data from the SSOT and applies uniform calculations to it.

Master Data Management MDM

In any enterprise, data is constantly generated across multiple departments, applications, and geographic locations.

This fragmented landscape inevitably leads to data silos, inconsistencies, and duplication.

MDM is a comprehensive approach to managing an organization's critical data across the enterprise. MDM uses technology, tools and processes to create a unified master data service that consolidates key enterprise data assets such as customer information, product details and location data.

Tim Mucci, IBM

Core business entities of an organization include:

Master Data

The core nouns of a business. These are non-transactional, persistent entities that provide context to business operations (e.g., *Customers, Products, Suppliers, Employees, Locations*).

Transactional Data

The verbs of a business. These are time-bound events or actions associated with master data (e.g., *Invoices, Sales Orders, Log entries*).

Reference Data

Static or semi-static lookup values used to classify other data (e.g., *Currency codes, Country codes, Order status types*).

MDM

Before implementing an MDM system, organizations must operate under several fundamental assumptions regarding data behaviour and enterprise architecture:

Data Silos are Inevitable

Modern enterprises run on Best-of-Breed applications (e.g., Salesforce for CRM, SAP for ERP,) and each is optimized for its specific function and captures entity data differently.

MDM assumes that these silos will always exist, and trying to force all departments into a single monolithic software system is impractical.

The Multiplicity of Identity

Without MDM, the same real-world entity exists in multiple systems under different formats. For example: Customer in an ERP system might be recorded as "IBM, Inc." in the CRM, and "IBM" in a support ticketing platform.

MDM assumes that identity resolution requires sophisticated matching algorithms, not just simple unique key joins.

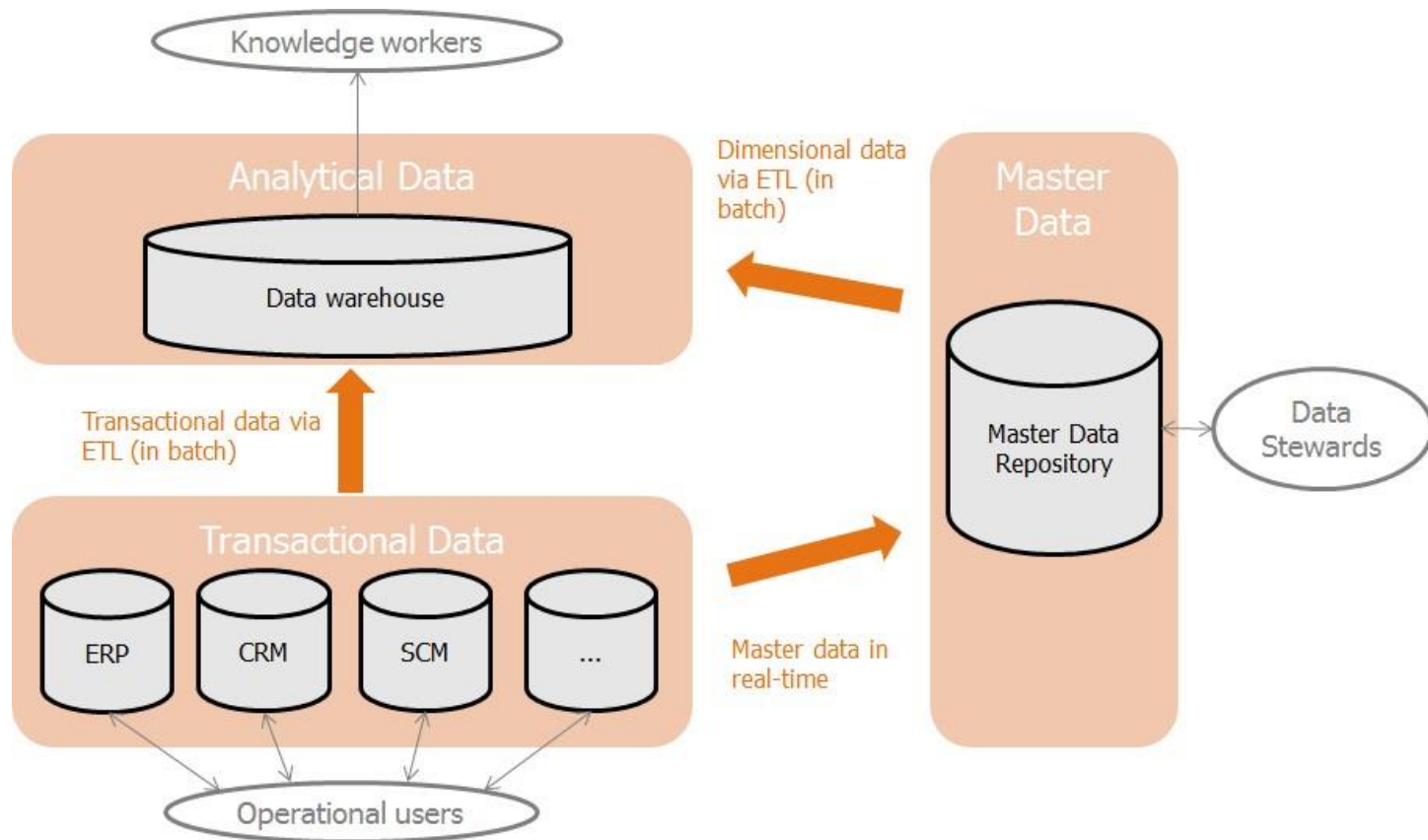
Data Decay and Degradation

Data naturally degrades over time. People move, companies merge, and product specifications change. MDM assumes data is dynamic and requires continuous stewardship, data profiling, and validation rules to maintain its integrity.

Governance is Foundational

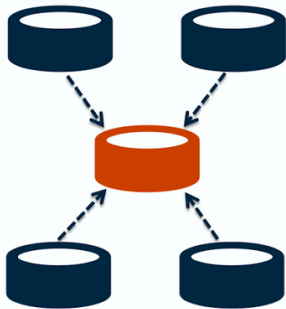
An MDM system is only as good as the human processes governing it. The core assumption of MDM is that technology alone cannot fix data quality; it must be coupled with Data Stewardship (assigning accountability for data quality) and strict governance workflows.

MDM

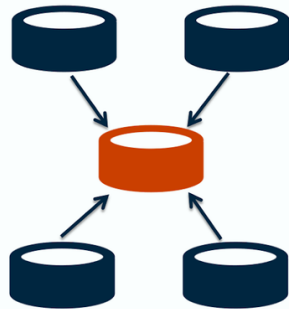


MASTER DATA ARCHITECTURE PATTERNS

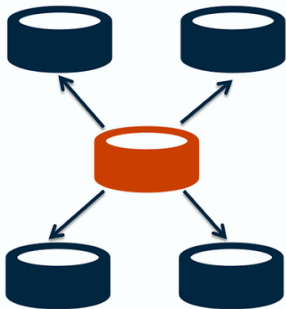
Registry architecture



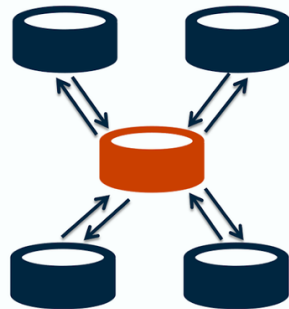
Consolidation architecture



Centralized architecture



Coexistence architecture



MDM

Architectural patterns:

Registry Architecture

In a Registry model, the MDM hub does not store the master data attributes. Instead, it maintains a central index of cross-reference keys and identifiers.

Consolidation Architecture

The Consolidation model pulls data from various source systems into a centralized hub where it is cleansed, de-duplicated, and matched to create the Golden Record.

Coexistence Architecture

The Coexistence model builds on the Consolidation pattern but adds a feedback loop.

Centralized (Repository) Architecture

In this fully centralized model, the MDM hub becomes the only place where master data is created, updated, and stored.

MDM

In addition to architectural styles, MDM solutions are also categorized based on what they are used for in an organization:

Operational MDM

focuses on providing a consolidated view of core data assets to employees who handle data as part of their day-to-day operations.

ensures that data is consistent and reliable across departments for efficient operational processes, such as customer service or order fulfillment.

Analytical MDM

designed to support business intelligence (BI) and data analytics by consolidating and standardizing master data

helps organizations generate insights from accurate, consistent master data, which is crucial for strategic decision-making, reporting, and performance analysis.

MDM

In a mature data ecosystem, MDM serves as a critical pre-processor or companion to the Data Warehouse.

Driving Conformed Dimensions

By feeding the DW with master data tables (e.g., Dim_Customer, Dim_Product), the DW can reliably join diverse fact tables (e.g., Fact_Sales and Fact_Support_Tickets) without encountering join failures or mismatched entity rows.

Enhancing ETL/ELT Pipelines

Without MDM, data engineers must build complex, brittle deduplication and string-matching logic directly inside the ETL/ELT code of the Data Warehouse.

When MDM is introduced upstream, it handles data cleansing, survivorship rules, and deduplication. The ETL pipeline to the warehouse becomes a clean, straightforward ingestion process, reducing data pipeline failures and compute costs.

Resolving Historical Discrepancies

MDM system tracks dimensional changes using master data lineage and relationship hierarchies.

When this managed data is pushed to the DW, it allows for easier handling of SCDs.

MDM

Master Data Management and **Metadata Management** are distinct disciplines – yet are deeply intertwined.

Master Data represents the core nouns of your business

e.g., who bought the product

while **Metadata** is the documentation that explains what that data means, where it came from, and how it is structured

e.g., what the "Customer_ID" field represents.

A Data Warehouse cannot achieve an **SVoT** without first establishing an **SSoT**, and Master Data Management (MDM) is the bridge between them.

MDM creates the SSOT

MDM deduplicates systems, master entities (like Customer or Product), and ensures that a single row in the DW represents a single real-world entity. This satisfies the physical requirement of an SSOT.

Data Governance transforms SSOT into SVOT

Once MDM delivers clean, consolidated master data to the DW, the Data Governance council establishes the metadata rules. They define precisely how that master data is allowed to be chopped, skewed, and aggregated for reporting.



Data warehouse

The general idea of a data warehouse is as follows:

- build a new dedicated database

 - update-driven approach*

- load it with appropriately selected data from various sources

 - data must be previously selected (extraction) and converted to the target warehouse format (transformation)*

 - extraction, transformation and loading processes, i.e. ETL processes, are performed cyclically.

 - usually the data already loaded into the warehouse does not change and is stored for a long time.

- supports multidimensional analysis (OLAP)

 - and data mining*

STEPS INVOLVED IN OLAP

Most OLAP workflows involve four primary steps:

Data extraction:

The first step is to extract the relevant data from original sources, including transactional databases and cloud applications.

Once extracted, the data is typically loaded into a unified storage system.

Data preparation:

To ensure data quality, the extracted data must be transformed, tested, and documented.

includes integrating, cleaning, de-duplicating, restructuring, filtering, aggregating, and joining as needed.

Building the OLAP cubes:

After the relevant data has been prepared, if necessary, data engineers then create OLAP cubes for data analysis.

includes modeling the OLAP cubes before deploying them to a dedicated server.

Accessing data:

Now, the data is ready for analytics and BI tools, and users can access it through dashboards, query the data, and execute data analysis.

Data warehouse system



Inmon / Kimball

There has been much debate on the issue of DW architecture:

Bill Inmon and Ralph Kimball, who are pioneers in the field of DW, are among the debaters

Inmon

argues in favour of a hub-and-spoke architecture

Corporate Information Factory



Inmon

Kimball

argues for the data mart bus architecture with conformed dimensions



Kimball

Inmon / Kimball

A data warehouse is a subject-oriented, integrated, time-variant, and non-volatile collection of data in support of management's decision-making process

W.H. Inmon, 1992

A data warehouse is a database with these particular features

A copy of transaction data specifically structured for query and analysis. (..) A data warehouse is a system that extracts data from source systems, transforms and loads to multidimensional structures, and further supports queries and reporting for decision support.

Kimball

A data warehouse is an architecture, the entire process focused on these particular tasks

Requires multiple tools and separate approaches

Inmon / Kimball

Definition for a data warehouse:

These are not opposing definitions

Kimball's data warehouse should follow all characteristics of Inmon's definition

As such Inmon defines a data warehouse as a subset of Kimball's data warehouses

None of the definitions tackles the shape of the database

Focusing on features and functional aspects

As such there are many different designs and models of data warehouses

Inmon / Kimball

Inmon's **Top-Down** Approach

A centralized data warehouse acts as an enterprise-wide data warehouse from which data marts are built as per the requirements of the specific departments

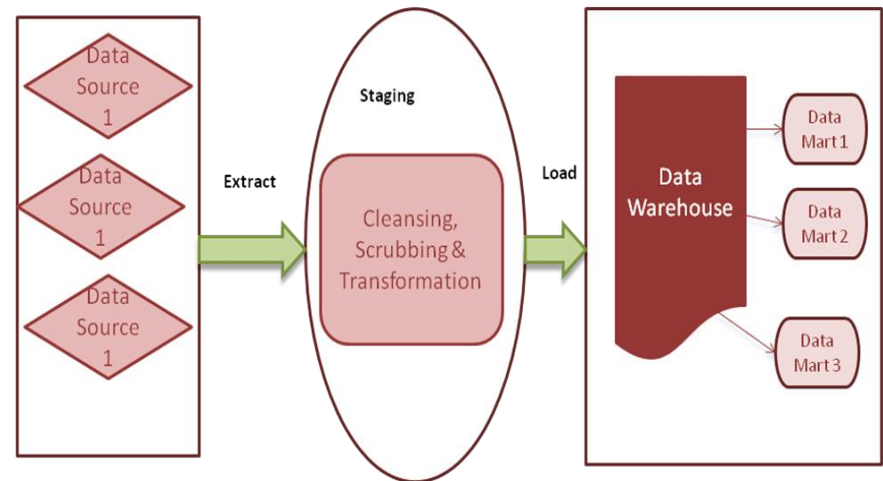
Persistent dimensional views of data across data marts can be viewed since all data marts are loaded from a data warehouse

This data warehouse design is efficient against all business changes.

Creation of a data mart from a data warehouse is very simple

The analytic systems can access data in a data warehouse via the data marts

Inmon



Inmon / Kimball

Kimball's **Bottom-Up** Approach

A business process is built using data marts which are joined together using common dimensions

A dimensional data model with facts and dimensions is implemented here

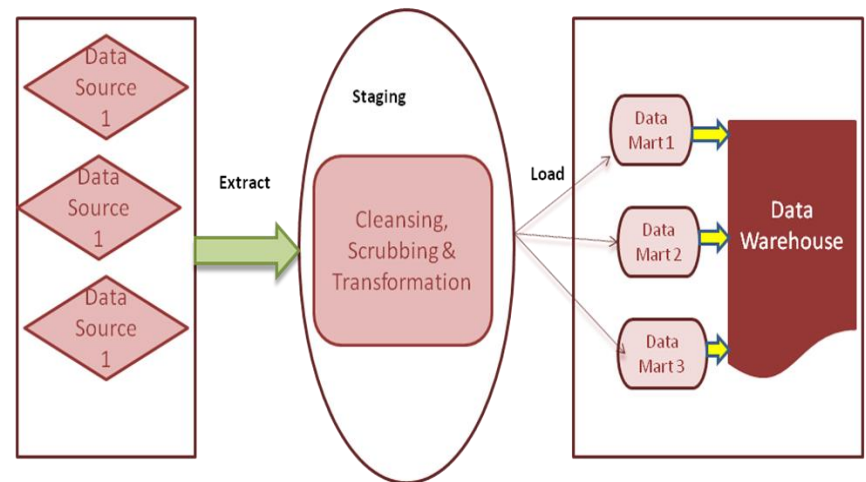
The reports can be generated quickly since the data marts are built first

The data warehouse can be easily expandable to accommodate new units.

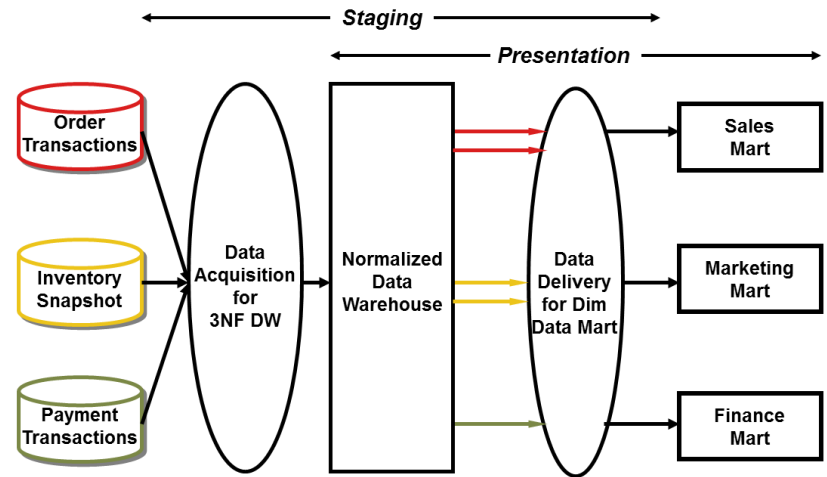
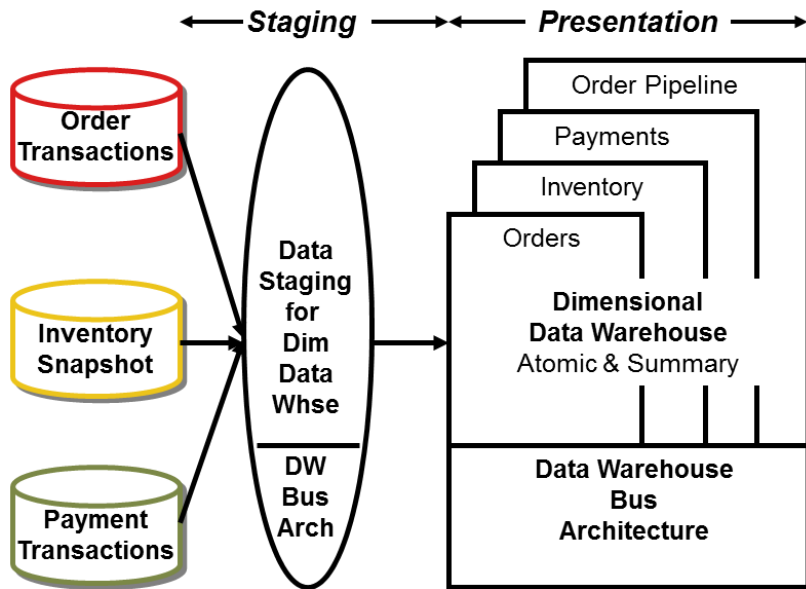
It involves the creation of new data marts and then integrating with the other data marts

The analytic systems access data via data marts

Kimball



Inmon / Kimball



Inmon / Kimball

Inmon

Data warehouse begins with the corporate data model

key subject areas and key entities the business operates with

detailed logical model

Normalized conceptual model

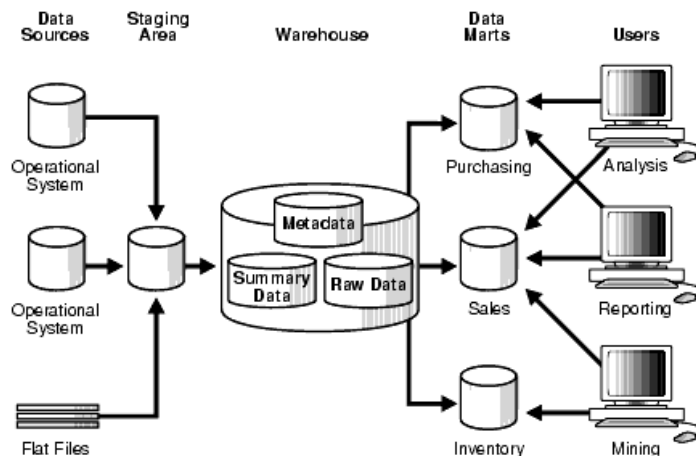
data redundancy is avoided as much as possible

leads to clear identification of business concepts and avoids data update anomalies

Physical model

implementation of the data warehouse is also normalized

single version of truth for the enterprise is managed here



Inmon / Kimball

Inmon

Normalized model makes loading of the data less complex

but using this structure for querying is hard as it involves many tables and joins

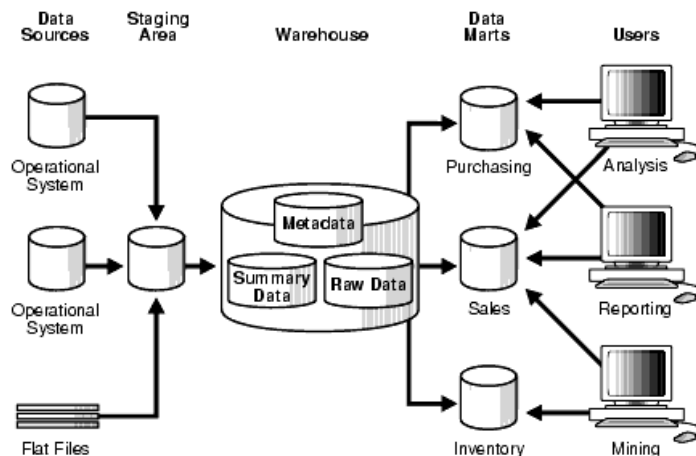
Suggested building data marts specific for departments.

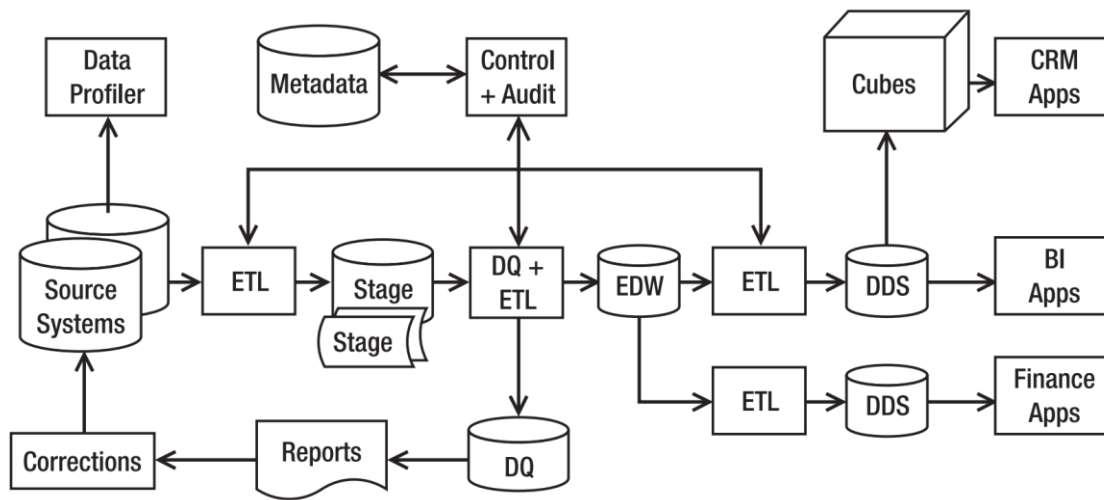
The data marts will be designed specifically for LOB

The data marts can have de-normalized data to help with reporting.

Any data that comes into the data warehouse is integrated, and the data warehouse is the only source of data for the different data marts.

This ensures that the integrity and consistency of data is kept intact across the organization.





Data warehouse system

Inmon

Inmon / Kimball

Key advantages of the Inmon approach:

The data warehouse truly serves as the single source of truth for the enterprise,

Is the only source for the data marts and all the data in the data warehouse is integrated.

Data update anomalies are avoided because of very low redundancy.

This makes ETL process easier and less prone to failure.

The business processes can be understood easily, as the logical model represents the detailed business entities.

More flexible

As the business requirements change or source data changes, it is easy to update the data warehouse as one thing is in only one place.

Can handle varied reporting needs across the enterprise.

The disadvantages of Inmon method:

The model and implementation can become complex over time as it involves more tables and joins.

Need resources who are experts in data modelling and of the business itself.

These type of resources can be hard to find and are often expensive.

A fairly large team of specialists need to be around to successfully manage the environment

The initial set-up and delivery will take more time, and management needs to be aware of this.

More ETL work is needed as the data marts are built from the data warehouse.

Inmon / Kimball

Kimball

Building the data warehouse starts with identifying

the key business processes

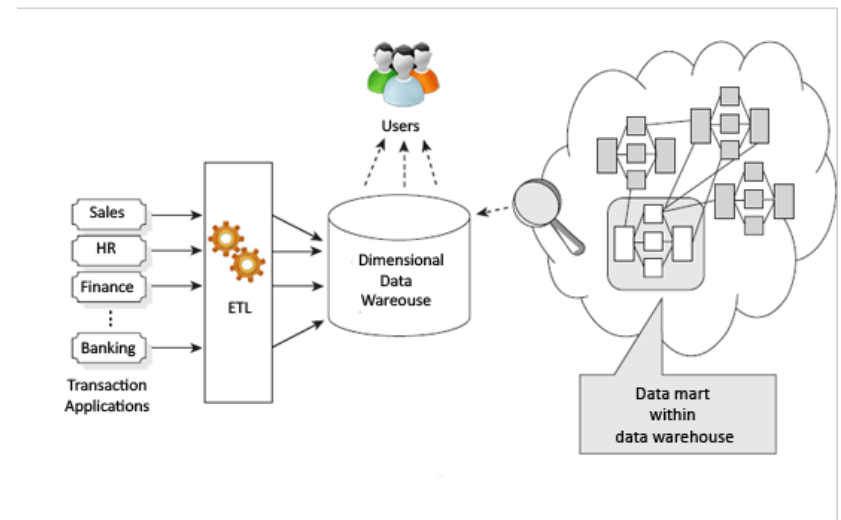
the key business questions

The model

the dimensional model

not normalized

Multiple star schemas are built to satisfy different reporting requirements



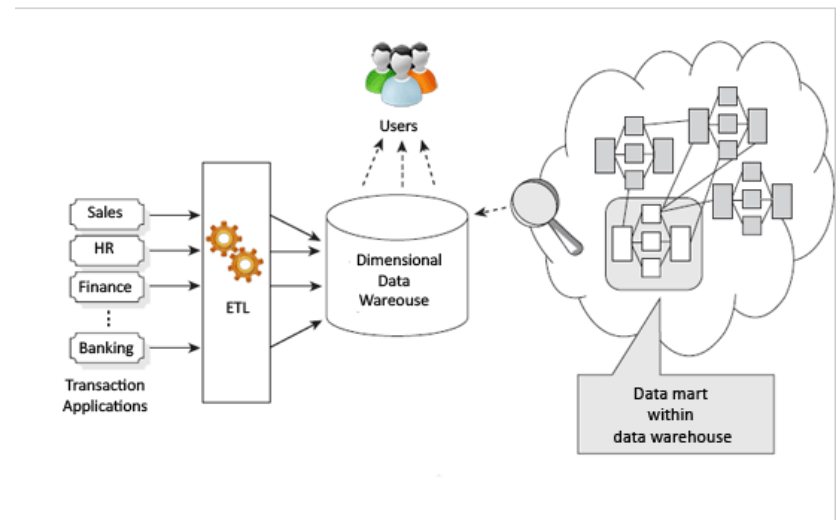
Inmon / Kimball

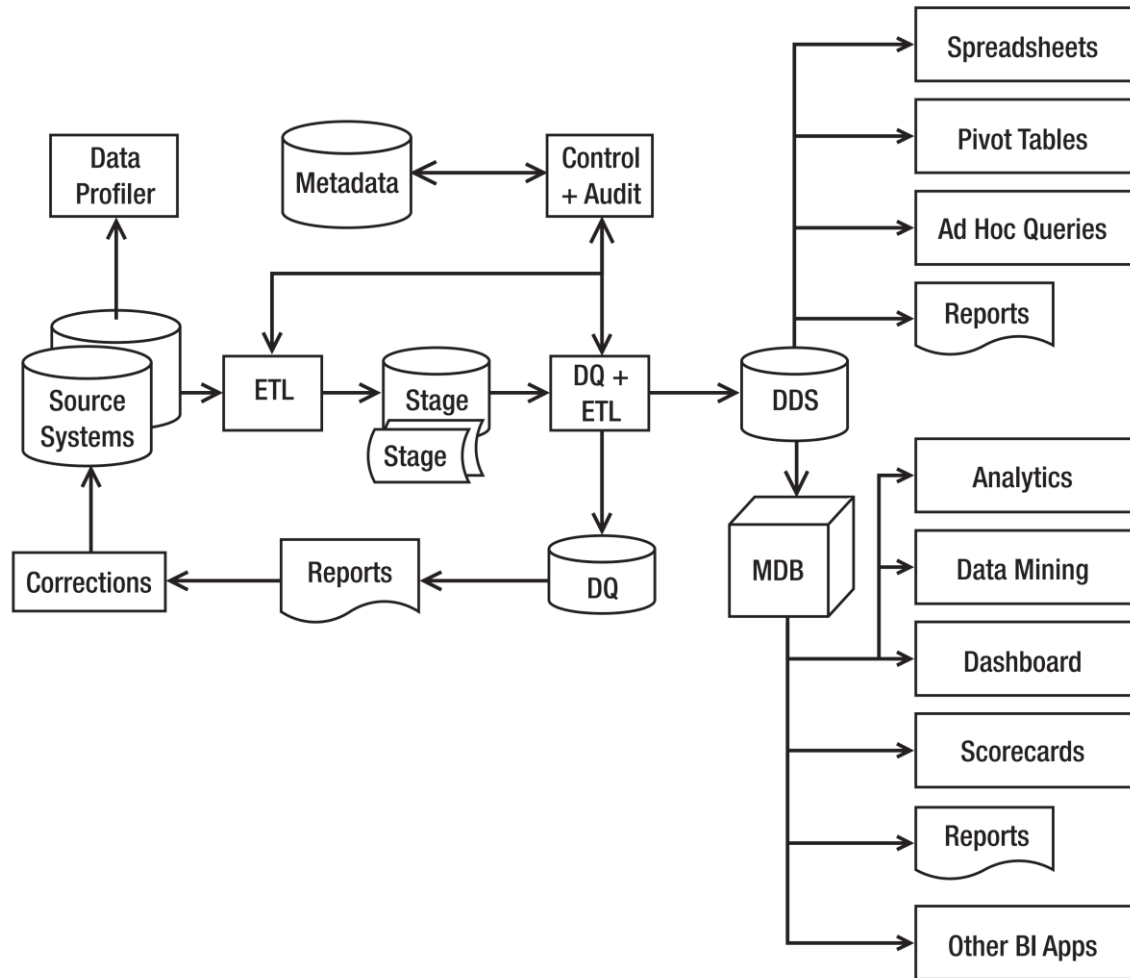
Kimball

Kimball proposes the concept of 'conformed dimensions'

Data integration

key dimensions, like customer and product, that are shared across the different facts will be built once and be used by all the facts





Data warehouse system

Kimball

Inmon / Kimball

Key advantages of the Kimball method:

- Quick to set-up and build – first phase of the data warehousing project will be delivered quickly.

- The star schema can be easily understood by the business users and is easy to use for reporting – most BI tools work well with star schema.

- The performance of the star schema model is very good.

- A small team of developers and architects is enough to keep the data warehouse performing effectively

- Works really well for department-wise metrics and KPI tracking

 - data marts are geared towards department-wise or business process-wise reporting.*

- Drill-across

 - Using multiple star schemas to generate a report using conformed dimensions*

Disadvantages of the Kimball method:

- The essence of the 'one source of truth' is lost

 - data is not fully integrated before serving reporting needs*

 - Redundant data can cause data update anomalies over time*

- Adding columns to the fact table can cause performance issues.

 - fact tables are designed to be very deep*

 - the dimensional model is hard to change as the business requirements change*

- Cannot handle all the enterprise reporting needs because the model is oriented towards business processes rather than the enterprise as a whole

The conformed dimensions approach

Tied disparate data marts into a logical “single version of the truth” without the need for a DW

ETL layer staged all the data from the source systems and then directly parsed the data to the data marts.

Very appealing, but there were crucial stumbling blocks to implementing this approach:

Only a small percentage of companies had the discipline to design and implement the required data integration processes.

Inconsistency and data quality needed overly complex real-time DI that was beyond the ETL tools' capabilities at that time.

In the real world, the timing of transactional data and the updates of its associated reference data were often out of sync, preventing the on-the-fly validation and integration of data from multiple source system to multiple data marts.

Inmon / Kimball

Two prevailing views on data warehousing:

Kimball, in 1997, stated that

"...the data warehouse is nothing more than the union of all the data marts",

Kimball indicates a bottom-up data warehousing methodology

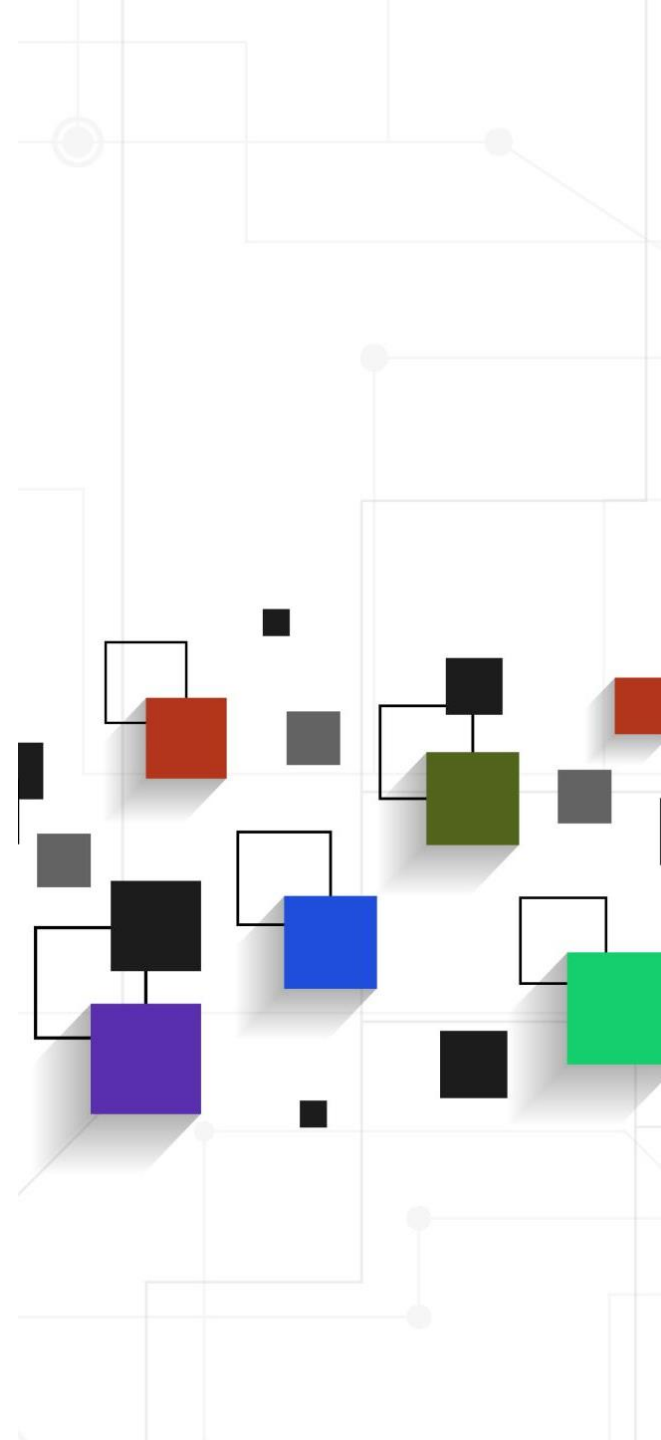
in which individual data marts providing thin views into the organizational data could be created and later combined into a larger all-encompassing data warehouse.

Inmon responded in 1998 by saying,

"You can catch all the minnows in the ocean and stack them together and they still do not make a whale,"

This indicates the opposing view that the data warehouse should be designed from the top-down to include all corporate data.

In this methodology, data marts are created only after the complete data warehouse has been created.



Inmon / Kimball

Even later:

In 2014 Inmon criticized Cloudera for associating Big Data with the data warehouse, two totally unrelated terms according to him

*“Turbocharge Your Porsche - Buy An Elephant”,
Bill Inmon*

Whereas Kimball took the opposing view by presenting a webinar with Cloudera about building a data warehouse with Hadoop

Inmon / Kimball

Kimball's approach of data marts seems to be more popular since companies prefer to start with something small that works rather than spec endlessly only to create a monster.

Sometimes there is a data warehouse in place.

It's usually implemented by a relational database which is queried directly and used for online analytical processing (OLAP).

Although Inmon argues that a data warehouse is just an architecture

people use the term on a day-to-day basis to refer to an actual technology



DW Components

DW Components

Enterprise warehouse:

collects all the information

all the subjects spanning the entire organization

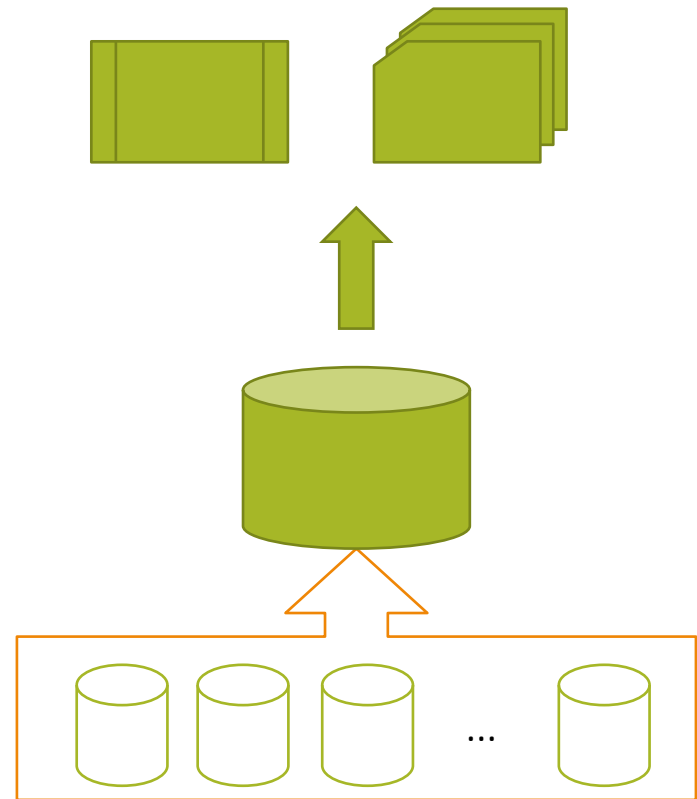
provides the enterprise-wide data integration

integrated from operational systems and external information providers

cross-functional in scope

typically contains detailed data as well as summarized data

can vary in size



DW Components

Data Mart:

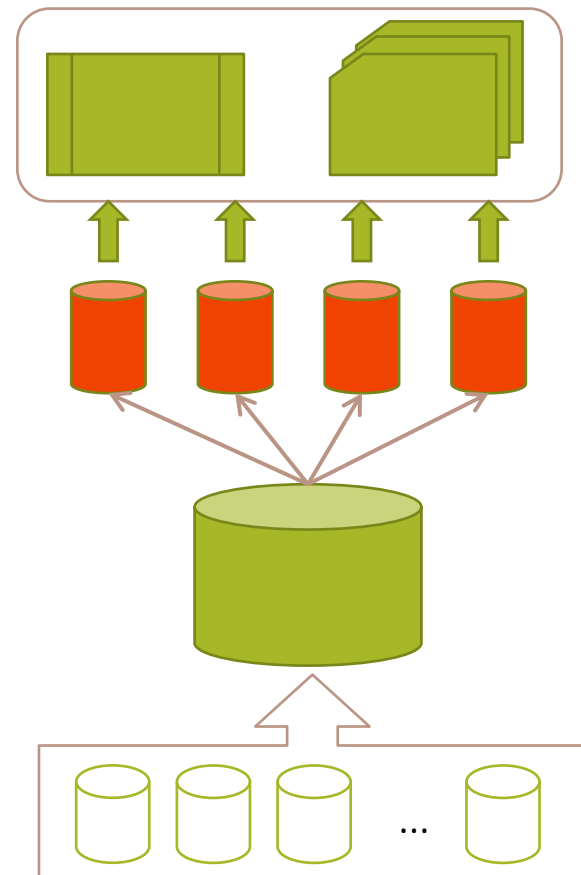
Specialized

*a single-line of business
warehouses*

*in its most simplistic form
represents data from a single
business process*

Departmental subsets that focus
on selected subjects

*Eg. Marketing data mart: customer,
product, sales*



DW Components - DW / DM

Data Warehouse	Data Mart
SCOPE	
Application independent	Specific DSS application
Centralised	Decentralised
DATA	
Historical, detailed, summarized	Some history, detailed, summarized
Lighty denormalized	Highly denormalized
SUBJECTS	
Multiple	One central single-subject, LOB
SOURCES	
Many internal & external	Few internal & external
OTHER	
Flexible, Data-oriented, Long life	Restrictive, Project oriented, Short life
100GB to more than 1TB	less than 100GB
Implementation - months to years	Implementation - Months

DW Models

A data mart (DM)

is a simple form of a data warehouse that is focused on a single subject (or functional area),

this subset of data is valuable to specific group of an organization

such as Location, Sales, Finance, or Marketing

single LOB – line of business

given their single-subject focus, data marts usually draw data from only a few sources

the sources could be internal operational systems, a central data warehouse, or external data

The implementation cycle of data mart is measured in short period of time

i.e. in weeks rather than months or years.

The life cycle of a data mart may be complex in long run

planning and design is organisation-wide

Data mart is small in size and more flexible

data warehouse, unlike a data mart, deals with more complex and general data

Data in data marts can be highly customized according to the requirements of the users

in the EDW as it houses for the data of the entire organization

DW Models

Why use data marts:

single data warehouse may be too complex and inefficient

functional areas have specialized needs

data marts are used by a little group of selected users and meet their specific needs

e.g. one department

needs for data in multiple formats

process separation

reconciliation for data quality (→DW)

transformation for end users (→DM)

DW Models

Why use data marts:

The following are the reasons to consider using data mart:

To partition data

in order to impose access control strategies

To speed up

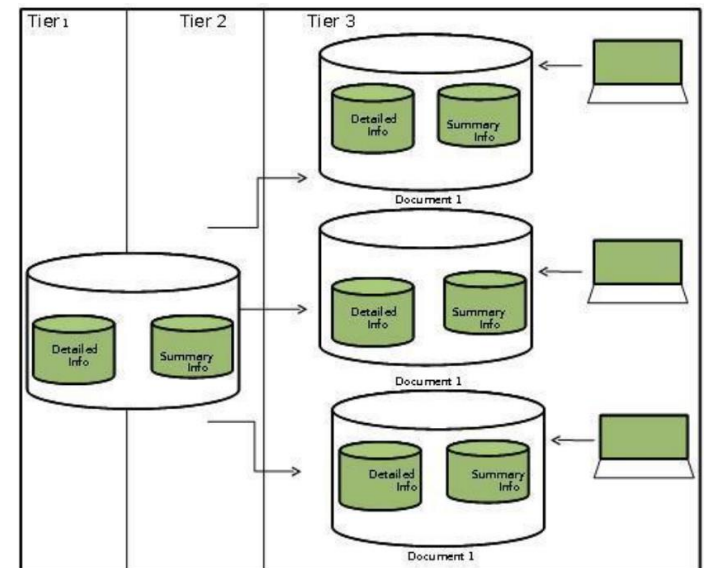
the queries by reducing the volume of data to be scanned

To segment data

into different hardware platforms

To structure data

in a form suitable for a user access tool



DW Models

Data Mart

Depending on the source of data, data marts can be categorized as

Stand-alone data mart (Independent data marts)

- stand alone data

 - from multiple transaction systems in one subject area or department to support specific business needs

- focuses exclusively on one subject area and it is not designed in an enterprise context

 - For example, manufacturing has their data mart, human resources, etc.

- analytic / business intelligence tools query data directly from data mart

Dependent data marts

- are sourced directly from enterprise data warehouses

- analytic / business intelligence tools query data directly from data mart

 - but there is also something else

DW Components

Virtual warehouse

a set of views over operational databases

only some of views may be materialize

user directly accesses the operational data

via middleware tools

suitable for infrequent queries and ad hoc analysis needs

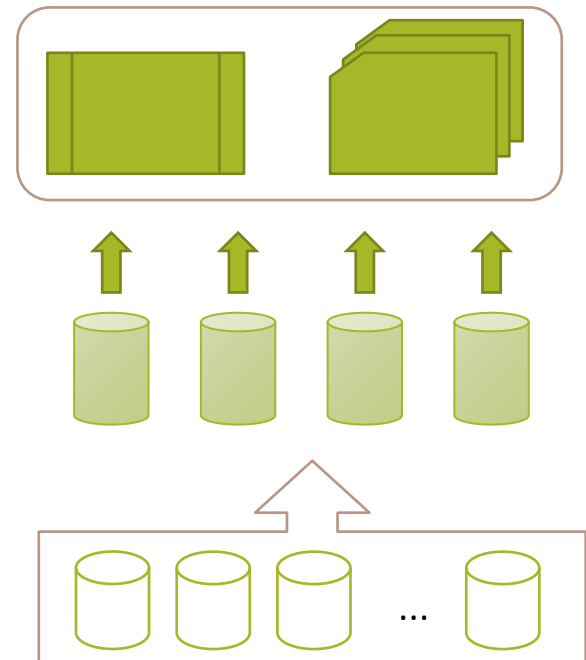
requires excess capacity on operational database servers

is easy to build

it does not provide a real data warehouse solution

since it does not contain historical data, does not contain centralized metadata, and does not have the ability to clean and transform the data

virtual data warehouse can severely impact the performance of operational databases.



DW Components

Data Virtualization:

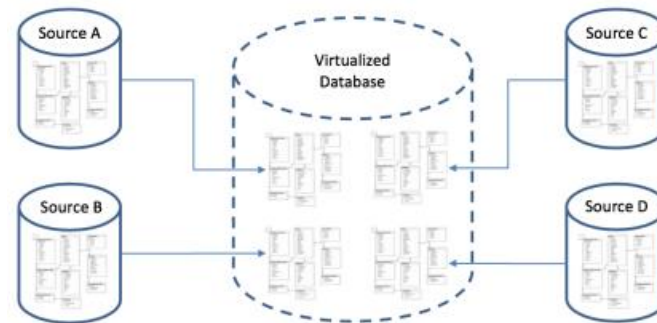
is a comprehensive data integration approach that creates an abstraction layer over diverse data sources, allowing users to access, combine, and transform data without physically moving it

keeps source data where it is, for each source, but makes it look like all the data is in one place

allows users to access that data using one common interface

while it removes a data layer in the environment, it adds a processing layer

decouples source systems from consuming applications while providing a consistent semantic layer meaningful to business users



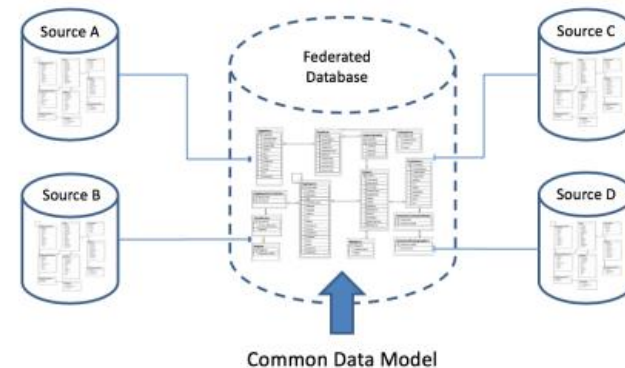
Data Federation:

is a more specific data integration technique that focuses on creating a unified view of data from multiple autonomous data stores

enables two or more databases to appear as one

virtually, without ever actually moving the data

allows underlying data stores to continue operating independently



Data federation is a subset of data virtualization.

Both data virtualization and data federation are usually accomplished using specialized software applications that connect to a variety of different source systems

DW Components

Data sources

Systems which are responsible for data generation and/or storage and handoff

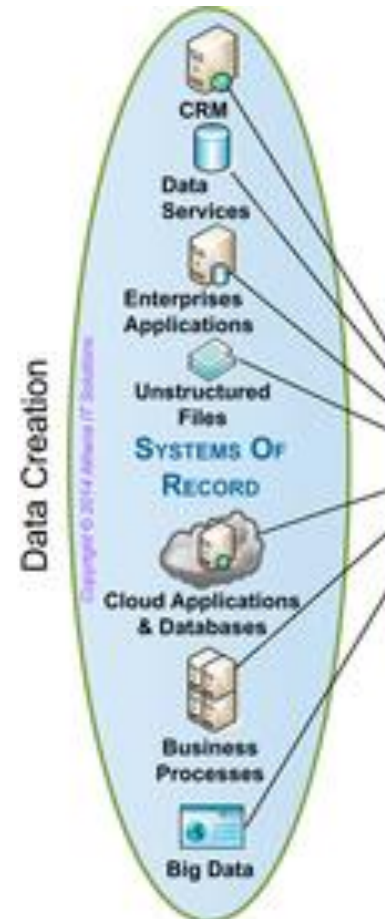
Systems of record (SOR))

data is captured and updated in operational and transactional applications.

applications are designated as the SOR so that people and processes know what the authorized sources are for any particular data subject

implies an expectation level in regards to the integrity and legitimacy of the data.

For example, an application would be designated as the SOR for accounting data.



Operational data store

An operational data store (ODS)

a central database that provides a snapshot of the latest data from multiple transactional systems for operational reporting

enables organizations to combine data in its original format from various sources into a single destination to make it available for business reporting.

ODS is not a data warehouse or data mart.

The purpose of an ODS is to integrate corporate data from different heterogeneous data sources in order to facilitate operational reporting in real-time or near real-time.

Usually data in the ODS will be structured similar to the source systems, although during integration the data can be cleaned, denormalized, and business rules applied to ensure data integrity.

This integration will happen at the lowest granular level and occur quite frequently throughout the day.

An ODS is targeted for the lowest granular queries

Normally an ODS will not be optimized for historical and trend analysis

DW Components

Operational data store (ODS)

The ODS is an optional architecture component and is used for operational reporting purposes

serves two operations:

loading data from transactional systems

running report queries from users

Operational construct used to prepare detailed data between various operational systems

Keep single version of the truth

Helps gather and integrate nonaggregated data from the various source systems before data is loaded into the warehouse

ODS processing involves a lot of detailed data and simultaneous updates

The amount of history stored on the ODS is normally less than that kept on the data warehouse

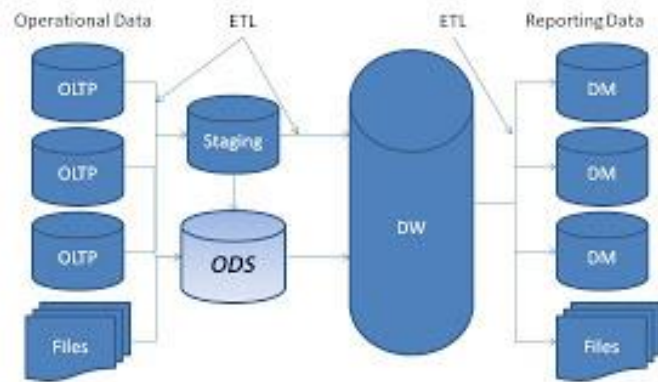
Depending on architecture

Same hardware platform as data warehouse

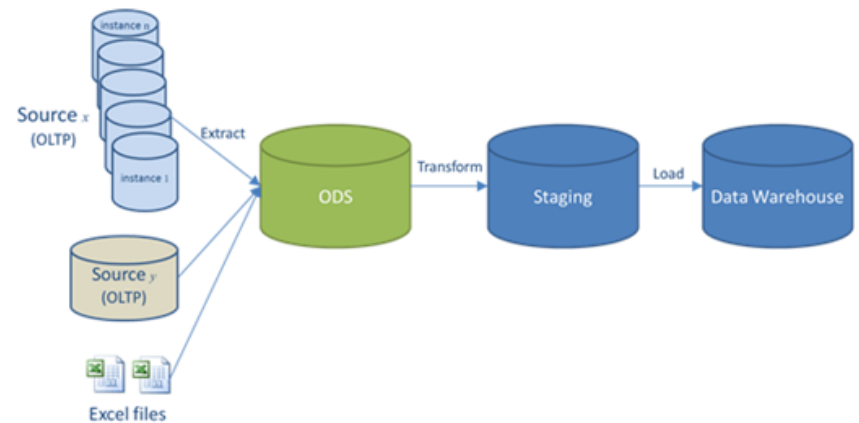
Separate platforms

Operational data store

ODS within Data Warehouse Architecture



<http://randygrenier.blogspot.com/>



<https://www.iri.com/>

Data warehouse

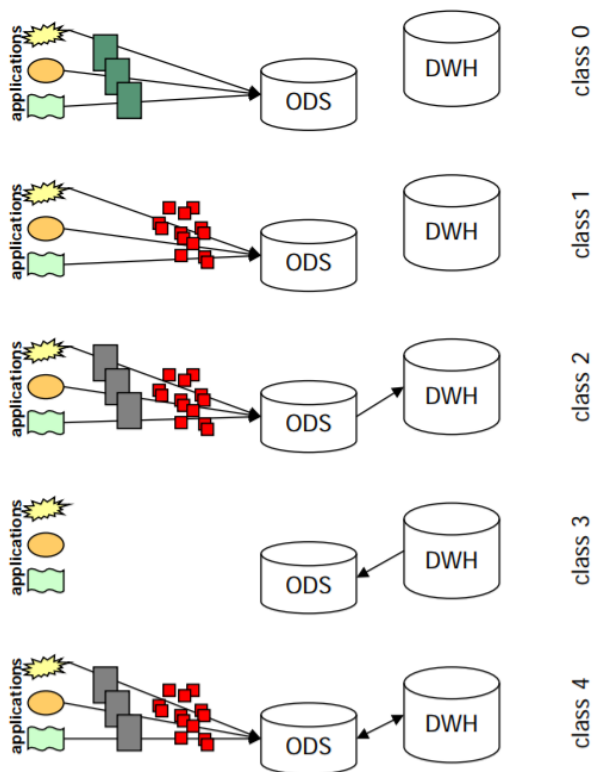
Operational Data (TP)	Operational Data Store (ODS)	Data Warehouse	Data Mart
Ranges from application-oriented to subject oriented	Subject-oriented within solution domain	Subject-oriented	Subject-oriented within application domain
Ranges from non-integrated to integrated	Integrated within solution domain.	Integrated	Integrated within application domain
Current, may have limited history and/or archiving	Current or near current	Time-variant	Time-variant
Volatile, continuously updated	Volatile and updateable	Non-volatile	Non-volatile

Data warehouse

Operational Data (TP)	Operational Data Store (ODS)	Data Warehouse	Data Mart
Ranges from application owned to enterprise managed	Solution focused	focused	Application focused
Optimized for transaction processing	Optimized for OLTP or data access	Optimized for distribution, query, and reporting	Optimized for decision support, analysis
Ranges from application specified to architected	Designed as a local or interim solution	Architected as long-term information asset	Architected as a point solution to a business need
Most commonly driven by a legacy requirements.	Driven by scope of legacy requirements.	Implemented with an evolutionary approach	Rapidly deployed and re-deployed

DW Components

Operational data store (ODS)



- Tables are copied from the operational environment
- Transactions are moved to the ODS in an immediate manner (range of one to two seconds)
- Activities in the operational environment are stored, integrated, and forwarded to the ODS
- ODS is fed aggregated analytical data from the data warehouse
- Combination of integrated data from the operational environment and aggregated data from the analytical environment

DW Components

ODS

The primary objective is to bring data together from multiple source systems on as close to a real-time basis as possible to enable specific business processing or operational reporting.

Approach

data in one location, for ease of access without any extensive DI or transformation.

data to remain as is

current data but generally not maintaining historical data.

application-specific data that supported business processes

MDM

Master data management includes the tools and processes an organization uses to establish a single source of truth for all its critical data.

MDM relies heavily on the principles of data governance with the goal of creating a trusted and authoritative view of a company's data.

Approach

Master data management creates a master record that contains the essential information upon which a business or organization relies.



DW Architectures

Architecture

Architecture is the structure of something.

the term is usually associated with the kind or style of a building, it is used more broadly

for example, the structure of human societies or the solar system.

A reference architecture

is a particular kind of architecture.

helps in understanding or explaining what currently exists or serves as a guide for creating something new

such as a building or a data warehouse

In the context of DW data warehouse reference architectures

identify the alternative ways that data is extracted, transformed, loaded, and stored in a data warehouse

Data warehouse

Before deciding to build a data warehouse you need to ask yourself the following basic and fundamental questions and address the relevant issues:

Top-down or bottom-up approach?

Enterprise-wide or departmental?

Which first—data warehouse or data mart?

Build pilot or go with a full-fledged implementation?

Dependent or independent data marts?

Here are the two different basic approaches:

(1) overall data warehouse feeding dependent data marts

(2) several departmental or local data marts combining into a data warehouse

But before going into details let us look at the past ...

A bit of History

1979:

First Spreadsheet Program—VisiCalc.

1985

Procter and Gamble utilises first commercial system focused on business analytics

Excel 1.0

1988

B. Devlin and P. Murphy publish “An architecture for a business and information system”

First time the term business data warehouse was used

1989

SQL

1990

Inmon publishes “Building the Data Warehouse”

Cognos PowerPlay

A bit of History

1993:

Introduction of the term OLAP

1996

Kimball publishes “The Data Warehouse Toolkit”

1992:

Essbase (Extended Spreadsheet Database) Published by Hyperion Solution this became a major OLAP server product in the market in 1997.

1999:

Microsoft OLAP Services published which became Microsoft Analysis Services in 2000

2002

Inmon updates book and defines architecture for collection of disparate sources into detailed, time variant data store.

Kimball updates book and defines multiple databases called data marts that are organized by business processes, but use enterprise standard data bus.

DW Architectures

Early days Data Warehouse

Years ago it became apparent that there needed to be a database, called an central data warehouse (CDW)

for reporting purposes separate from the data source systems

The mantra in the early days of data warehousing was that data in the EDW was read-only and any data-quality issues needed to be pushed back to the SORs to be fixed.

From a pragmatic perspective, it turned out that the CDW became the preferred location to perform data quality processing

The classic CDW is a single, centralized database. The data workflow includes:

Data being created, updated, and modified in the SORs

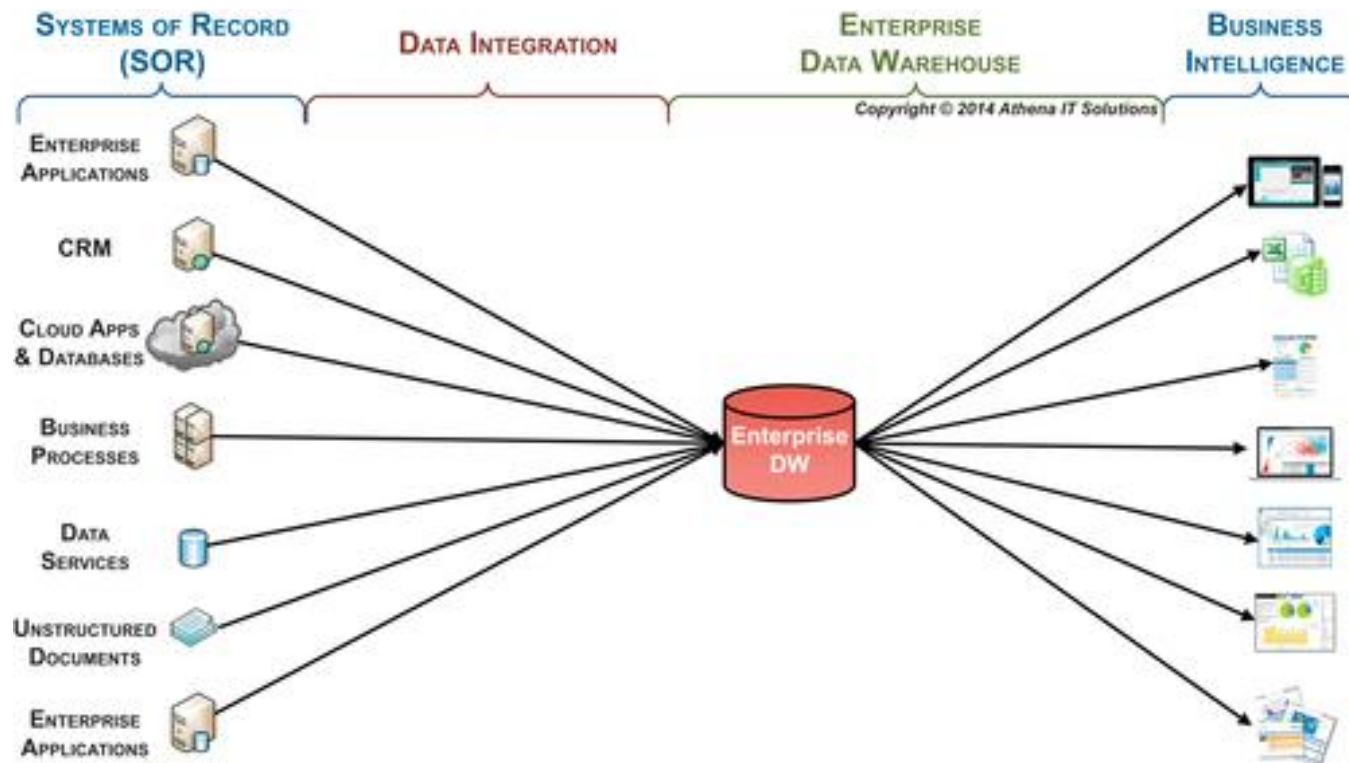
Data from SORs being integrated, transformed, and cleansed

Data being loaded into the CDW

Data being accessed by the BI tools for reporting and analysis

DW Architectures

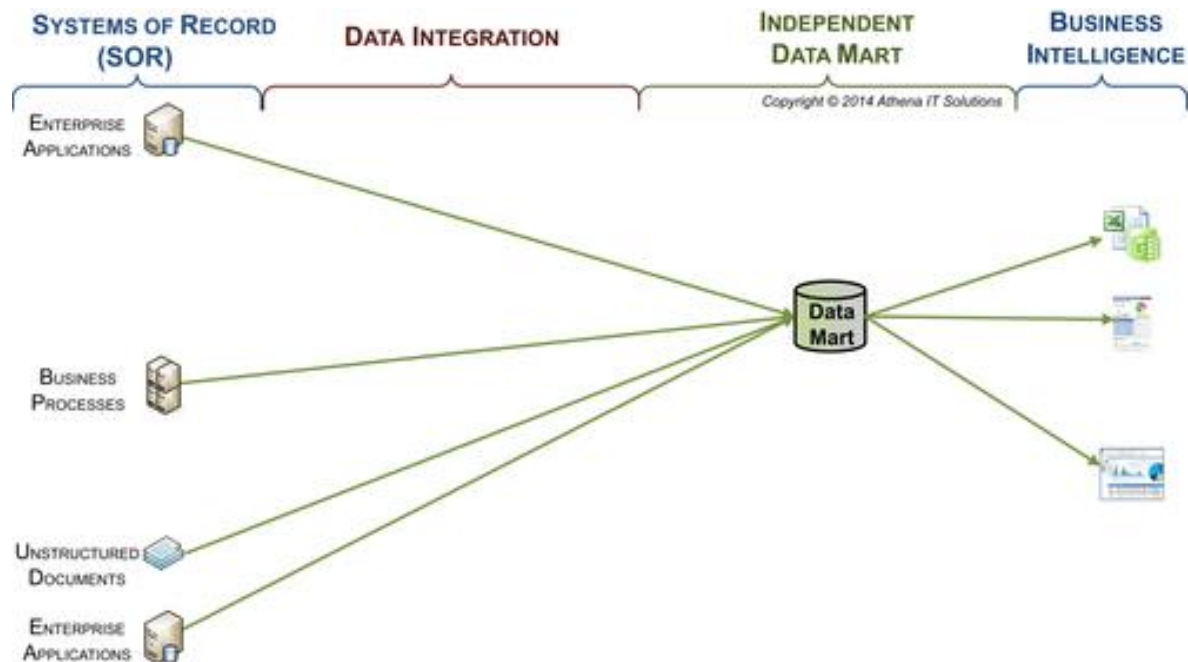
Early days Data Warehouse



The next era was the rise of data marts.

Data marts promised to be quicker and cheaper to build and provided many more benefits—including the benefit of being able to finish building them!

Rather than being part of an overall enterprise data architecture, data marts were the architecture.



Companies sprouted multiple data marts

each built to accommodate only their sponsor with their own data definitions, data transformation, reporting, and technical platform

moved from trying to achieve one version of the truth to building multiple single versions of the truth (?!)

to top it off, many of these projects overpromised and under-delivered, just like the CDW projects

Data marts were perceived as a great success

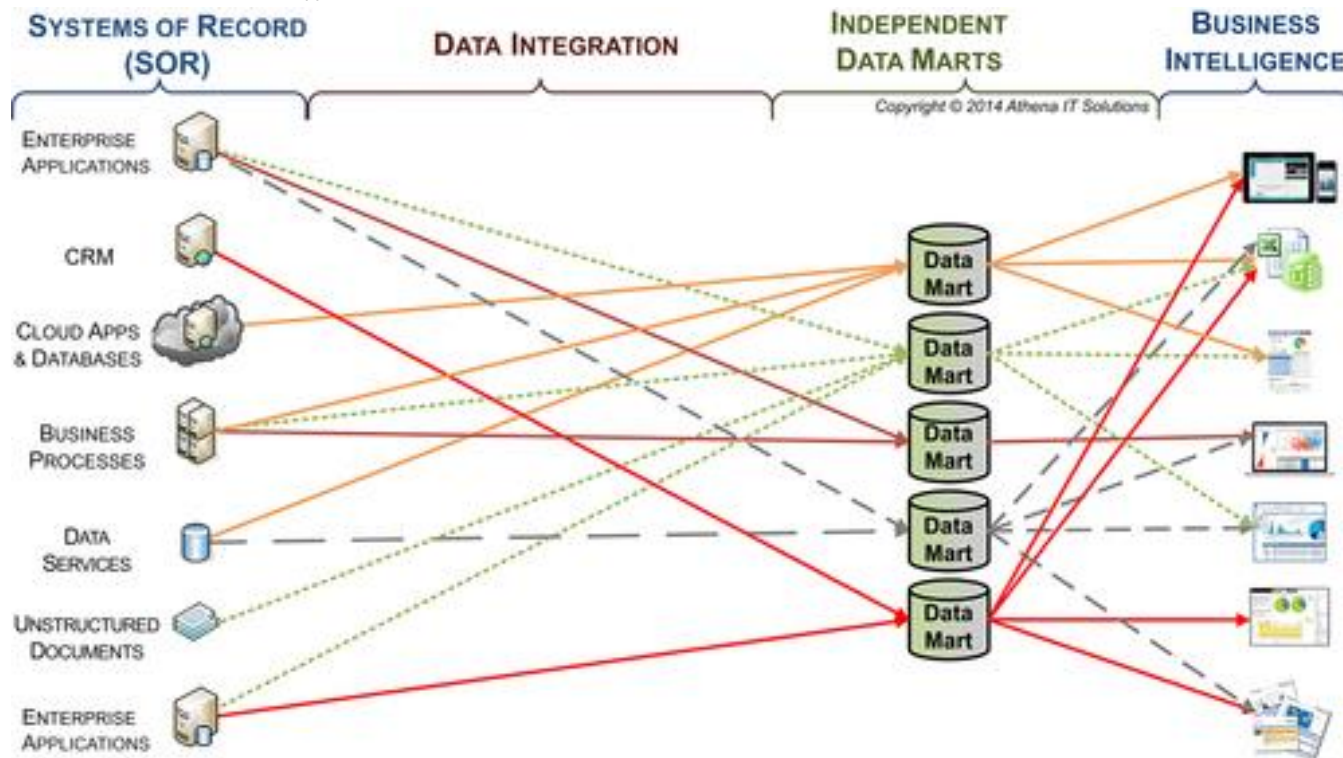
until business groups realized they were debating in meetings which one of their reports (and associated data marts) had the “right” numbers.

Each group had its own data silo.

BYOD approach – bring your own data

Later, this type of data mart would be become known as an independent data mart

The so-called „accidental data architecture”



DW Architectures

CDW and early data marts serve two different data roles:
systems of integration (SOI) and systems of analytics (SOA).

The purpose of each role is as follows:

Systems of Record (SOR)

System of Integration (SOI)

gathers, integrates, and transforms data from SORs into consistent, conformed, comprehensive, clean, and current information.

implies a particular level of integrity and legitimacy of the integrated data.

if integrated data is needed, then the SOI should be the source used.

System of Analytics (SOA)

provides business information that has been integrated and transformed

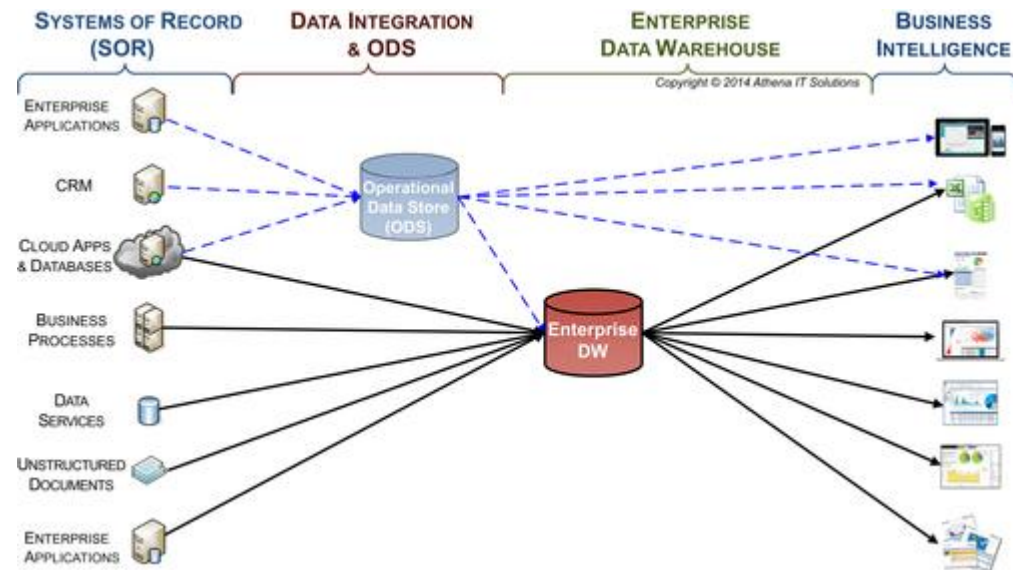
implies a particular level of integrity and legitimacy of the information being used

if integrated and transformed data is needed, the SOA needs to be the source.

A new data architecture component called an ODS emerged, as the SOR to the EDW for the data it extracted from the source systems

EDW would extract data from the source systems that the ODS did not use

Enterprises could get their reports from: ODS only, EDW only, ODS and EDW



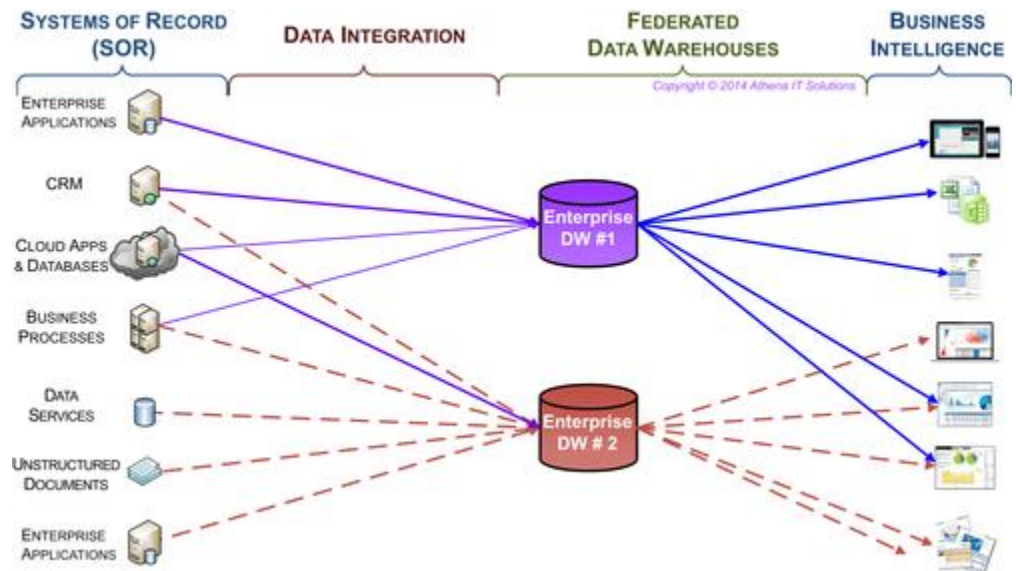
Federated DW is another architecture option that emerged.

split the EDW into multiple physical DWs, using some of the following options to make the divisions:

Geographical regions or countries

Business functions such as finance, sales, marketing, or HR

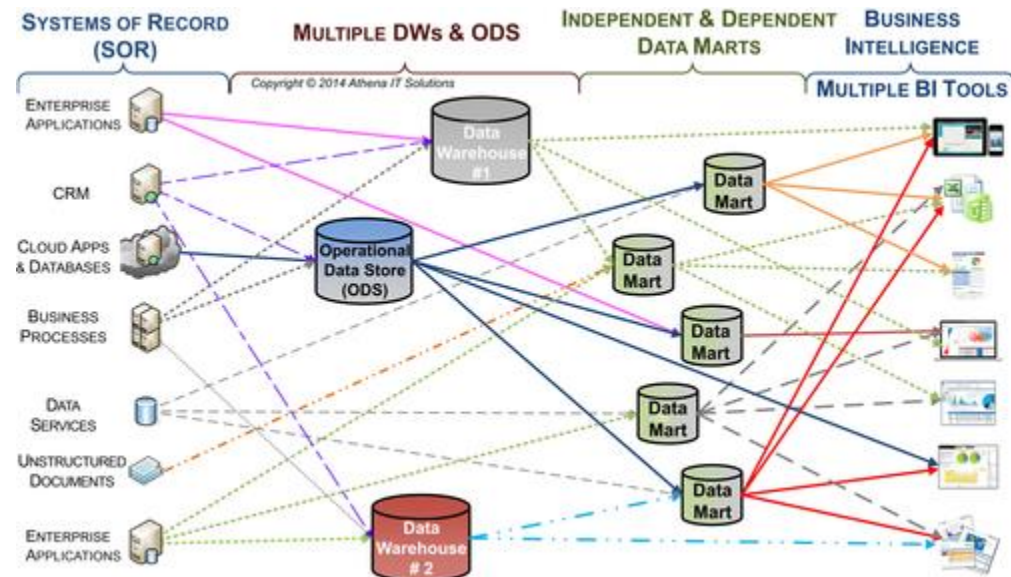
Business entities such as divisions or subsidiaries



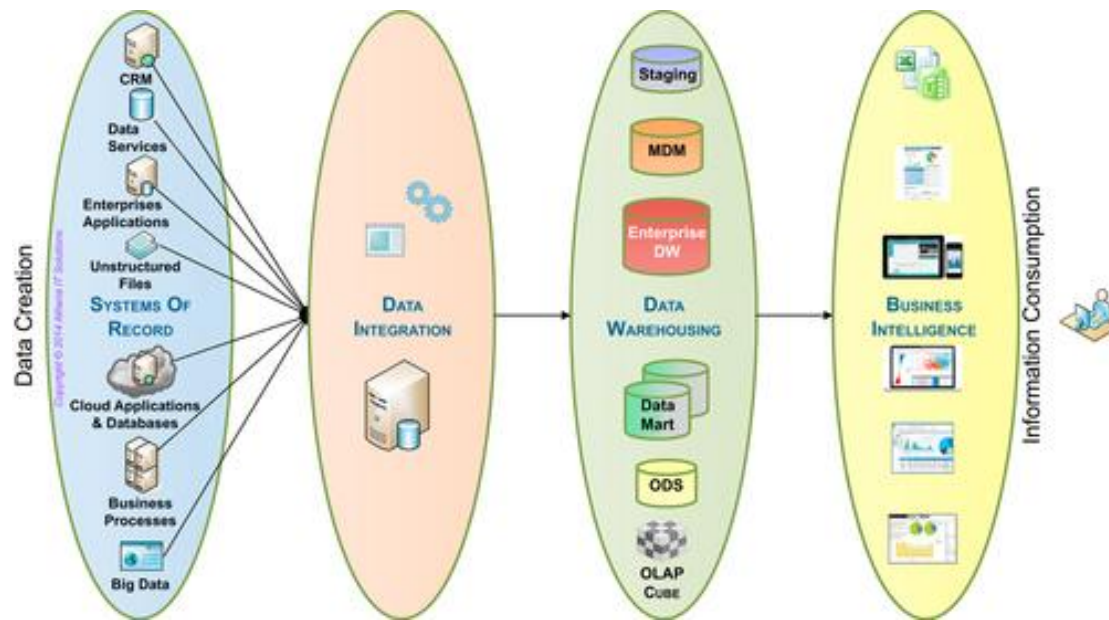
The reaction to the independent data marts was to create DWs or operational data stores and drop them into an architectural pool

it was common for an enterprise to have a patchwork of DWs, ODSs, and data marts along with multiple databases, ETL tools, and BI products

Each of these BI data silos addressed a specific need, but looked at holistically they created their own silo problems.



Advances in technology have prompted the evolution from the classic CDW model to more sophisticated data architecture – Enterprise Data Warehouse



An answer to the data mart problem was needed.

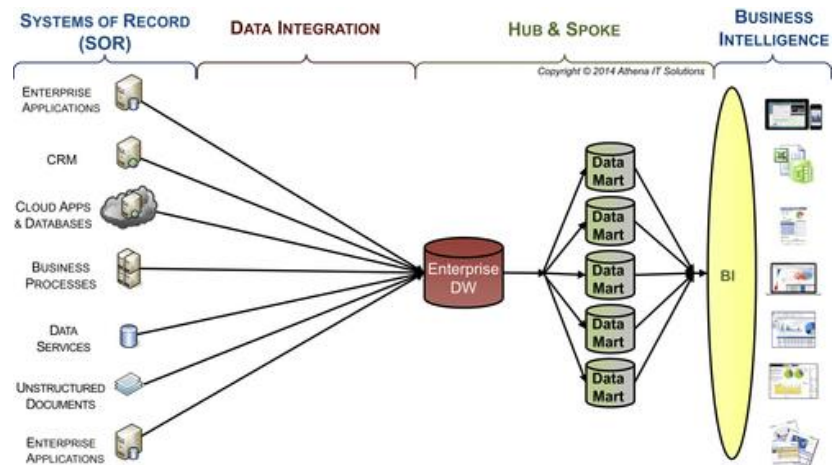
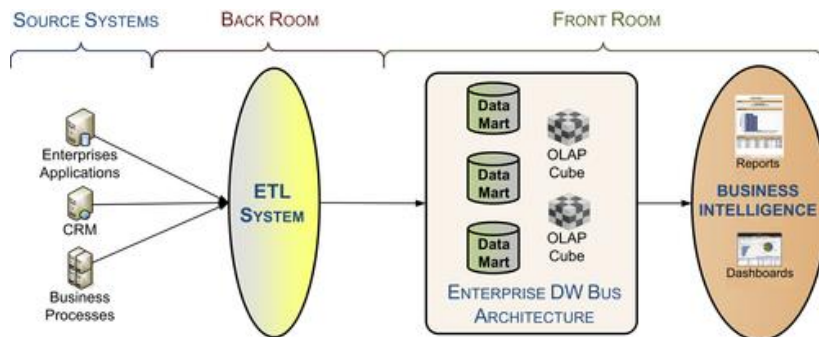
Two competing architectural approaches emerged:

a data bus with conformed data marts

Kimball

a hub-and-spoke with a data warehouse feeding data marts

Inmon



Data Mart Bus

Data mart bus architecture with linked dimensional data marts (DBA)

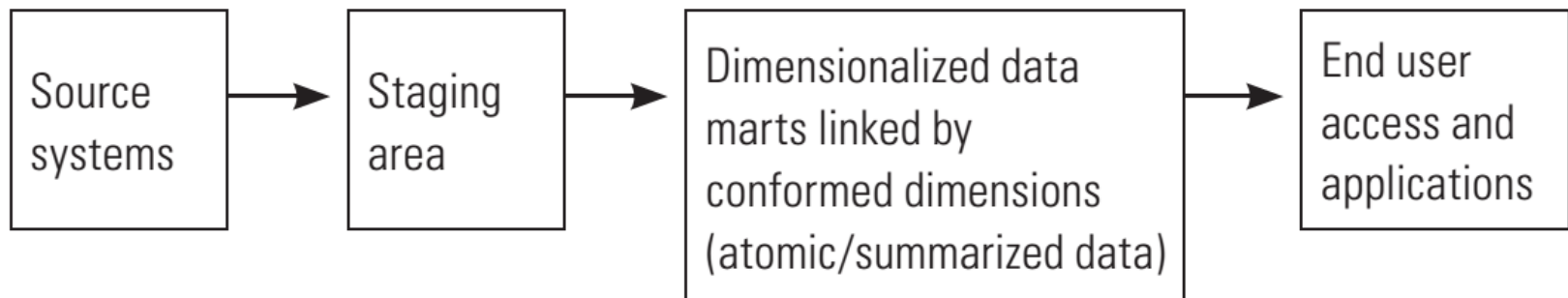
dependent data mart architecture

data marts that support various business processes

the first mart is built for a single business process using dimensions and measures that are used with other marts (i.e., conformed dimensions)

additional marts are developed using these conformed dimensions, which results in logically integrated marts and an enterprise view of the data

Data Mart Bus Architecture with Linked Dimensional Data Marts



Hub-And-Spoke

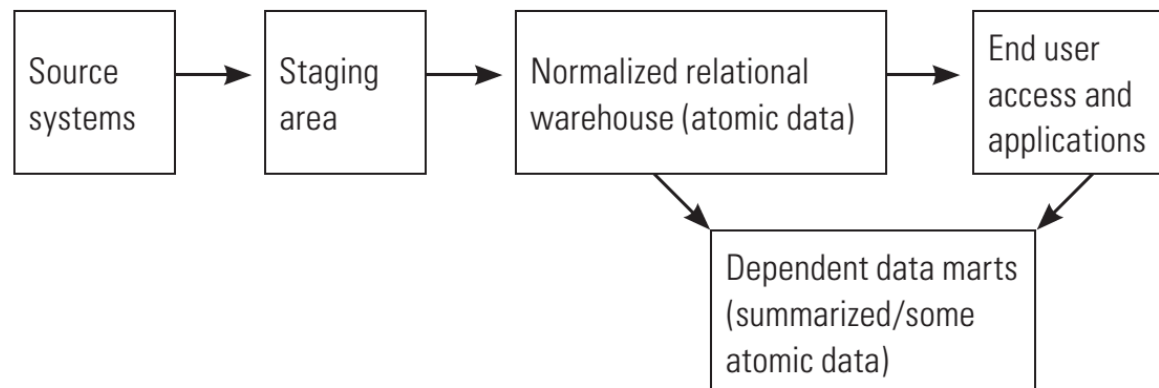
Hub-and-spoke

Developed after an enterprise-level analysis of data requirements

Focused on building a scalable and maintainable infrastructure

Dependent data marts obtain the data from the warehouse

Hub and Spoke Architecture (Corporate Information Factory)



The hub-and-spoke approach became more popular (judging by the number of companies that adopted it)

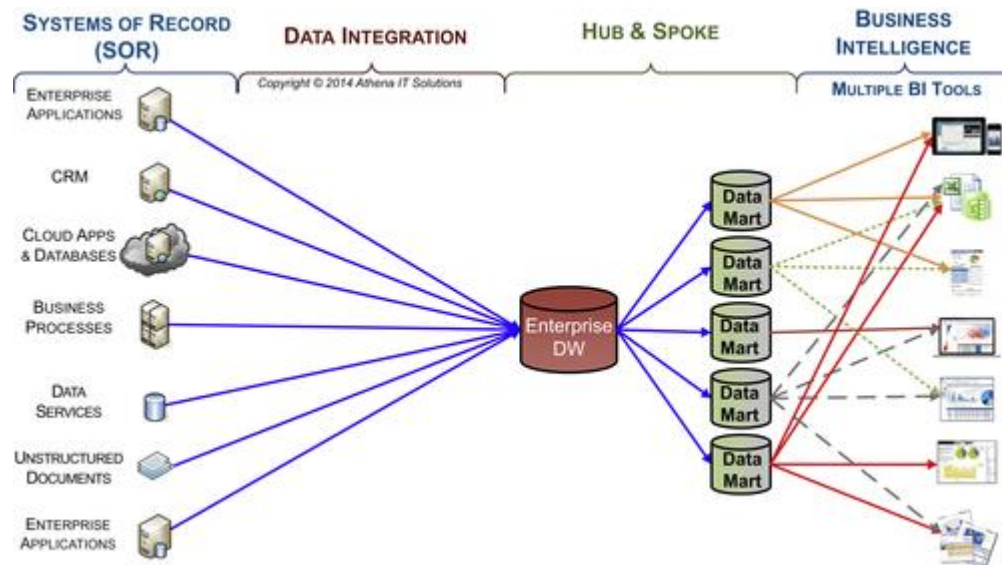
it ultimately became part of the architectural best practices.

The key difference was that the hub-and-spoke built a physical DW (data hub) rather than trying to achieve a virtual hub.

That virtual hub was simple to draw but difficult to achieve in real-world situations.

Hub-and-spoke enables the 5C's of information
(consistent, clean, comprehensive, conformed, and current),
Yet some enterprises delegate BI application development
and tool selection to departmental groups

This typically results in the scenario with multiple BI silos not only of different BI tools, but more damaging redundant and inconsistent reporting.





Data
Vault
Model

Let's start with a simple scenario:

We have a lovely Dim_Customer dimension, containing all the required attributes, properly prepared and verified.

Everything works as it should.

Suddenly, the company acquires a competitor and needs to integrate a new system

A different definition of 'Customer' appears, specifically with different attributes and different relationships.

Kimball's drawback:

We must rebuild the dimension, which means changing the ETL processes and migrating the history.

That's a lot of work.

Data Vault solution:

We model it so that adding a new source does not break what already exists.

Data Vault

As a hybrid architecture

combines the best features of the third normal form and the star schema

results in a set of normalised tables that are history-trackable, detail-oriented and linked in a unique way.

All of this helps to support one or more functional business areas.

Data Vault is a data modelling methodology designed to accommodate change – whether that involves evolving business rules, new source systems or a changing data structure.

Unlike traditional models, which can break when new data or schemas are introduced, Data Vault is built to evolve.

Dan Linstedt published the original Data Vault specification in 2000.

The hub-link-satellite modelling approach solved a real-world problem:

how to integrate multiple source systems without creating fragile star schemas that collapse as soon as a source system changes its primary key structure for the first time?

the modelling structure was robust.

However, Linstedt left the implementation details open to interpretation, and teams filled in the gaps in different ways.

Data Vault 2.0

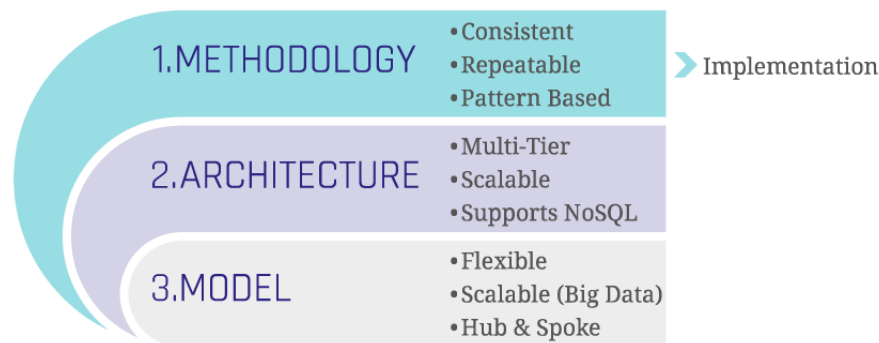
Data Vault 2.0 has standardized aspects that remained undefined in the original specification.

The hub-link-satellite modelling approach remains unchanged.

There are now clear rules for generating hash keys, standardized loading procedures, and architectural patterns.

Data Vault 2.0 bases its specification on three pillars: modelling, architecture and methodology.

The original version relied solely on modelling, leaving users to develop the remaining elements themselves.



Model like Lego bricks (Hub, Link, Satellite)

The modelling pillar defines how data is represented:

Hubs

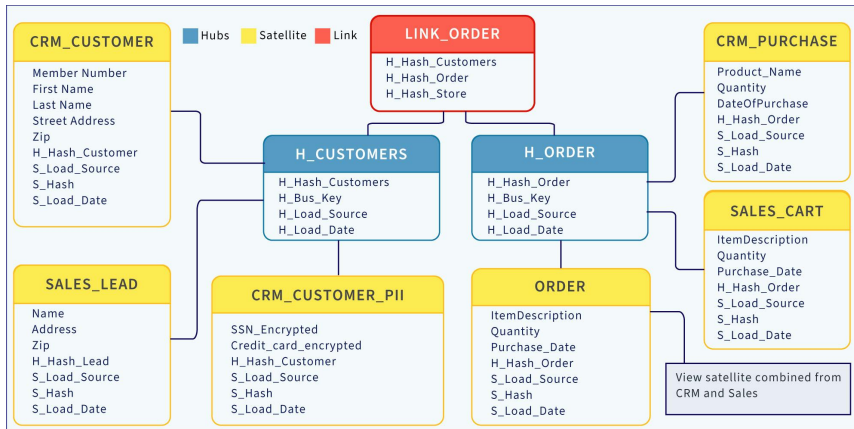
store business keys for entities.

Links

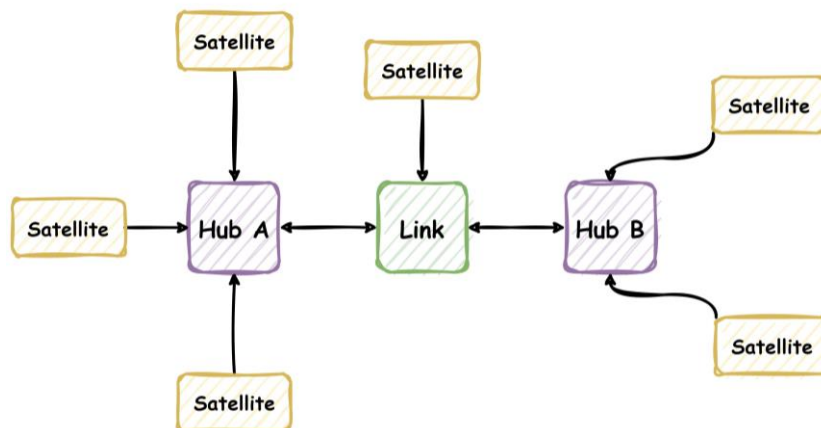
record relationships between business entities.

Satellites

store descriptive attributes and their change history.



<https://www.kyvosinsights.com/>



<https://databytesandinsights.substack.com/p/exploring-data-modeling-architectures>

Model like Lego bricks (Hub, Link, Satellite)

HUB (Node) – "WHO / WHAT?"

Definition:

Contains only a Unique Business Key. No descriptions, no relationships.

Purpose:

This is a hard reference point. Even if you change your name, gender or address, you are still the same entity (e.g. National Insurance number, vehicle VIN).

Example of the HUB_CUSTOMER table:

Hash_Key (MD5 of the customer number)

Customer_No (Business Key from the source system)

Load_Date (When did we first see this?)

Record_Source (Where did this come from? e.g. 'SAP')

Model like Lego bricks (Hub, Link, Satellite)

LINK – "RELATIONSHIP / EVENT"

Definition:

A table linking two or more Hubs.

Purpose:

To separate relationships from entities.

If a relationship changes (e.g. a Customer transfers ownership of a Car to their wife), we add a new entry in the Link table without altering the Hubs.

Example of the LINK_SALE table:

Link_Hash_Key

Customer_Hash_Key (FK to the Customer Hub)

Car_Hash_Key (FK to the Car Hub)

Load_Date

Model like Lego bricks (Hub, Link, Satellite)

SATELLITE – "CONTEXT / DESCRIPTION"

Definition:

This is where (and only where) we store descriptive data (Name, Colour, Price, Address).

A Satellite is always connected to a Hub or a Link.

Purpose:

Tracking changes (SCD Type 2 on steroids).

We can have multiple Satellites for a single Hub!

SAT_CUSTOMER_SAP (data from SAP: Name, VAT number)

SAT_CUSTOMER_CRM (data from Salesforce: Email, Phone)

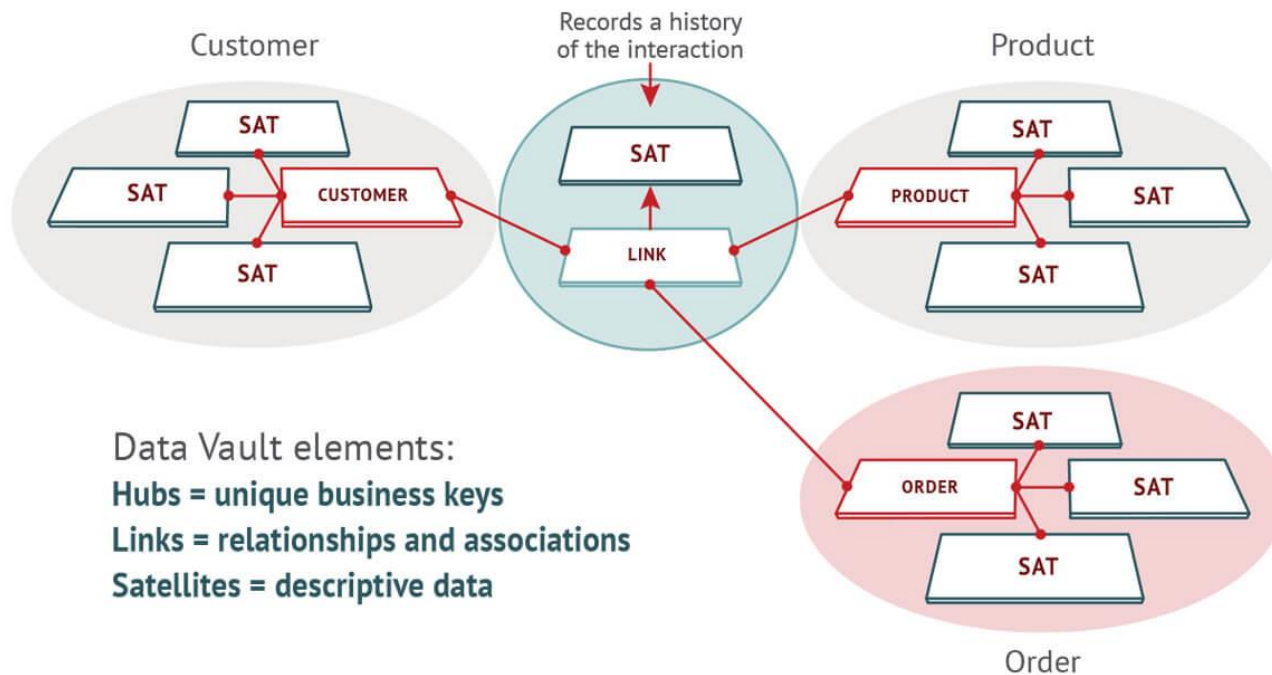
SAT_CUSTOMER_RODO (Marketing consents)

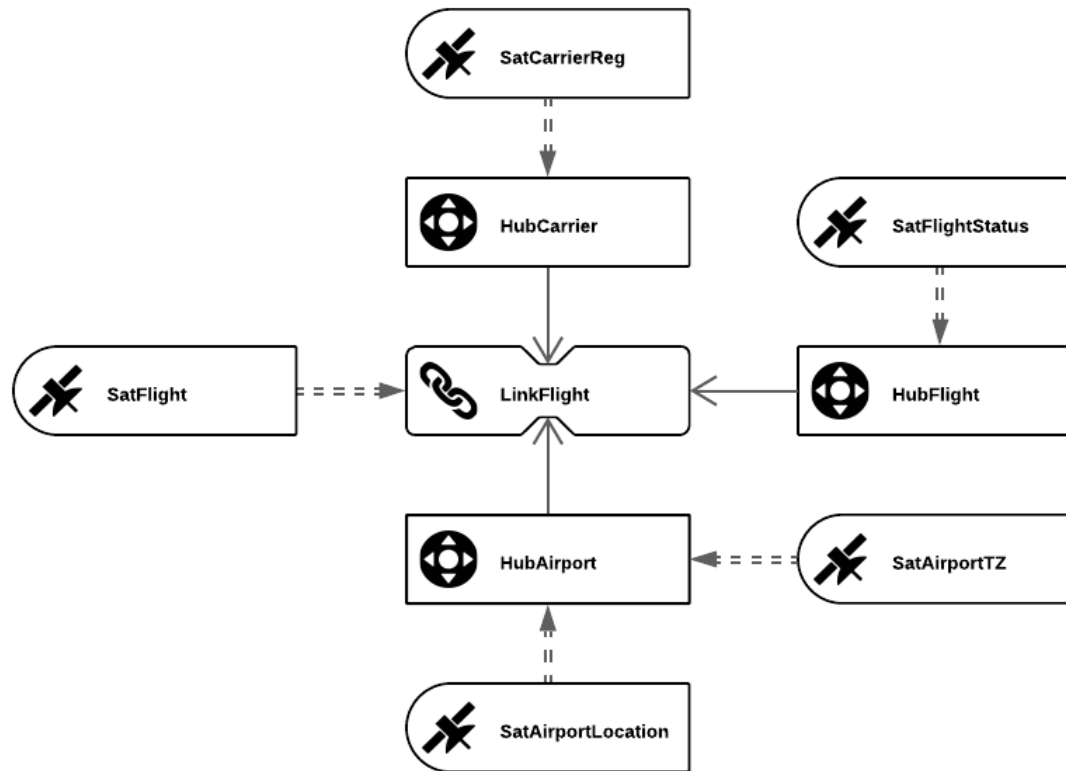
Advantage:

If the CRM system changes, we disconnect the old satellite and connect a new one. The Hub remains unaffected.

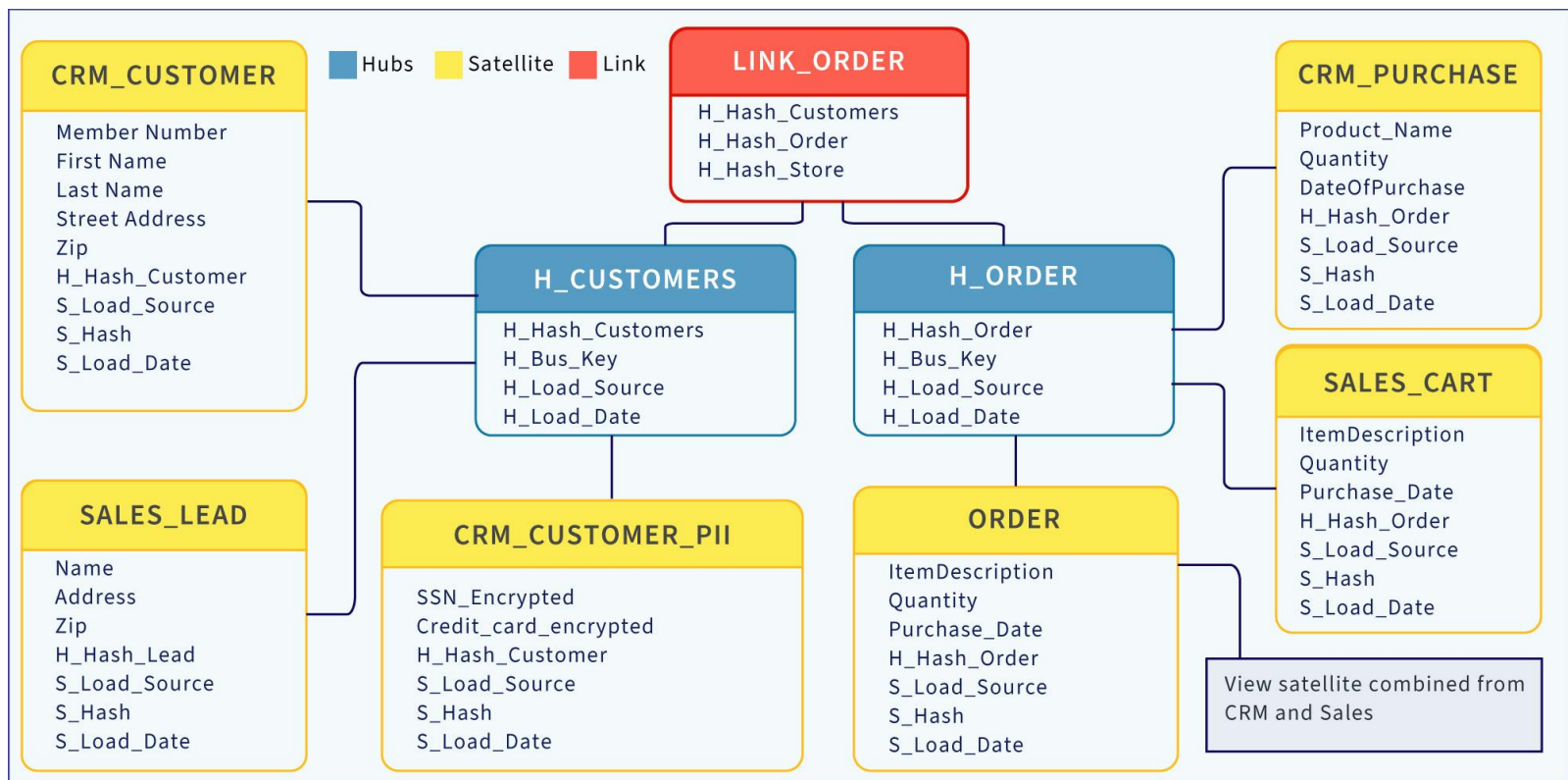
Model like Lego bricks (Hub, Link, Satellite)

Data Vault 2.0 Modeling Methodology





Model like Lego bricks (Hub, Link, Satellite)



Model like Lego bricks (Hub, Link, Satellite)

Data Vault is best suited to situations where flexibility and scalability are a priority, e.g. in organisations operating in highly dynamic environments or processing large volumes of diverse data.

For example, for organisations dealing with large volumes of data from multiple sources, such as social media, IoT sensors, commercial transactions and others.

It resolves a number of common problems faced by these organisations, including:

Difficulties in integrating data from heterogeneous sources.

The need to track and control changes in data over time.

Scalability requirements to cope with the constant growth in data volumes.

Flexibility in adapting the data model to new business requirements.

Model like Lego bricks (Hub, Link, Satellite)

Data Modelling:

This stage consists of 8 steps – we must:

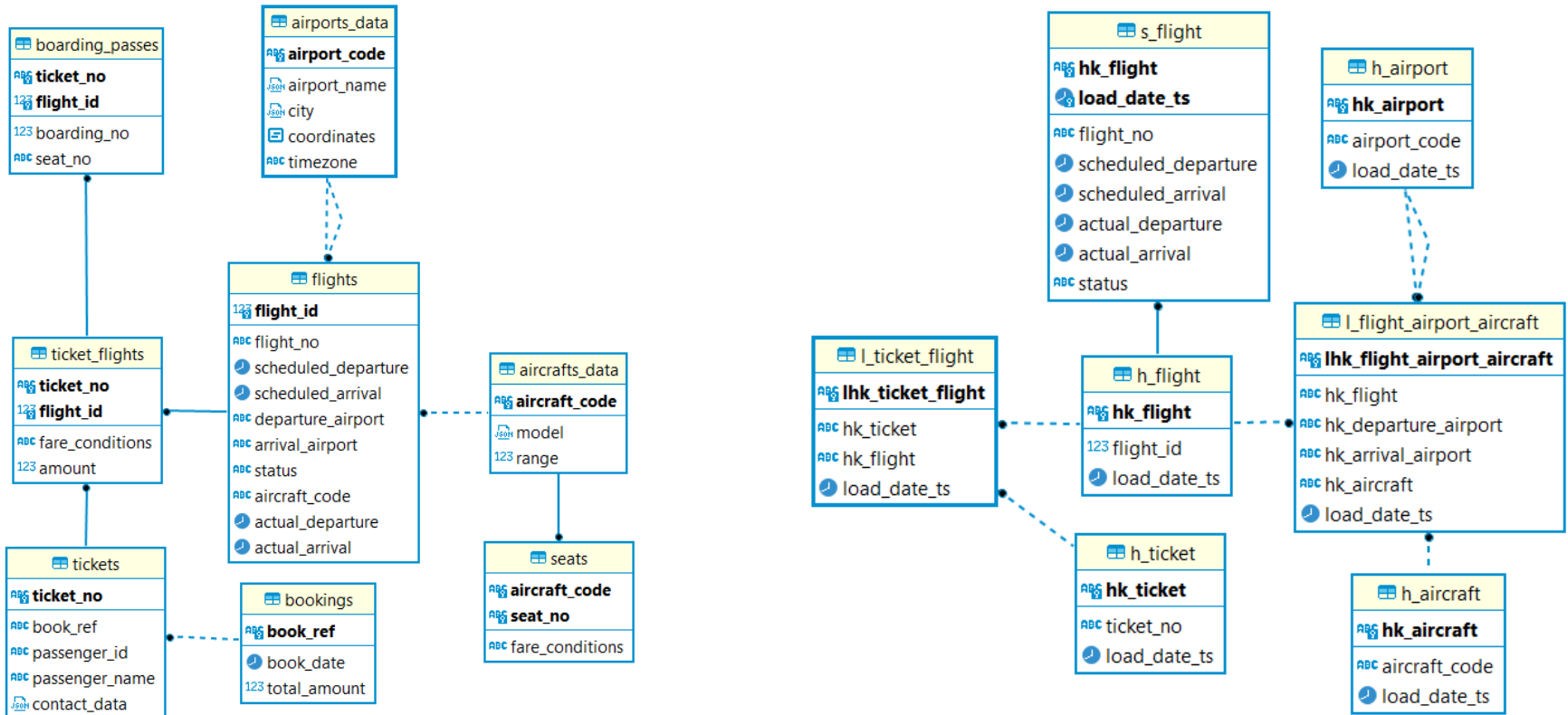
(1) identify the basic business concepts,

(2) identify the nodes, and (3) determine the business keys and (4) hash keys for these nodes.

Next, (5) we model the nodes and (6) identify the relationships between nodes.

Finally, (7) we model the connections and (8) identify the contextual attributes of each centre.

Model like Lego bricks (Hub, Link, Satellite)



Data Vault 2.0 layers – separate raw data from derived data

Raw Vault layer

A 1:1 copy from the source.

This separation ensures control over the raw data store.

No transformations, no business rules, no calculations.

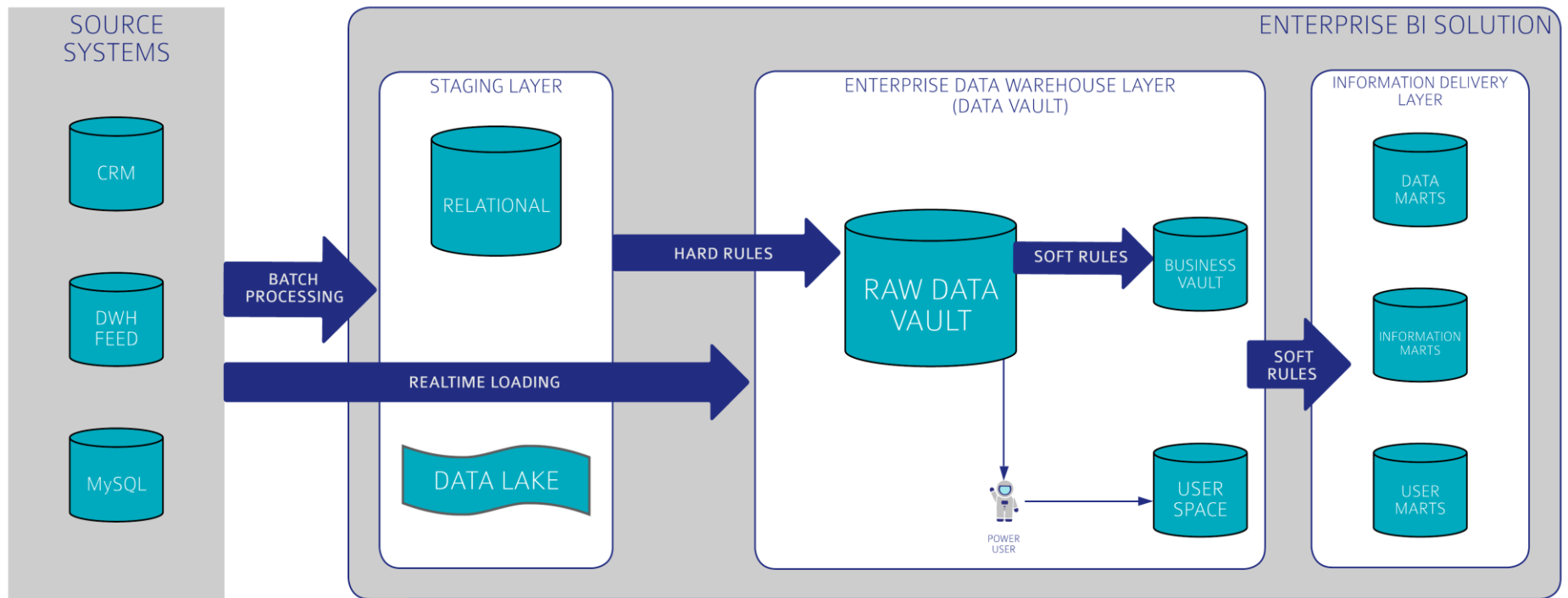
Business Vault layer:

Here we cleanse data, integrate systems and maintain history. This is our 'vault'.

All transformations, business rules, standardisation, calculations, and data enrichment

The challenge is that this doubles the number of tables.

Most teams start with the raw version of the vault and gradually add business vault views.



Modern companies use both models across different layers.

Raw Layer (Bronze/Raw):

A 1:1 copy from the source.

This separation ensures control over the raw data warehouse.

Raw Vault – no transformation, no business rules, no calculations.

Integration Layer (Silver/EDW):

DATA VAULT.

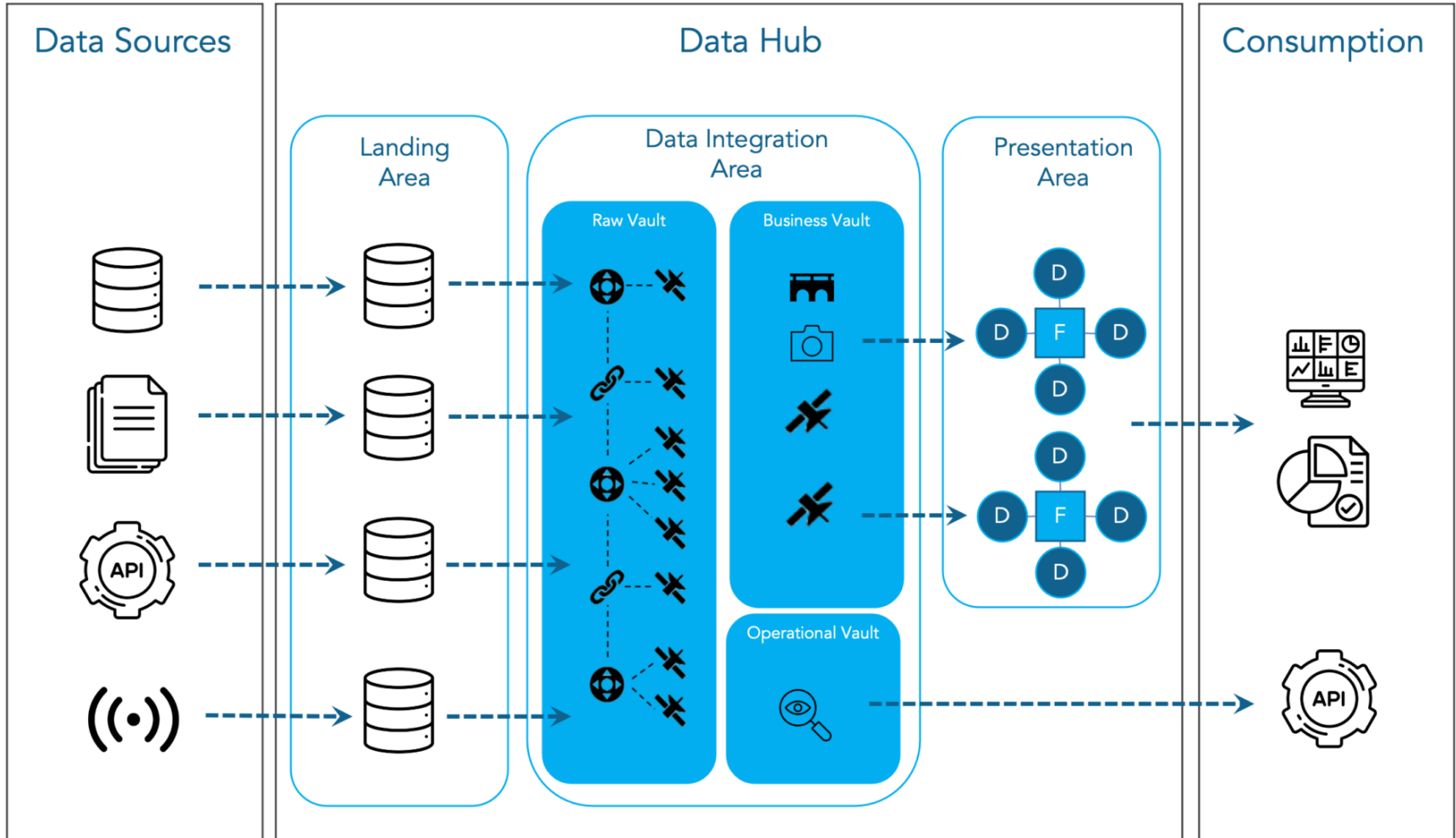
Here we clean the data, integrate systems and maintain historical records. This is our 'vault'.

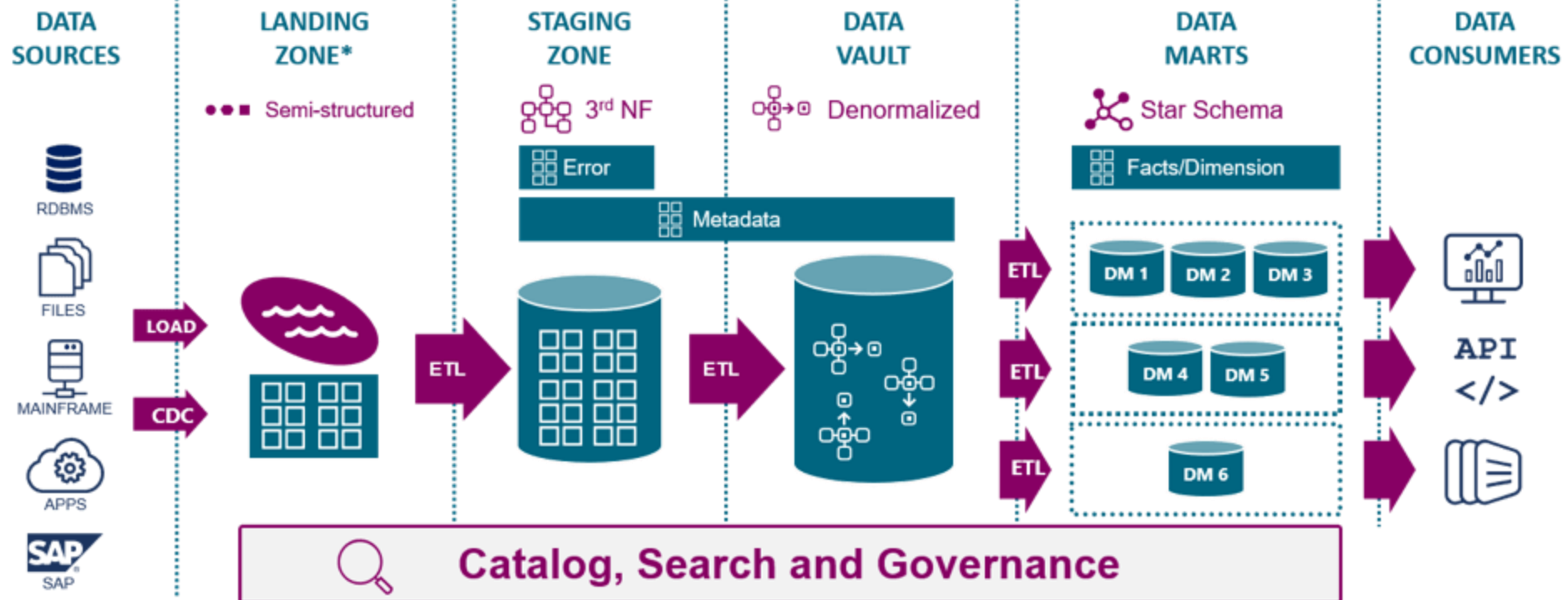
Business Vault – all transformations, business rules, standardisation, calculations, and data enrichment

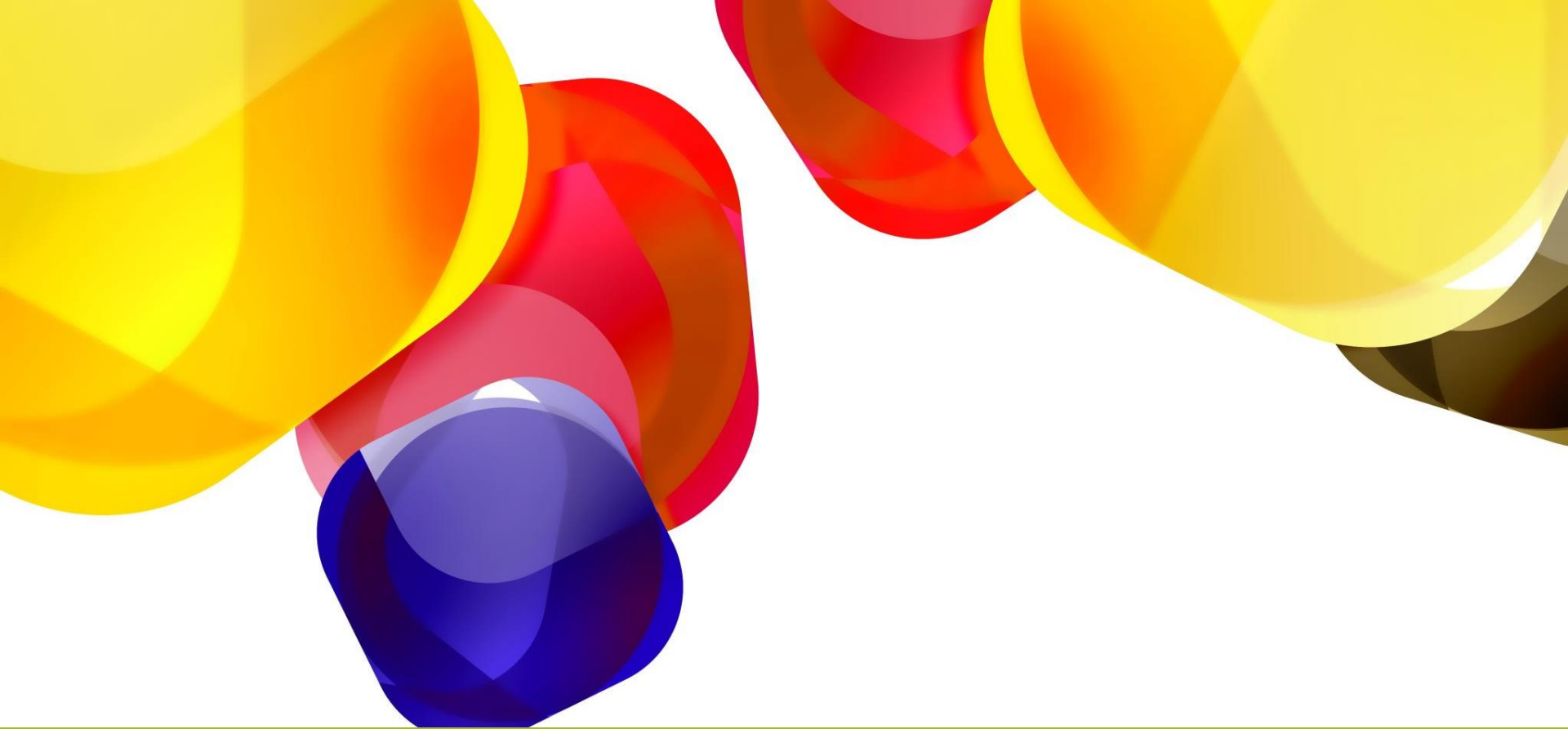
Presentation Layer (Gold/Marts):

KIMBALL.

From the Data Vault, we generate (virtually or physically) star schemas so that Tableau/PowerBI runs quickly.







Architectures

Summarized

In the DW literature – five predominant architectures

independent data marts,

bus architecture,

hub and spoke,

centralized

federated

Although

hybrid architectures are still possible

Centralised

Centralised

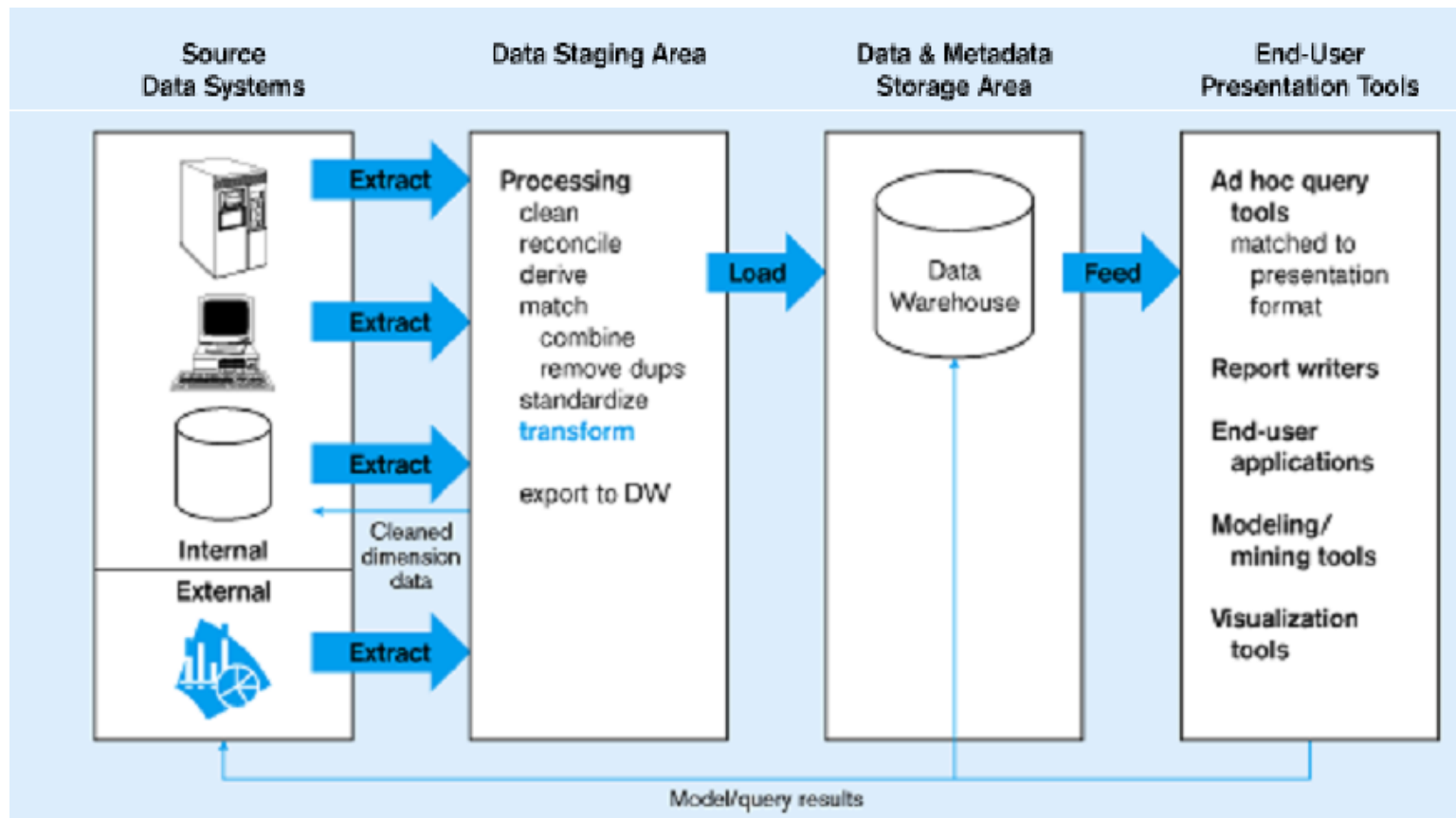
Enterprise data warehouse

Single ETL process

Single presentation thread

Centralized Data Warehouse Architecture





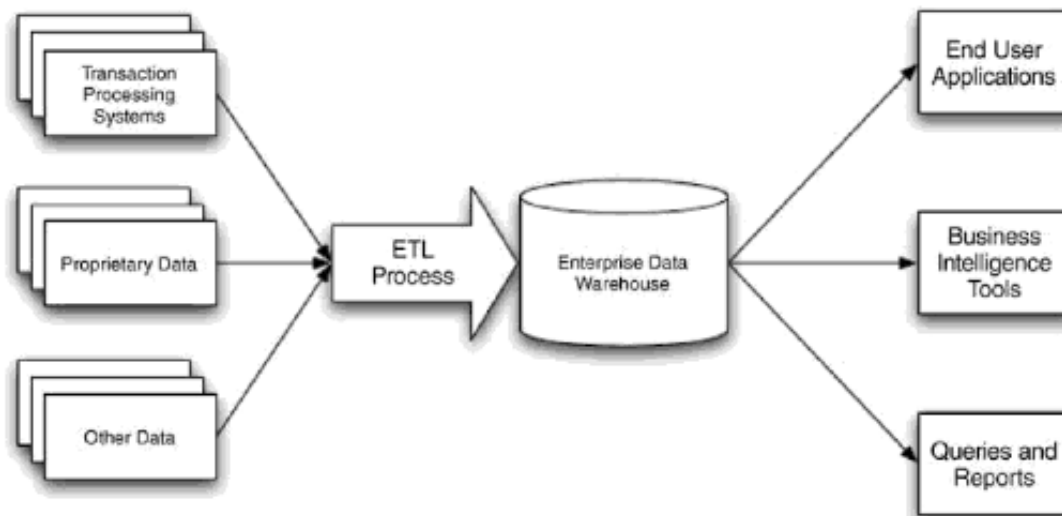
EDW

Simplest architecturally

Very large and complex development

ETL process

End user access



Independent data mart

Independent Data Mart

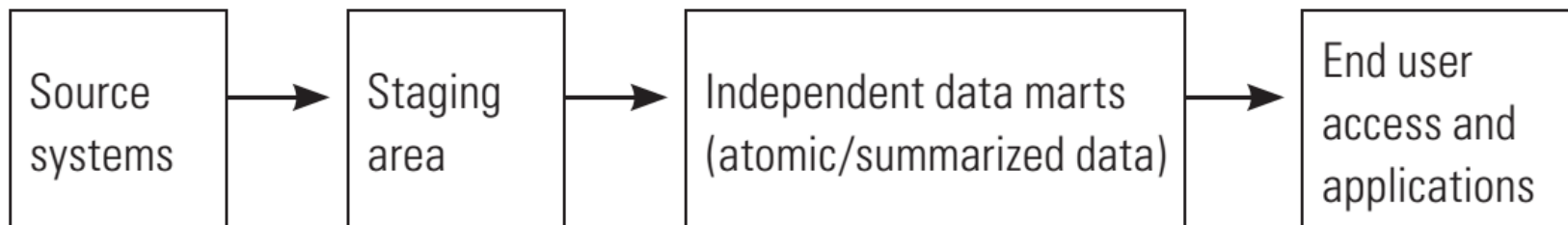
Mini warehouses – limited in scope

Separate ETL for each independent Data Mart

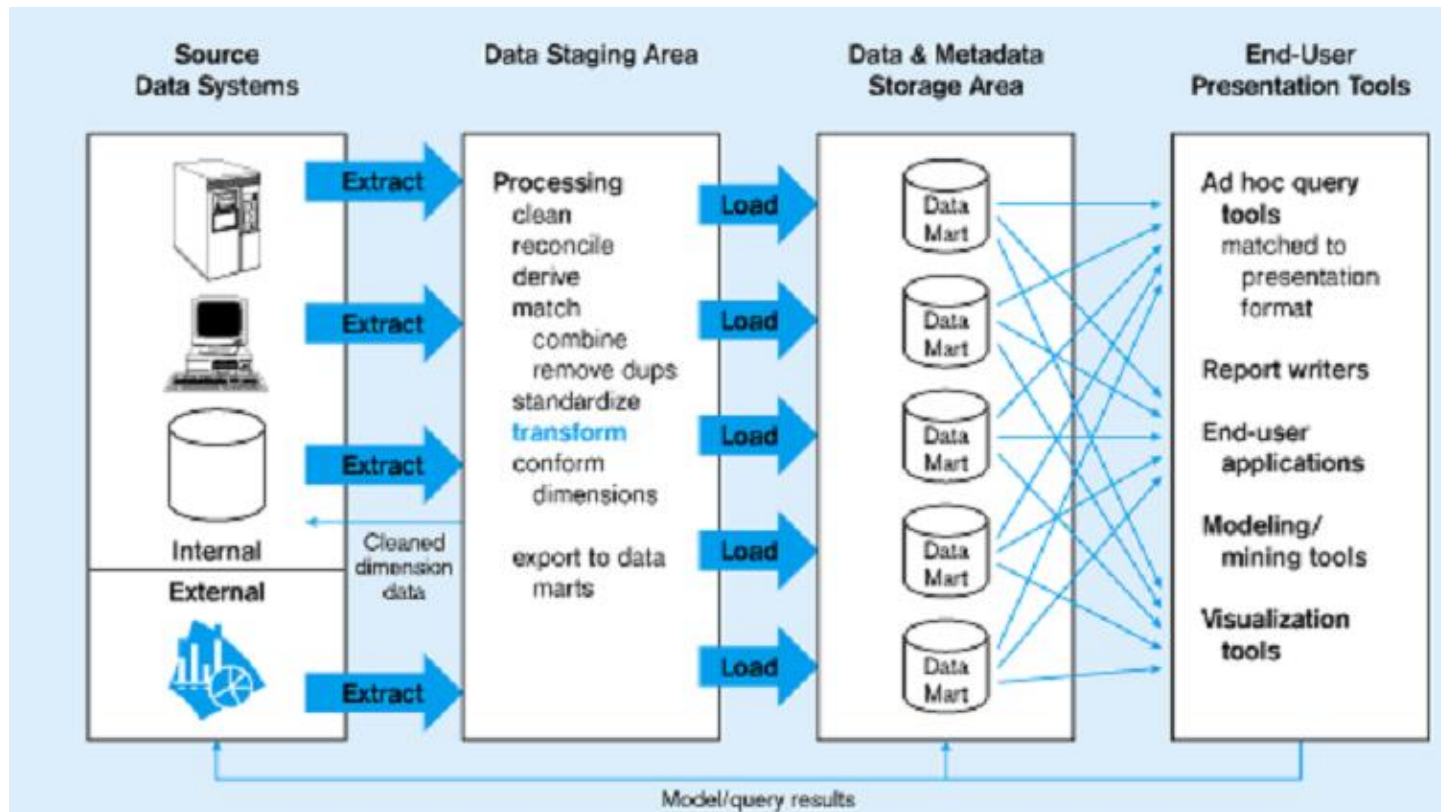
Multiple presentation threads

High Data Marts access complexity

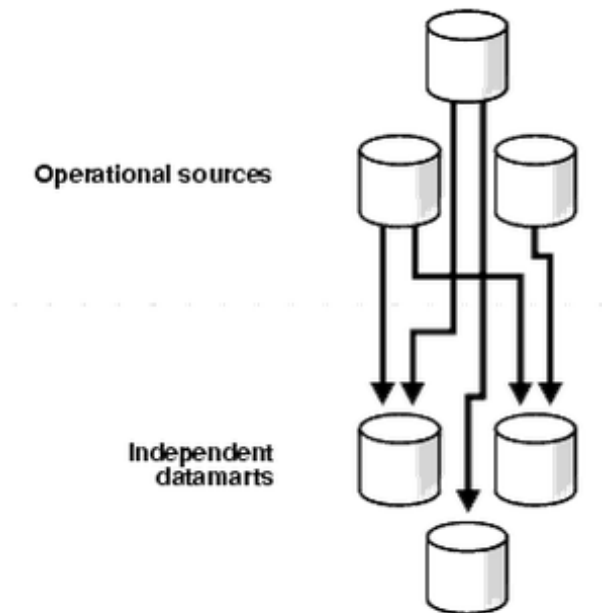
Independent Data Marts Architecture



Independent data mart



Independent data mart



Independent data mart

focuses exclusively on one subject area
is not designed in an enterprise context

For example,

manufacturing has their data mart, human resources does and so on

Data

from multiple transaction systems in one subject area or department to support specific business needs

Design

may use dimensional design or entity-relationship model

Analytic or business intelligence tools

query data directly from data mart and present information to user

Independent data mart

Independent data mart

takes very short time to build and bring the visible result to specific departments with less cost.

at the whole

different ETL tools needed

the data is duplicate in several data marts

inconsistent data definitions and different dimensions and measures

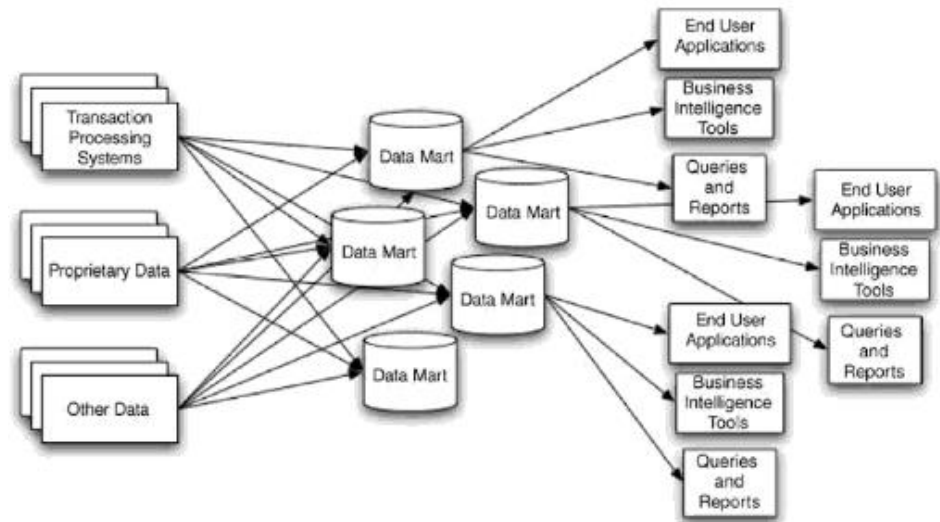
From the business perspective, each data mart is built to address a set of specific business needs,

what if the needs expand?

what if you want to analyze data across function or department?

The inconsistent data, such as the definition of a product

will make the information comparison between departments impossible.



Independent data mart

Independent data marts (IDM)

- the first endeavors to provide a repository of decision support data

- independent of other data stores and serve specific and localized needs

- such as providing data for a particular application or business unit*
- do not provide “a single version of the truth”

Dependent data mart

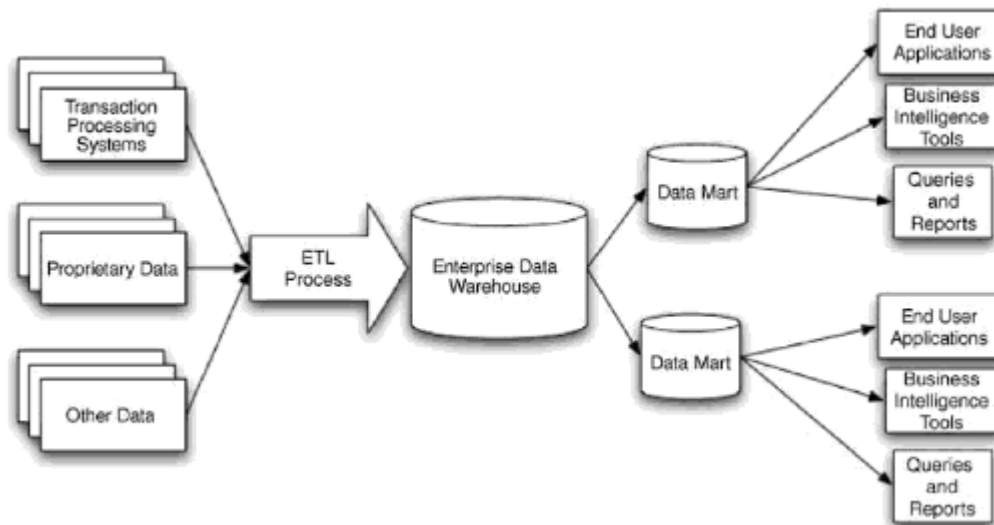
Dependent Data Mart and Operational Data Store

Single ETL for the DW

Data Marts are loaded from the DW

Simplified data access than in the previous case

Dependent data marts maintain a “single version of the truth”



Dependent data mart

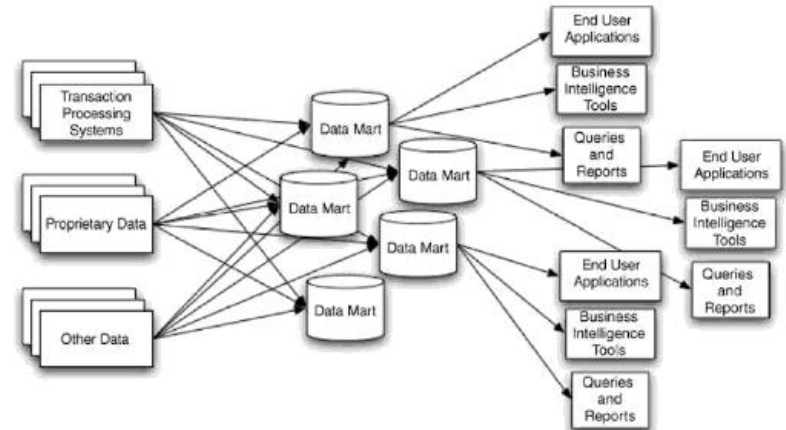
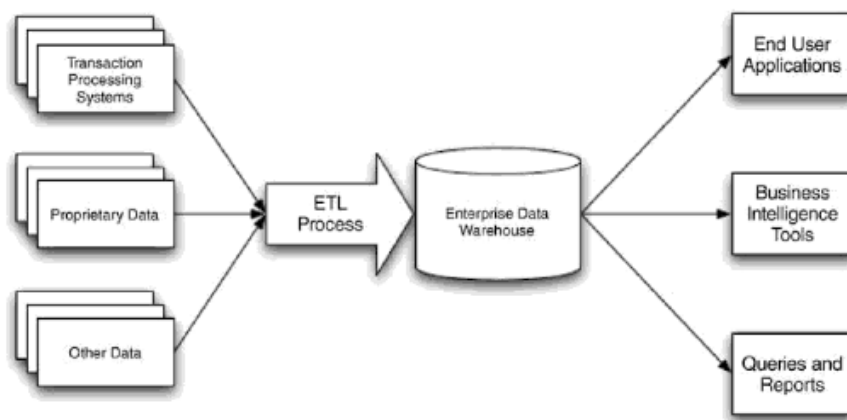
A compromise between EDW and IDM
Single ETL for the EDW – integration
Individual IDM – ease of access

At expense of having to maintain
EDW and IDM

The dependent data marts

may be developed for departmental, functional area, or specialized purposes (e.g., data mining)

may have normalized, denormalized, or summarized dimensional data structures depending on user needs



Dependent data mart

Dependent data mart

allows you to unite your organization's data

the usual advantages of centralization

Data

is aggregated, restructured, and summarized when it passes into the dependent data mart

integrated view

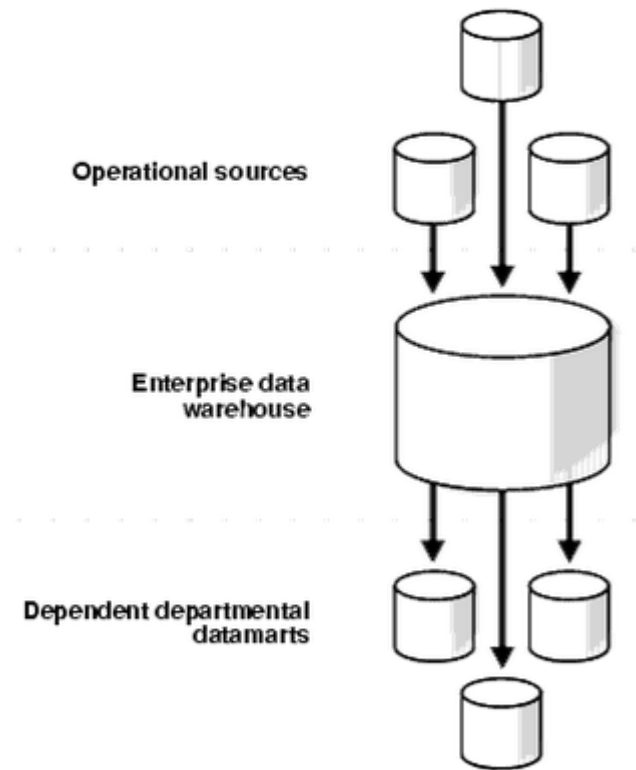
There are several benefits of building a dependent data mart:

Performance:

the data processing is performed outside the data warehouse

Security:

each department owns their data and has complete control over their data



Data Mart Bus

Data mart bus architecture with linked dimensional data marts (DBA)

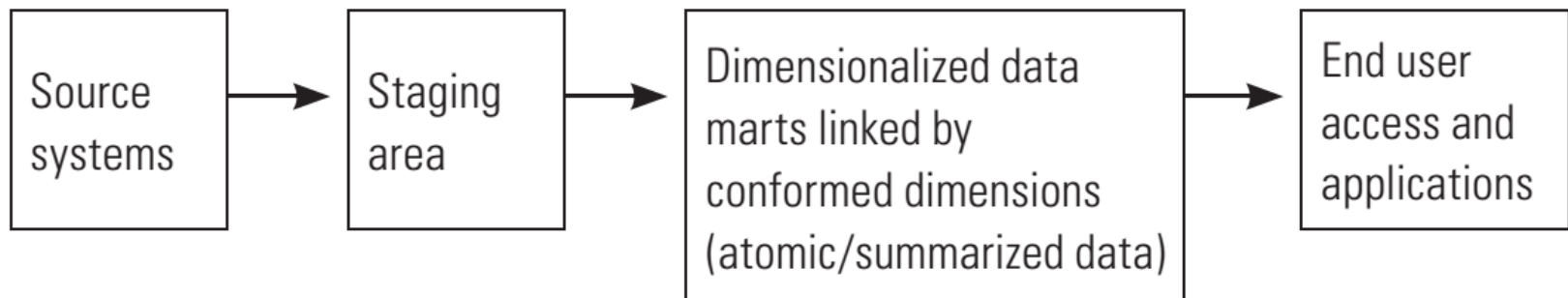
dependent data mart architecture

data marts that support various business processes

the first mart is built for a single business process using dimensions and measures that are used with other marts (i.e., conformed dimensions)

additional marts are developed using these conformed dimensions, which results in logically integrated marts and an enterprise view of the data

Data Mart Bus Architecture with Linked Dimensional Data Marts



Data Mart Bus

In this approach

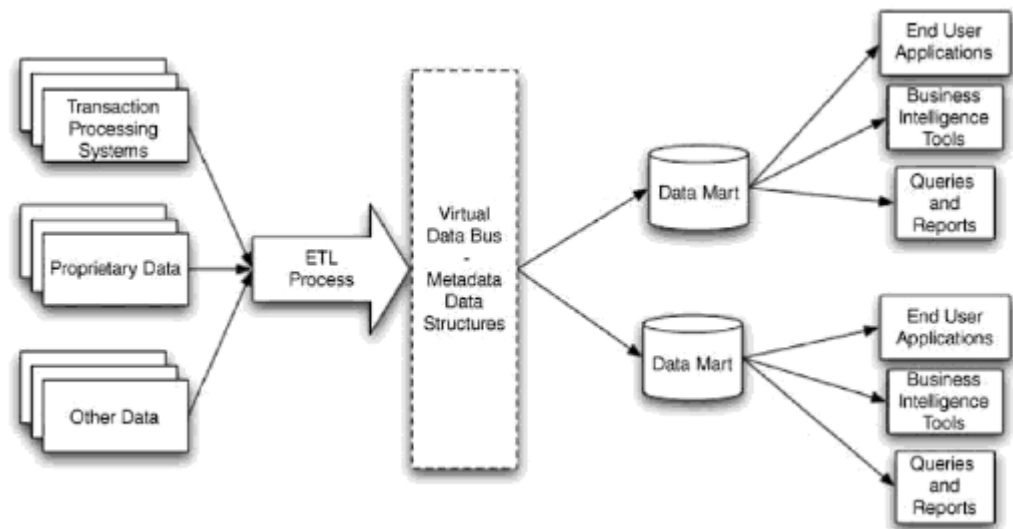
Centralised data sourcing routines are preserved

However data from ETL process are not stored in a EDW

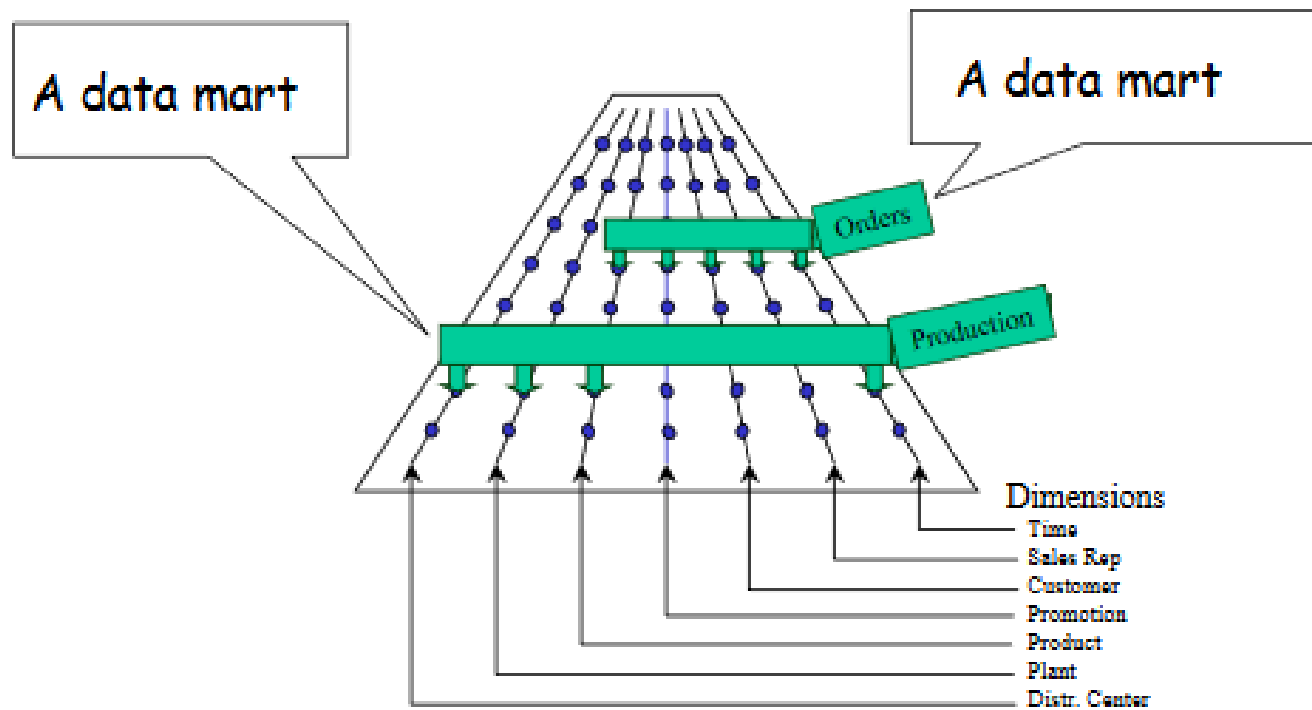
Single ETL process

Consists of a temporary storage area that represents a virtual warehouse

A common bus from which various data marts can draw data



Data Mart Bus



Hub-And-Spoke

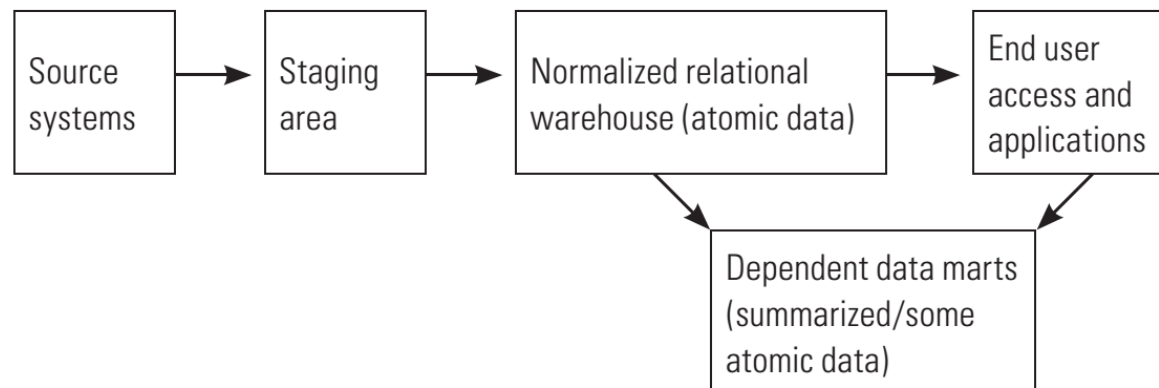
Hub-and-spoke

Developed after an enterprise-level analysis of data requirements

Focused on building a scalable and maintainable infrastructure

Dependent data marts obtain the data from the warehouse

Hub and Spoke Architecture (Corporate Information Factory)



Hub-And-Spoke

Hub and Spoke Architecture is developed in an iterative manner, subject area by subject area

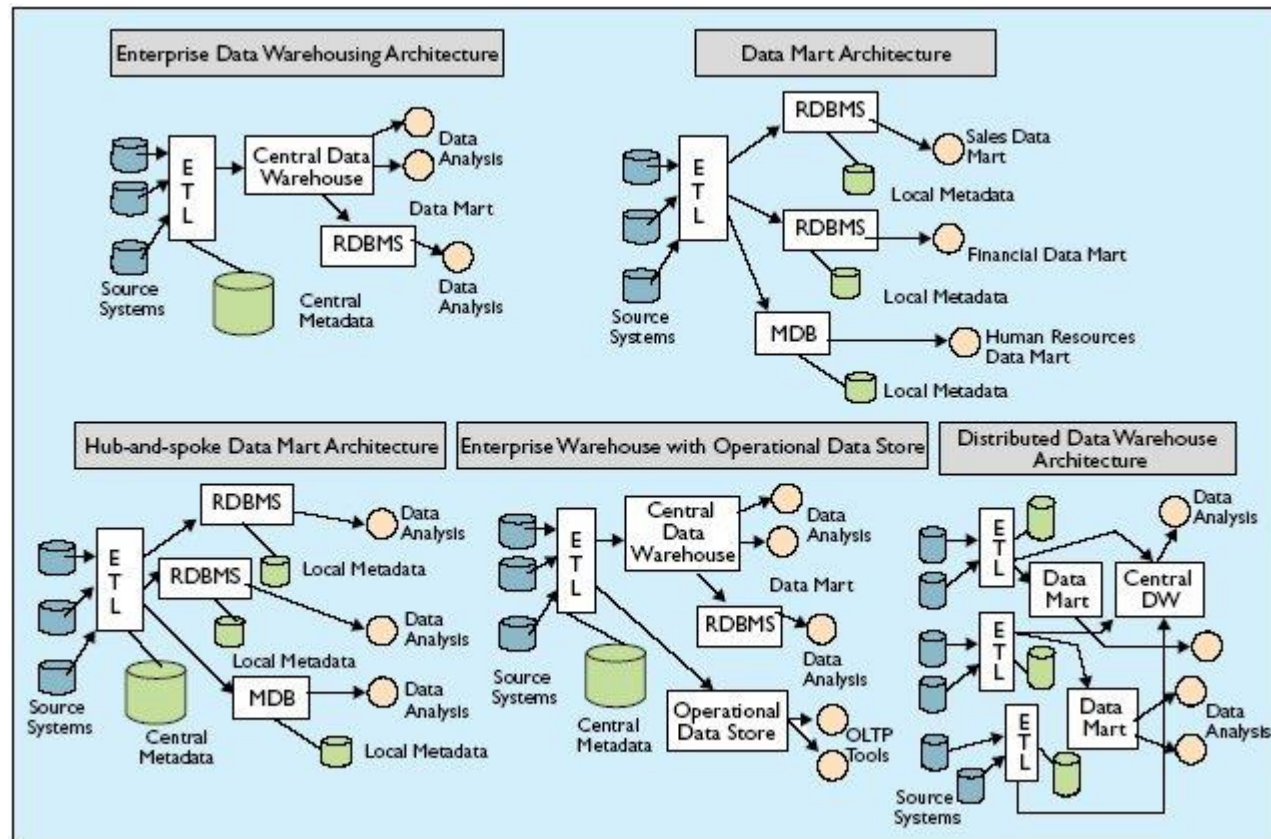
- atomic level data is maintained in the warehouse in 3rd normal form

Similarity between the centralised DW architecture and the hub and spoke

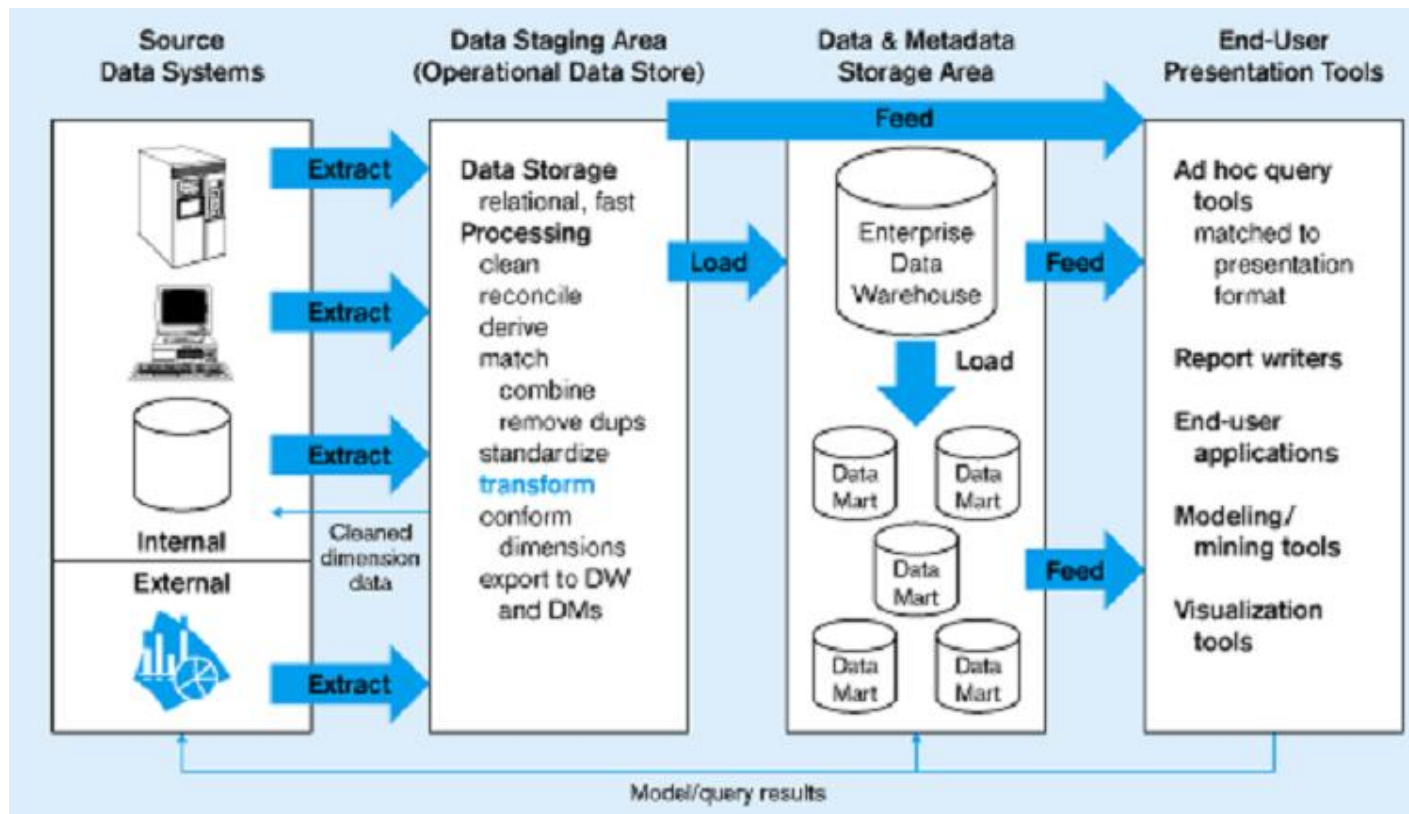
- except that in EDW there are no dependent data marts

- in H&S centralised DW contains only atomic level data, some summarized data, and logical dimensional views of the data

DW Architecture

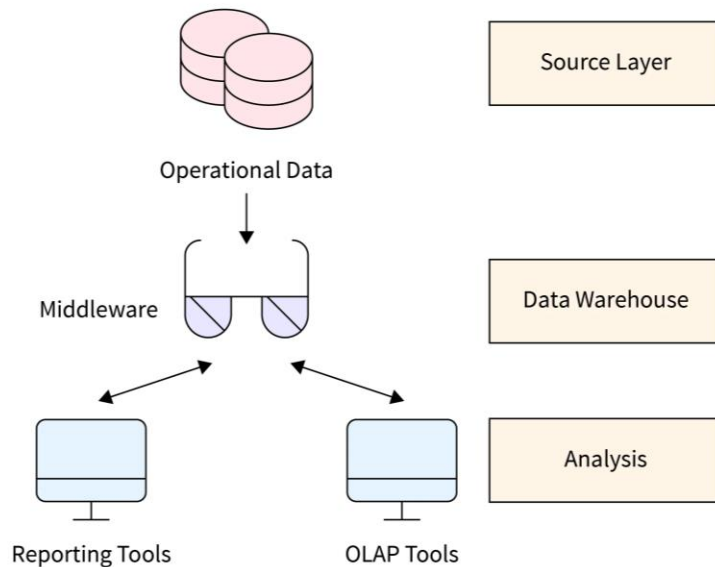


Dependent data mart with operational data store



Tier Architectures

Single-Tier Data Warehouse Architecture



SCALER
Topics

Single-Tier Data Warehouse Architecture

Simplest form, where the data warehouse acts as a centralized repository for all data.

Largely a conceptual or minimalist approach.

In this setup, there is no physical, separate data warehouse database

Reporting and Business Intelligence (BI) tools connect directly to the operational source systems (OLTP databases like production ERPs or CRMs) or a direct, un-transformed replica of them.

Pros:

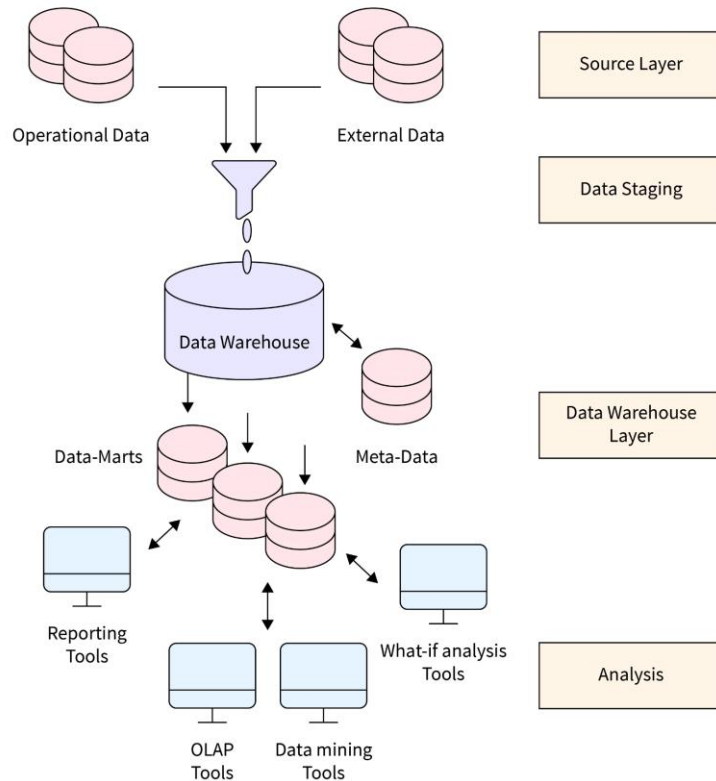
Extremely low deployment costs; zero infrastructure footprint to maintain; data is completely real-time (no ETL delays).

Cons:

Massive performance risk as a complex analytical query (e.g., calculating year-over-year revenue trends) can hog database memory and CPU, locking out operational users and slowing down the production application. It also lacks data cleansing, meaning users must deal with raw, messy data.

Tier Architectures

Two-Tier Data Warehouse Architecture



SCALER
Topics

Two-Tier Data Warehouse Architecture

introduces the first major separation of concerns

creating a dedicated, physical Data Warehouse (Database Server) that sits entirely apart from the operational source systems.

DW is separated from the source systems, creating two distinct tiers

Two tiers ... yet four data flow layers

Data is extracted from source systems, transformed, and then loaded into the data warehouse.

The warehouse itself handles both storage and querying.

Pros:

Better scalability compared to single-tier

Improved performance by offloading analytical queries from operational systems

Allows for more complex transformations

Cons:

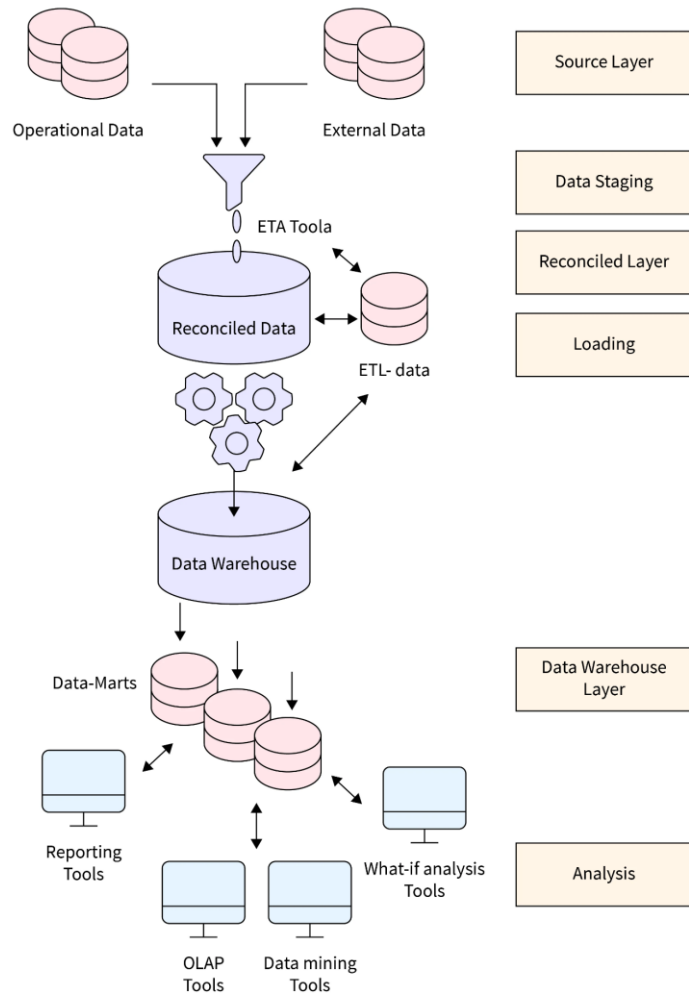
Increased complexity in design and maintenance

Potential for data latency between source systems and the warehouse

Use Case:

Medium-sized organizations with multiple data sources and more complex analytical needs.

Three-Tier Architecture for Data Warehouse System



Tier Architectures

Three-Tier Data Warehouse Architecture

The most common EDW type, which provides high scalability, performance, and integration with advanced analytics tools.

Consists of

Bottom Tier: Includes source systems and the staging area for initial data extraction and storage.

Middle Tier: Comprises the main data warehouse and potentially separate data marts.

Top Tier: Consists of query and analysis tools, reporting applications, and data mining tools.

Pros:

High scalability and flexibility

Clear separation of concerns between layers

Supports complex querying and analytics

Better performance for large-scale data processing

Cons:

More complex to design and implement

Higher initial costs

Requires more specialized skills to manage

Use Case:

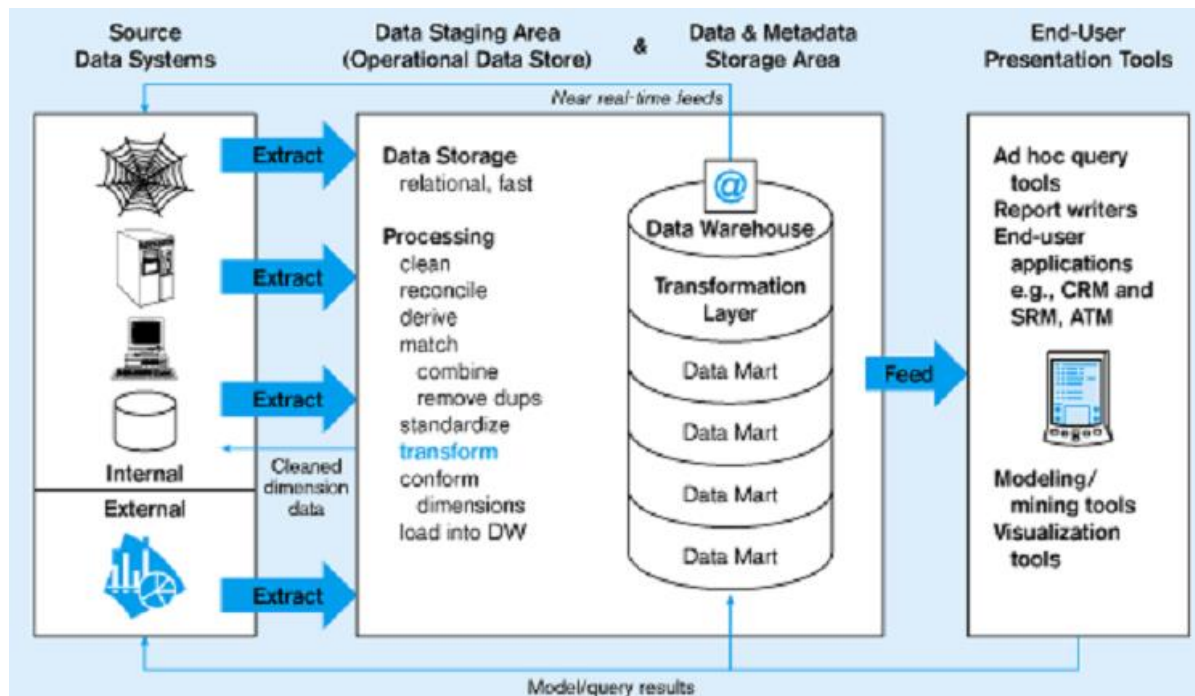
Large enterprises with diverse data sources, complex analytical requirements, and the need for high scalability.

Tier Architectures

Logical Data Mart and Active Warehouse

The ETL is near real-time

Data Marts are not separate databases, but logical views of the DW



Hybrid

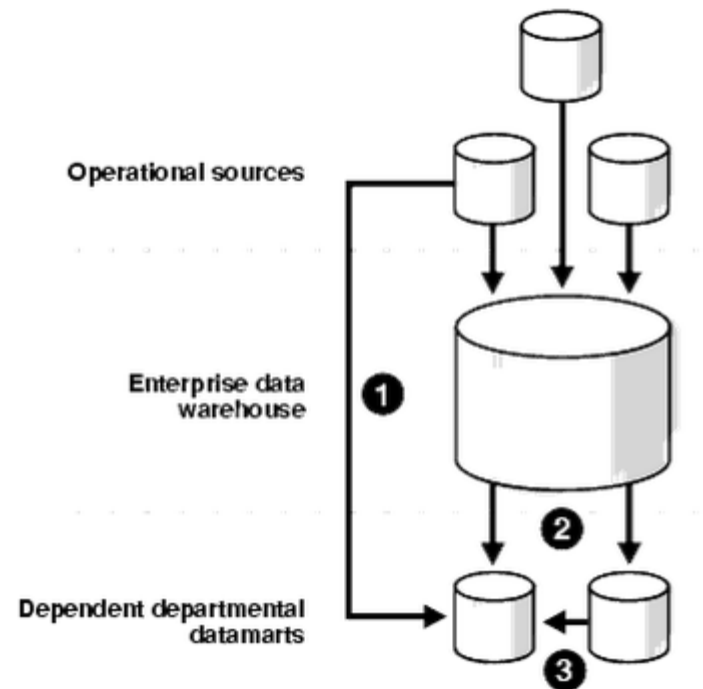
A hybrid data mart

allows to combine input from
sources other than a data
warehouse

This could be useful for many
situations

especially when you need ad hoc
integration

*such as after a new group or
product is added to the
organization*



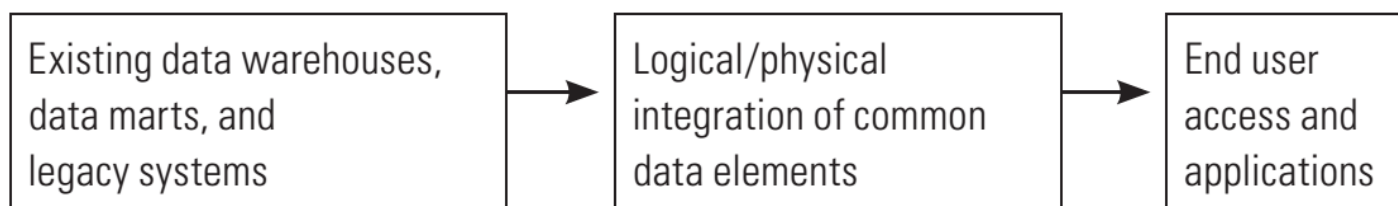
Federated

The big organization has various regions that provide businesses to customers of that region

Different regional data warehouses were built for each region to meet the specific business needs.

A global data warehouse also was built to provide analytical capabilities to the executive at the global level.

Federated Architecture



Federated

Two data flows between regional and global data warehouses:

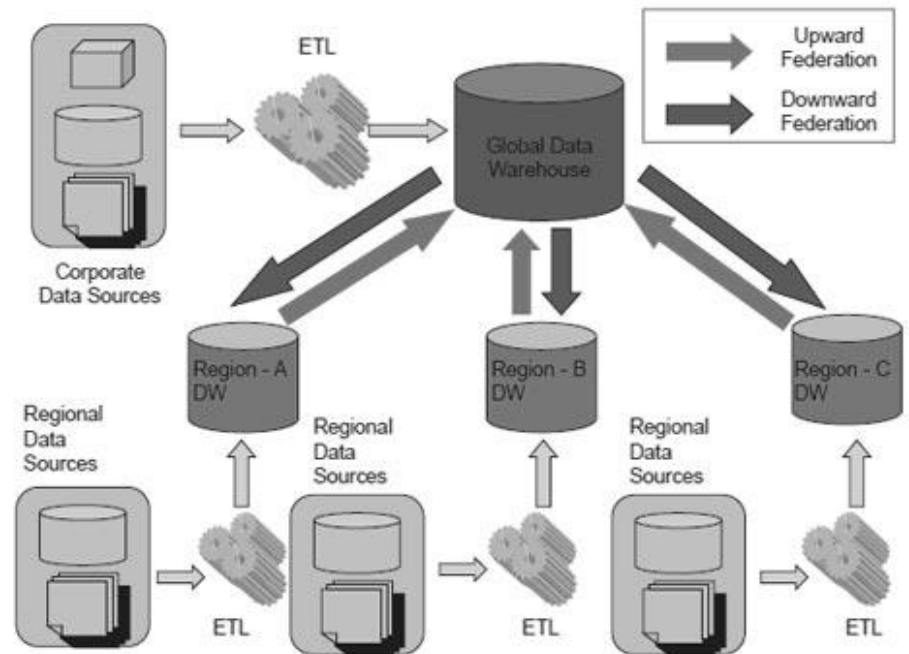
Upward federation

from regional to global
only fact data are moved
aggregation

Downward federation

from global to the regional
ensure consistency and integrity of data across the organization

Transactional data from corporate operational systems



Federated

Benefits of federated data warehouse

Ease of implementation

Integrate all legacy data warehouses, business intelligence systems into a newer system that provides analytical capabilities across function.

do not try to rebuild a new system

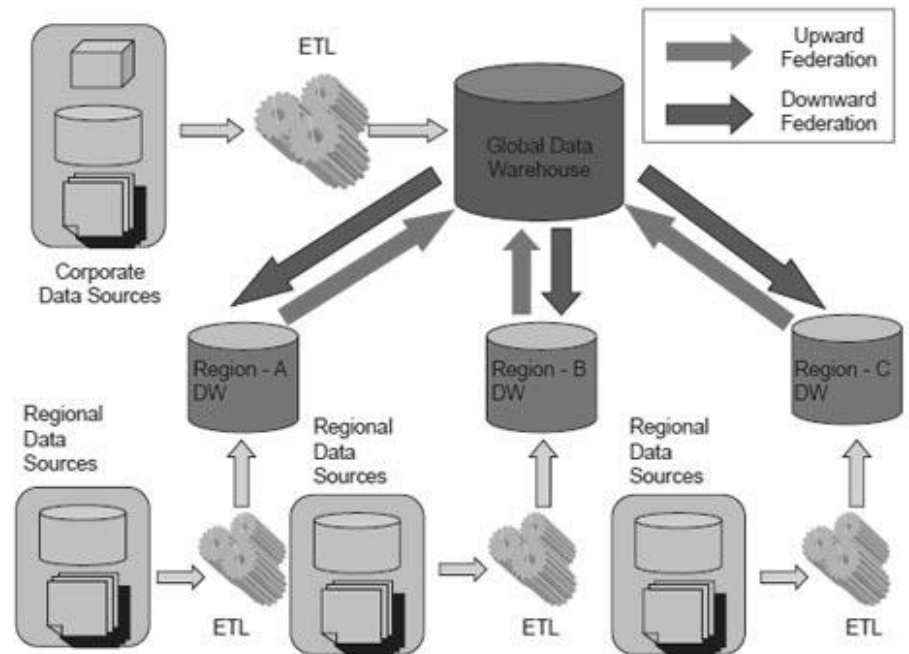
Shorter implementation time

By integrating all legacy BI systems, federated data warehouse approach has a shorter implementation time in compare with a lengthy processing of building enterprise data warehouse

Cross-functional analytics requirements

accomplished using common business module across different BI systems of each department

dynamic cooperation of various business intelligence systems



Federated

Federated

- Leaves existing decision-support structures in place

- Shares information among a number of different systems

- Data is either logically or physically integrated

 - Shared keys*

 - Global metadata*

 - Distributed queries*

Distributed DW

Geographically distributed

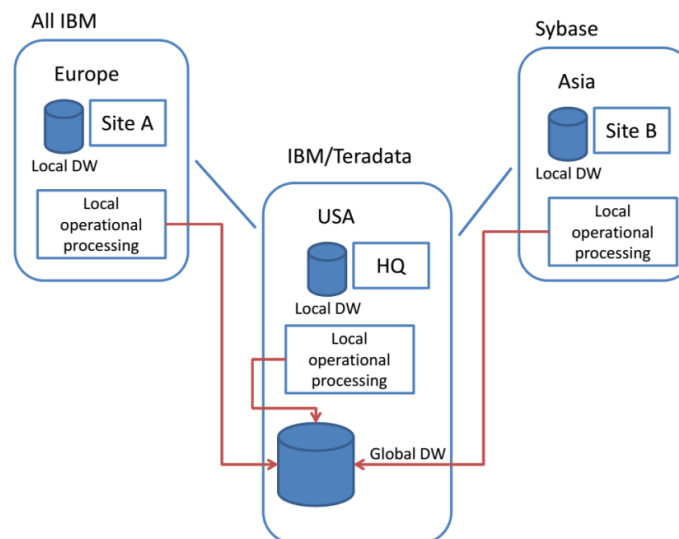
In the case of corporations spread around the world

Information is needed both locally and globally

A distributed DW makes sense

When much processing occurs at the local level

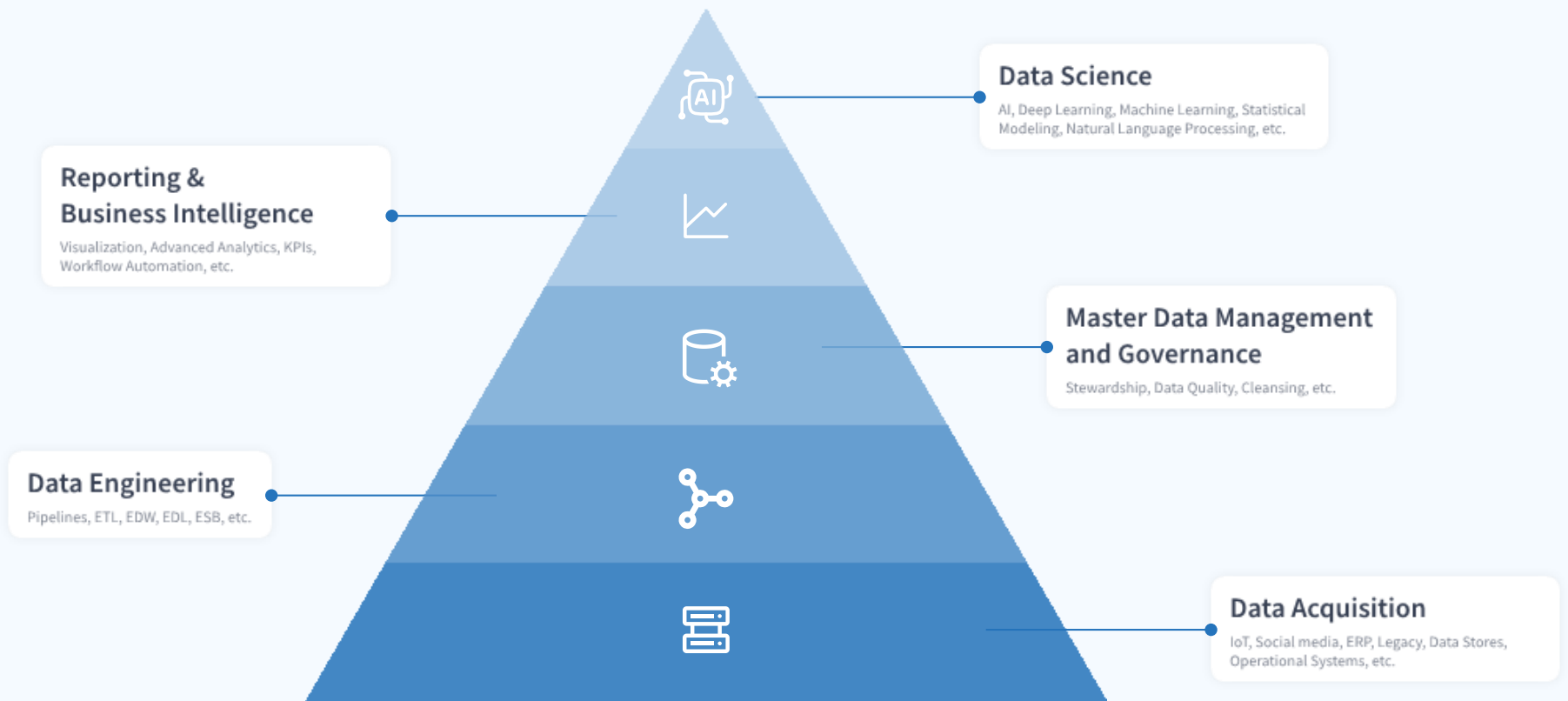
Even though local branches report to the same balance sheet, the local organizations are their own companies





Modernizing

Modern Data Warehouse Pyramid





Modernizing

Main trends have changed the way businesses treat their data assets and shaped a new generation of data warehouses:

Big data results in the need to **process petabytes of data within a single data warehouse** for business insights.

Cloud computing has created **access to near-infinite, low-cost storage with scalable computing power to analyse data with a pay-as-you-go plan.**

The rise of **data science and machine learning** demands **tighter integration among a data warehouse, ETL and data processing tools as well as data visualisation tools** to enable near real-time analytics for decision-making.

Modernizing a data warehouse with big data

Common technical scenarios for modernizing data warehouse with big data



ETL Offload Data Prep

- Leverage cost effective big data to offload ETL and data preparation
- Transformation is done using big data scripts/code instead of through an ETL tool
- When data is prepared, load into warehouse for interactive BI



Single view (DW + Big Data)

- Customer 360 degree view now includes both relational (customer purchases, contacts) and non-relational (sentiment, clickstream, etc.)
- Data warehouse and Data Lake are side-by-side collecting data and queried using federated query technologies (PolyBase)



Active Archive (DW Hot storage / Big Data cold)

- Leverage cost effective big data to archive all data indefinitely as "cold" storage that can be processed at anytime with big data
- Pull in frequently used hot data for interactive BI in the data warehouse



Run data science / experimentation

- Keep data warehouse for operational reporting and BI
- Leverage a separate big data implementation for running data science / experimentation with uncured, non-relational data to find new patterns and new insights
- When patterns are found, operationalize into the data warehouse for reporting
- Use R Server for data science

Core Components of a Modern Data Warehouse

The Data Ingestion Layer – ELT over ETL

The Storage Layer – Object Storage & Open Table Formats.

At the base of the modern warehouse is the storage tier. Instead of proprietary database storage formats, modern architectures often leverage cheap, infinitely scalable cloud object storage as their underlying data lake.

The Compute Engine – muscle of the MDW.

The compute engine is responsible for executing SQL queries, aggregating data, and powering reports.

Because compute is separate from storage, these engines can spin up or down automatically based on demand, preventing performance degradation during heavy peak usage.

The Transformation and Modelling Layer

Once raw data is loaded into the warehouse storage layer, it must be cleaned, structured, and modelled.

The Orchestration Layer

With hundreds of ingestion pipelines and transformation models running daily, data pipelines have complex dependencies.

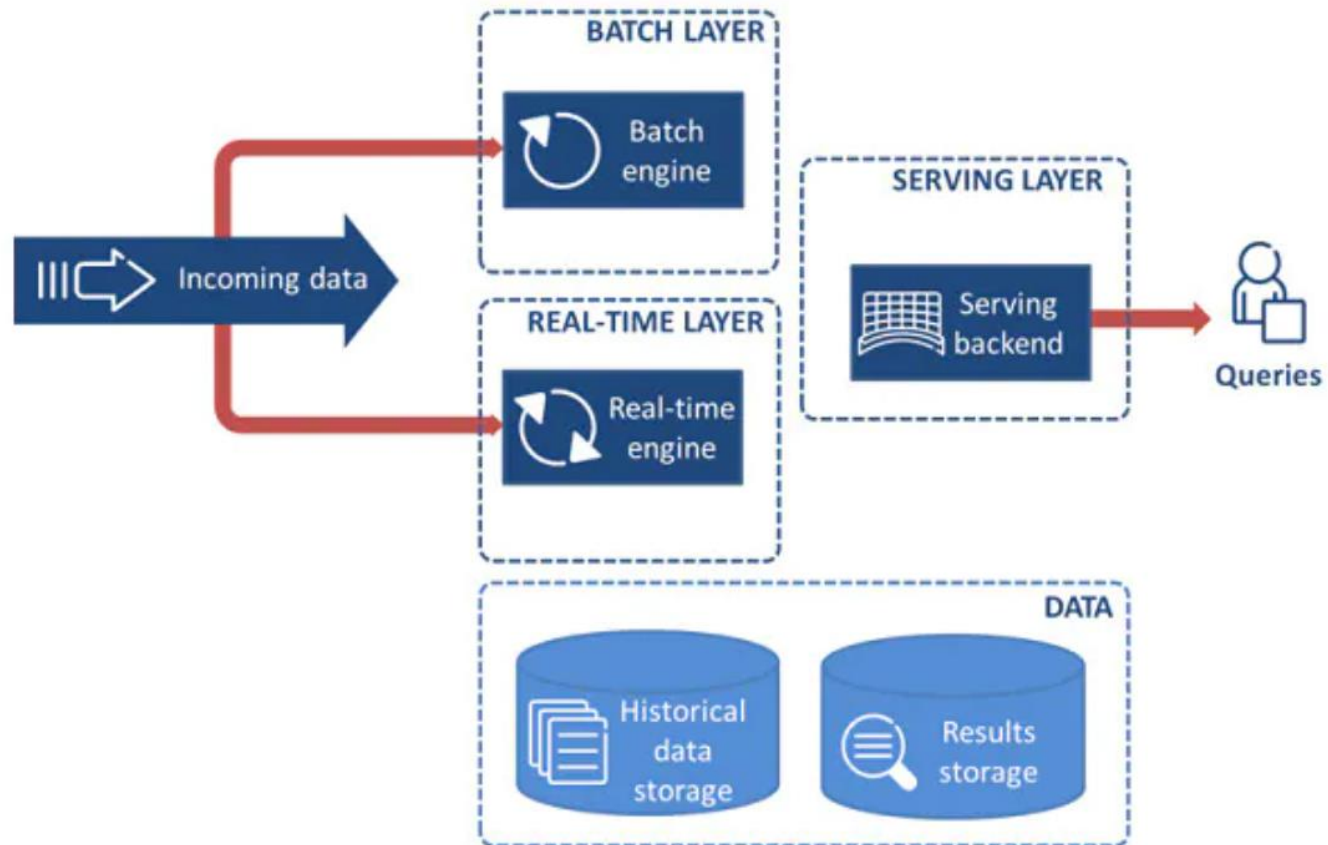
The Governance, Observability, and Metadata Layer

As data systems grow, keeping track of data health, regulatory compliance (GDPR/CCPA), and data definitions becomes critical. This layer acts as the safety and quality net for the MDW.

The Consumption and Activation Layer – where data provides business value.

In an MDW, consumption goes far beyond standard dashboards might include Semantic/Metrics Store (ensuring unified metric formulas before they reach the BI layer), Data Activation (Reverse ETL - pushing computed insights back into operational tools) or AI & LLM Consumption (feeding clean enterprise data into vector databases or Retrieval-Augmented Generation (RAG) pipelines to power internal AI agents).

Lambda



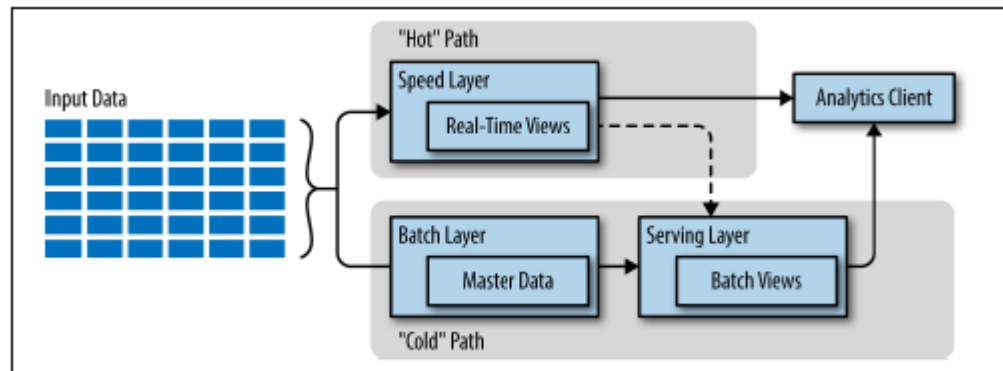
Modernizing DW – Lambda architecture

Typically in Lambda Architecture there are two paths for data flow,

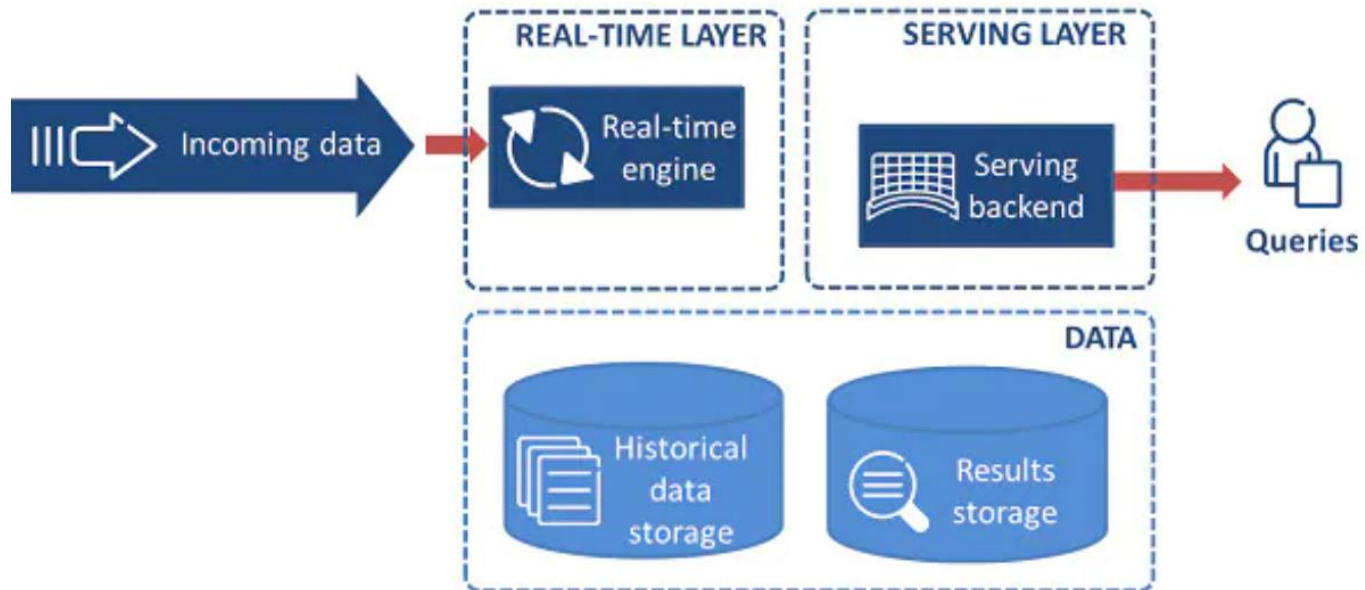
A “Hot” Path — This is for latency sensitive data which flows for rapid consumption for serving to end clients.

A “Cold” Path — This is for processing the entire data and is processed in batches for workloads that can tolerate greater latencies (several hours probably) until results are ready.

Processing logic appears in two different places — the cold and hot paths — using different frameworks.



Kappa

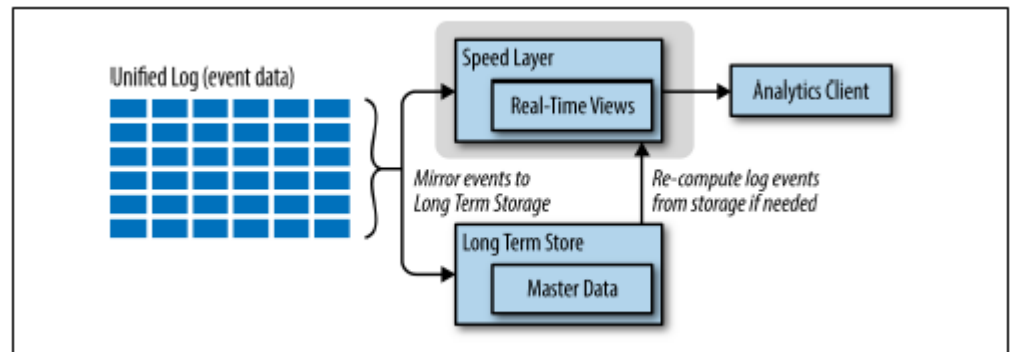


Modernizing DW – Kappa architecture

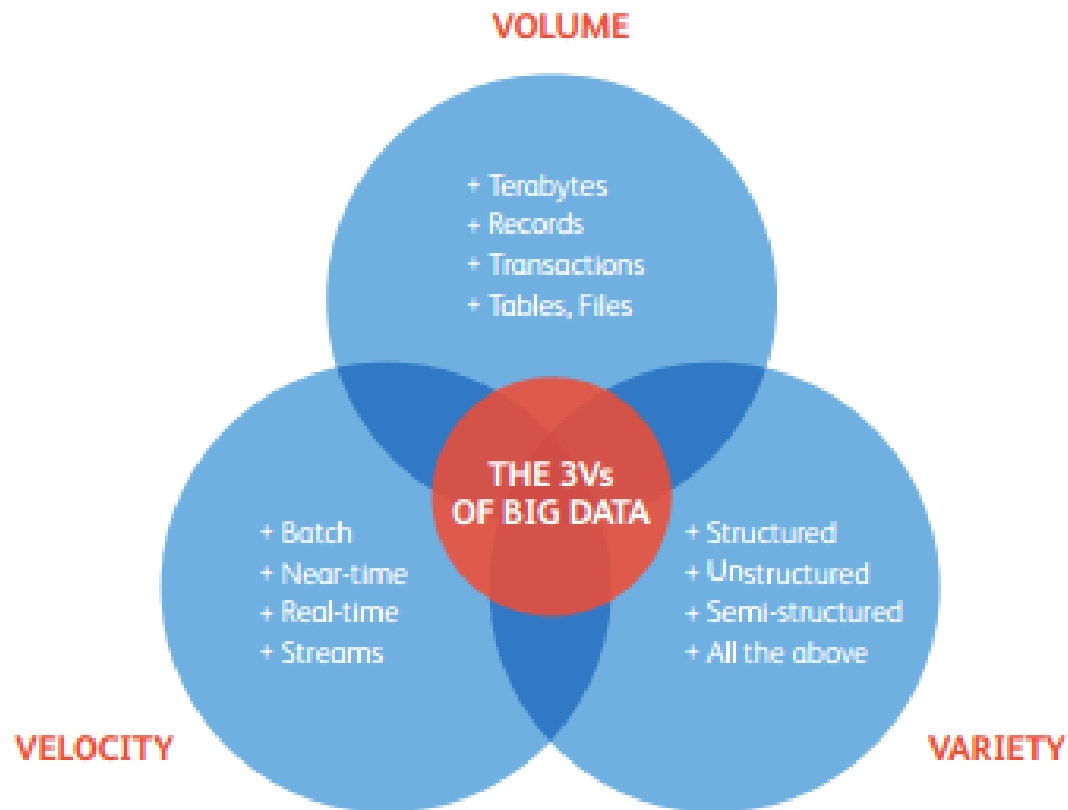
Simplifying the Lambda Architecture by eliminating the cold path

All processing happens in a near real-time streaming node and recomputation on the data happens when needed – being streamed through the Kappa pipeline again.

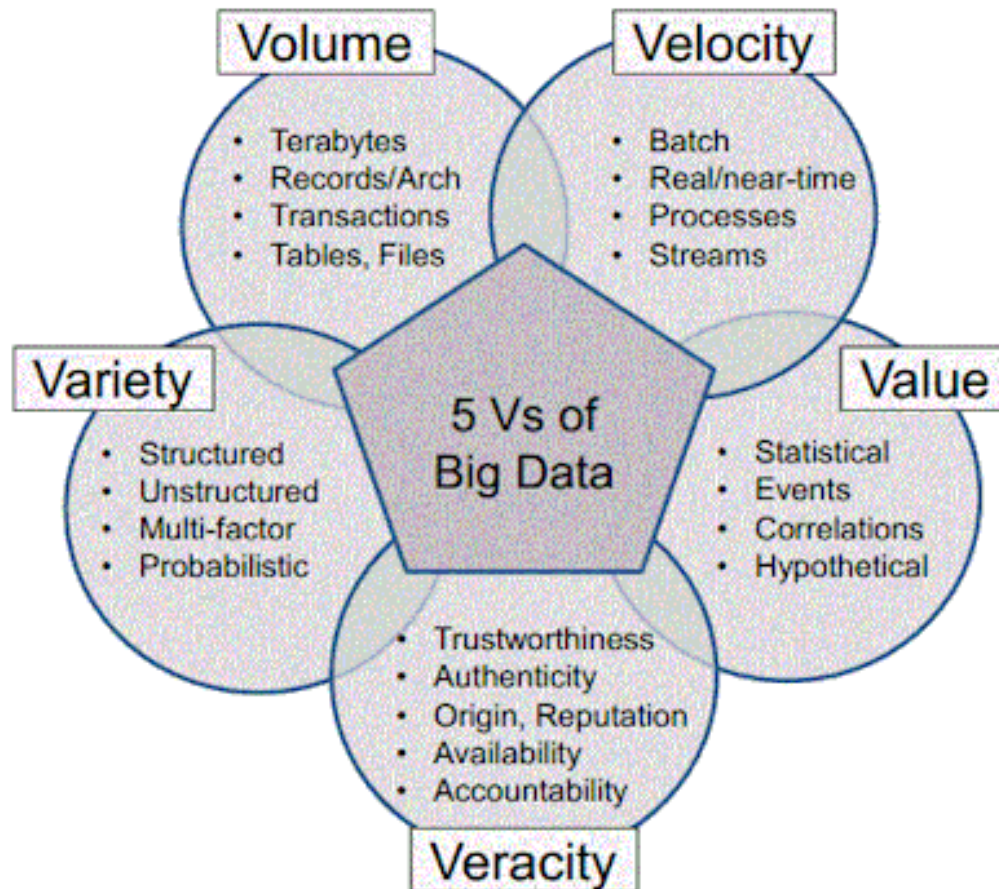
All data flows through a single path, using a stream processing system.



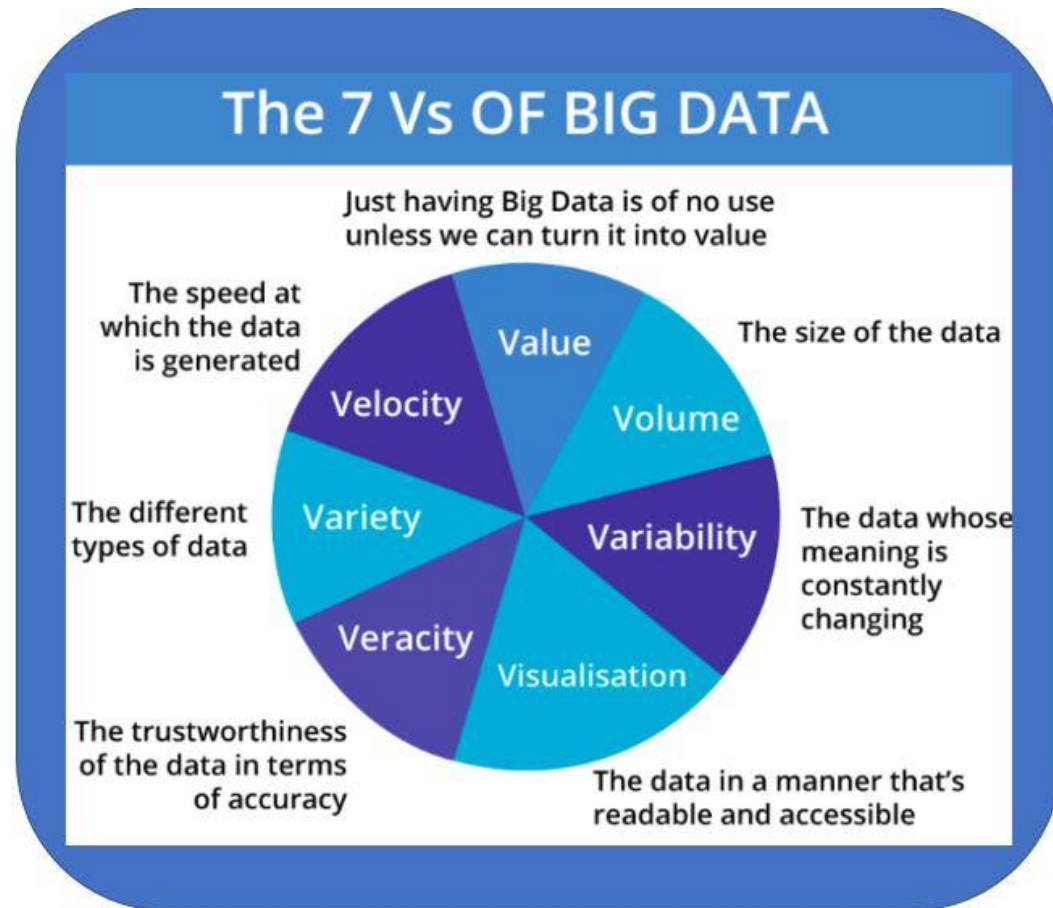
Big Data



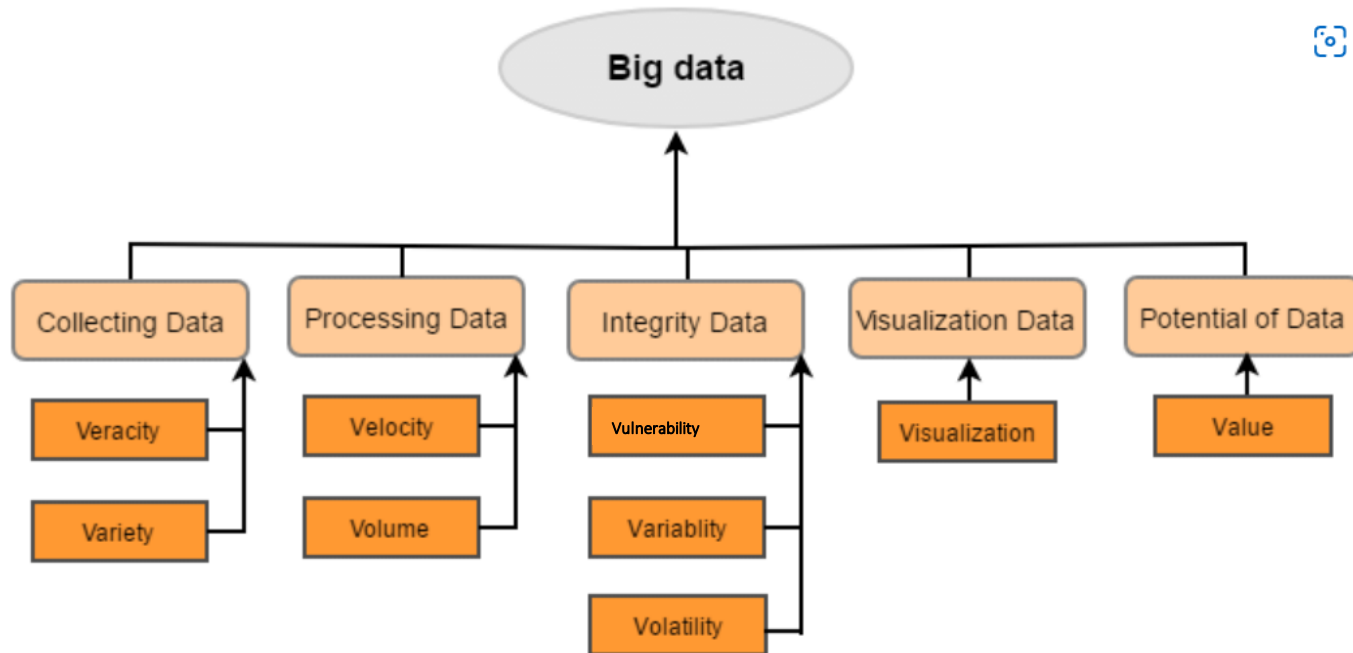
Big Data



Big Data



Big Data



Dixon (2010) first referred to the term data lake - as a large body of water in its natural state:

data lakes can contain raw data without any processing, and require further processing to be viable

This is actually one of the defining differences between a traditional data warehouse and a data lake.

In the acquisition phase, data can be stored without a schema.

Later, a schema can be defined when the data is ready to be read.

Schema-on-read

A data lake can contain various data

structured, semi-structured and unstructured

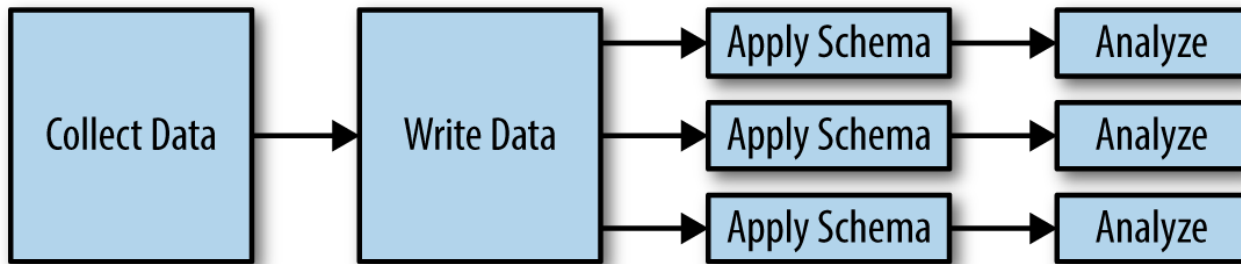
from both internal and external resource

It can serve as a source for other warehouses of large data sets,
such as data warehouses and data repositories.

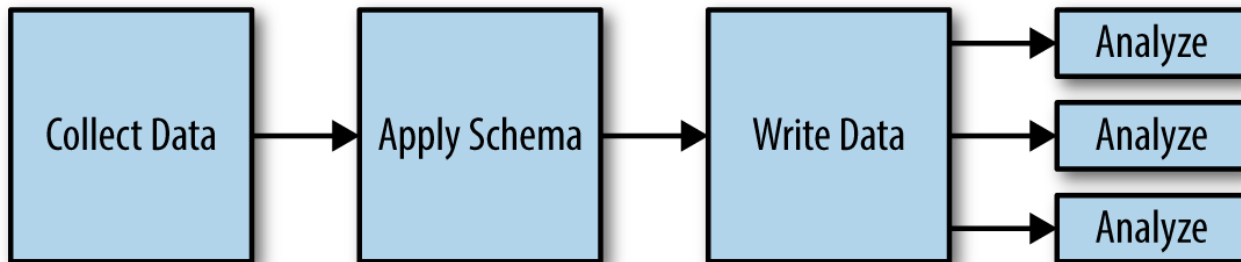
Once the data has been cleaned

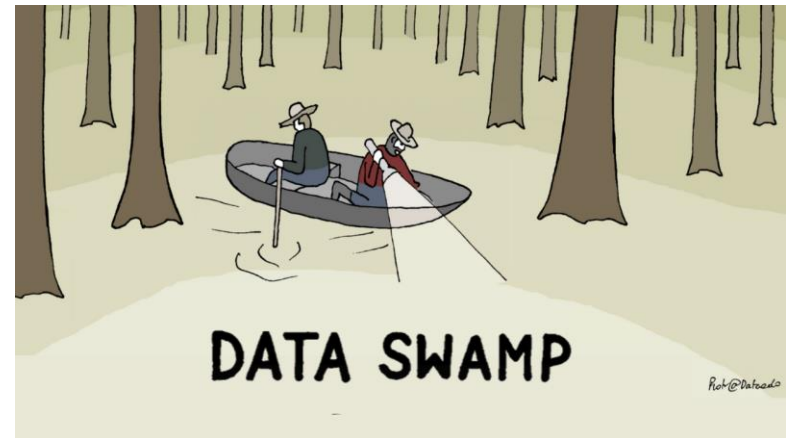
it can be moved from the data lake to a more carefully controlled warehouse.

Schema-on-Read



Schema-on-Write





4-zone system:

Transition zone

is used to store ephemeral data, such as temporary copies, streaming copies or other short-term data prior to acquisition.

Unprocessed zone

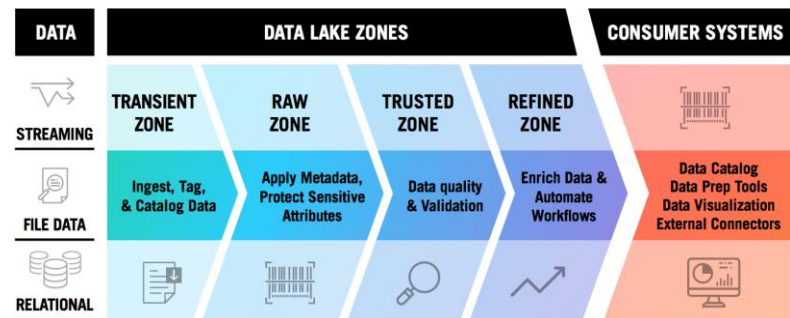
zone where unprocessed data is stored. zone where sensitive data must be encrypted, tokenized, or otherwise secured

Trusted zone

after data quality verification, validation, or other data processing in the raw zone, it becomes the "source of truth" in this zone for slave systems

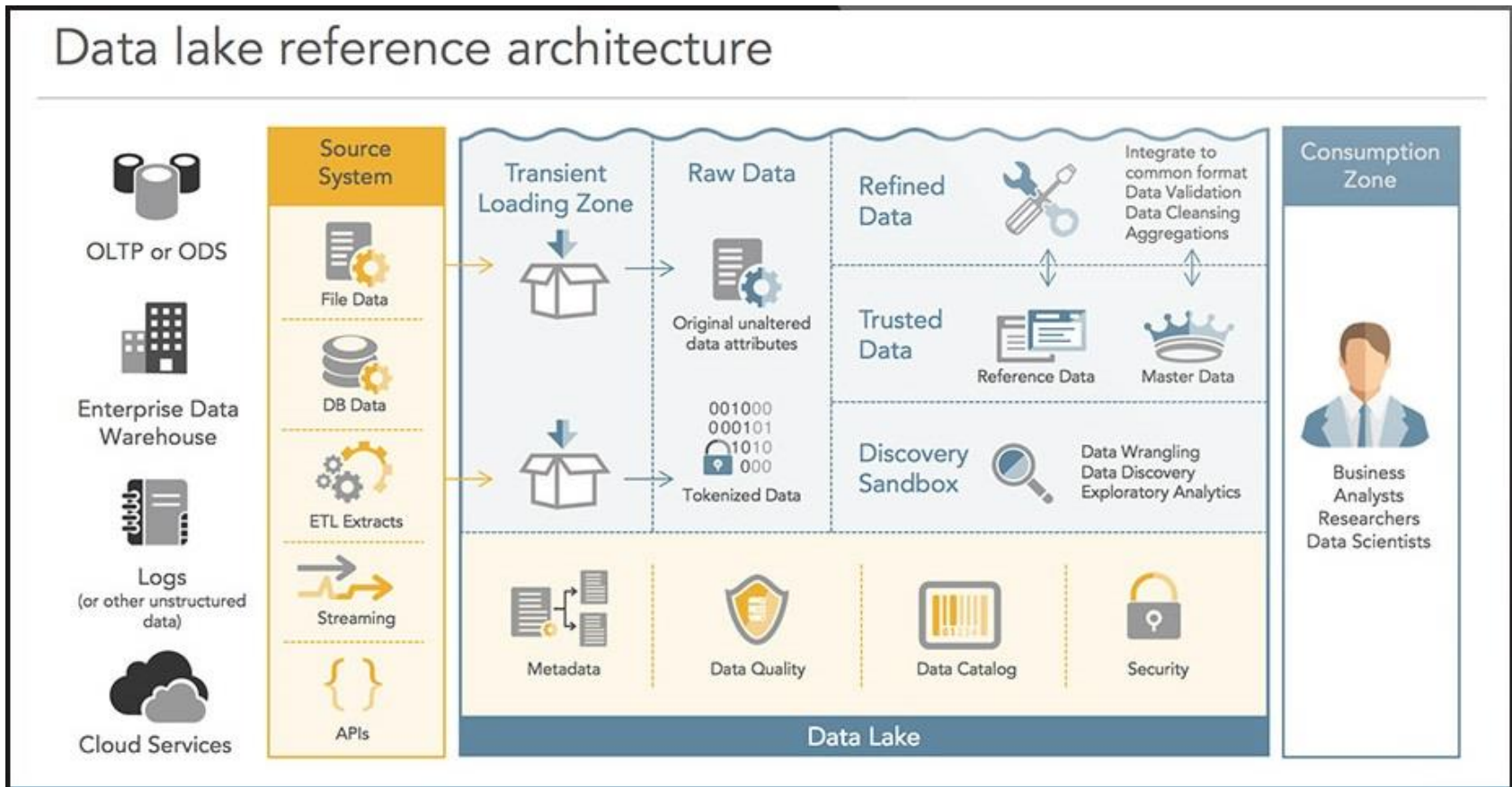
Enhanced zone

*manipulated and enriched data is stored in this zone
used to store output data.*



Data lake

Data lake reference architecture



The DELTA LAKE Architecture



Delta Lake allows you to *incrementally* improve the quality of your data until it is ready for consumption.

Bronze Layer (raw data)

Raw data from source systems, without processing

The goal is to capture raw data from sources in its original, unaltered format ("as-is")

Various formats, e.g. CSV, JSON, Parquet, Avro, raw logs

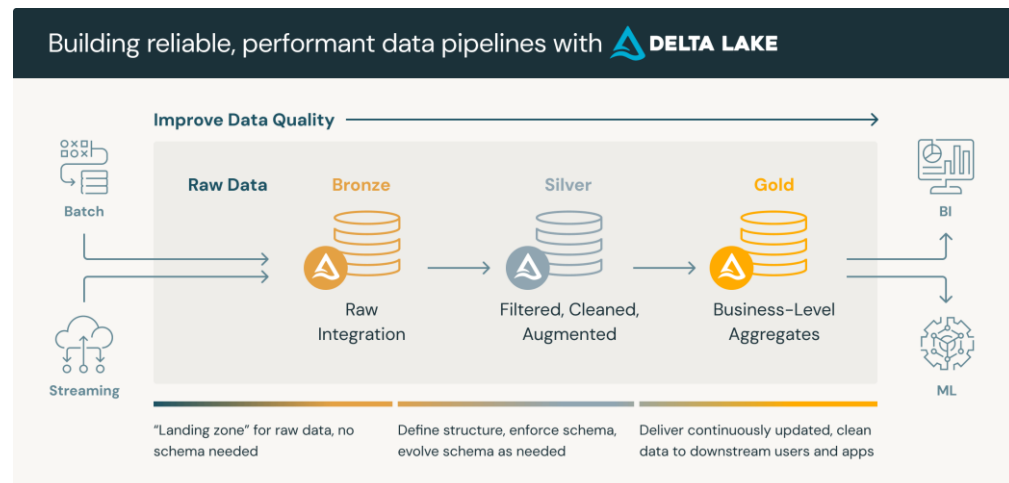
May be low quality data, may contain errors, duplicates, missing values

Silver Layer (cleaned and conformed)

Data is cleaned and conformed

Gold Layer (analytical data)

Data ready for analytical reports



Bronze Layer (raw data)

Raw data from source systems, without processing

Silver Layer (cleaned and conformed)

Data is matched, merged, customized and cleaned so that the silver layer can provide an "enterprise view" of all key business units, concepts and transactions.

Removes duplicates, standardizes formats, merges data from different sources

Enables self-service analytics for ad-hoc reporting, advanced analytics and machine learning

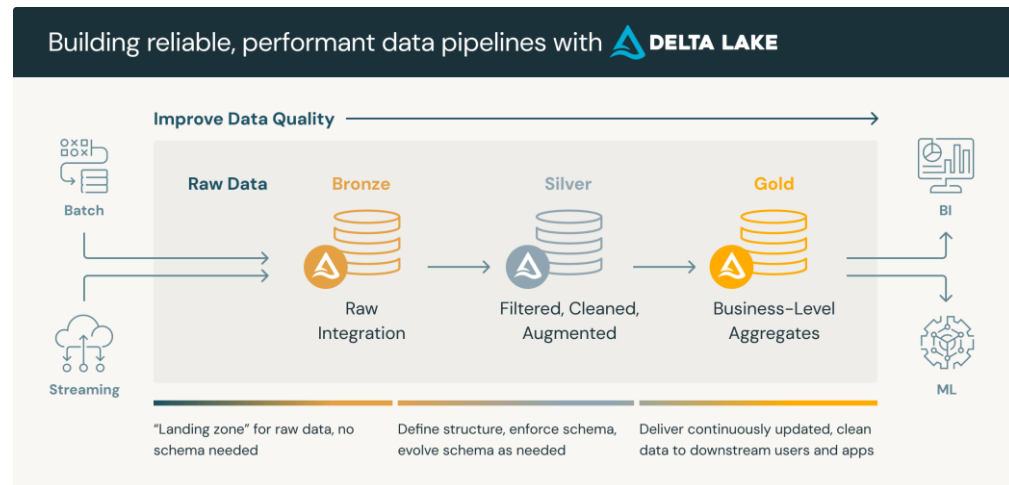
Speed and flexibility of data acquisition and delivery is prioritized

Only when loading data into the Gold layer are complex transformations and project-specific business rules applied

From a data modeling perspective, models like 3PN, generally write-efficient models, are used.

Gold Layer (analytical data)

Data ready for analytical reports



Bronze Layer (raw data)

Raw data from source systems,
without processing

Silver Layer (cleaned and conformed)

Data is cleaned and conformed

Gold Layer (dane końcowe/analityczne)

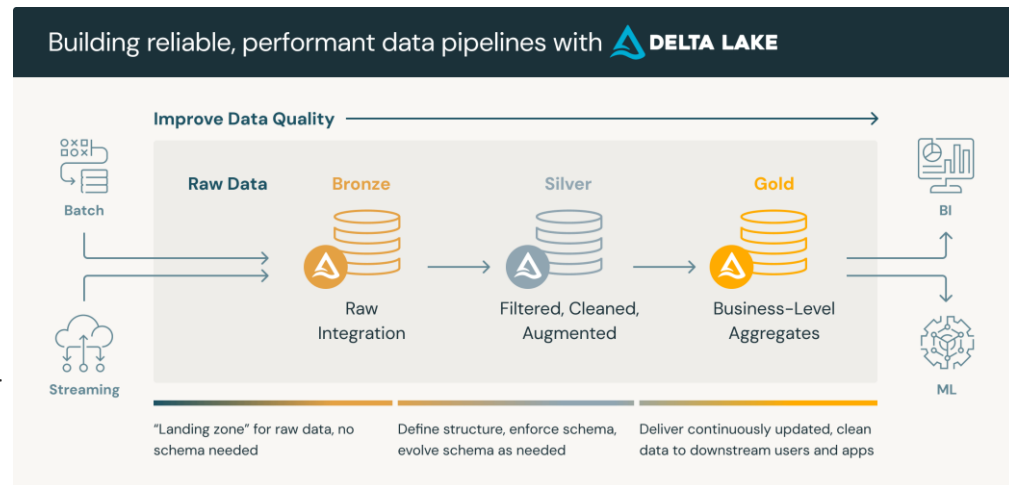
Data ready for reporting and
business analysis

Uses more denormalized and read-
optimized data models with fewer
joins

Last layer of data transformations
and data quality rules

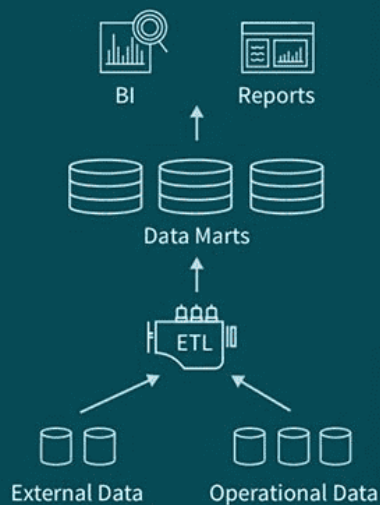
Oriented to specific business needs

Models solutions based on star
schema or far-restricted PTI.



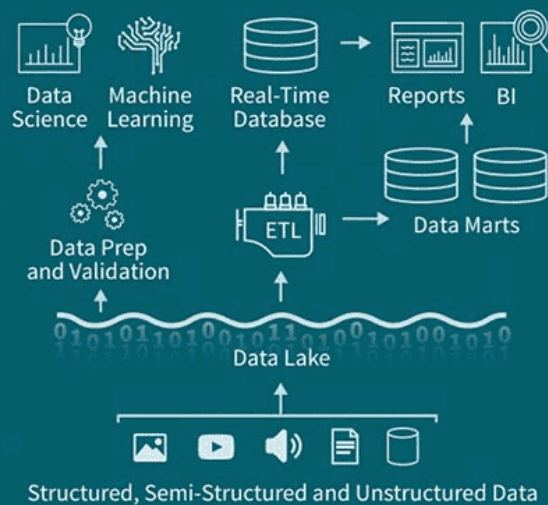
Late 1980's

Data Warehouse



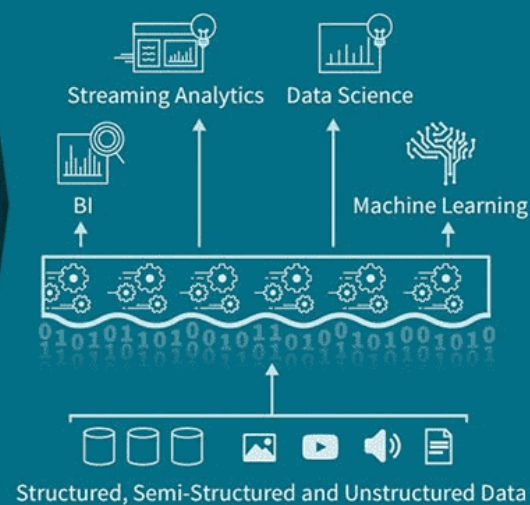
2011

Data Lake



2020

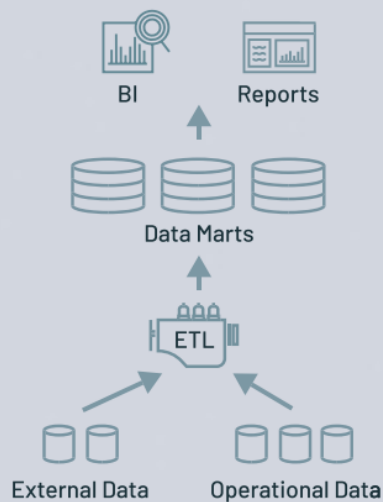
Lakehouse



SOURCE: DATABRICKS

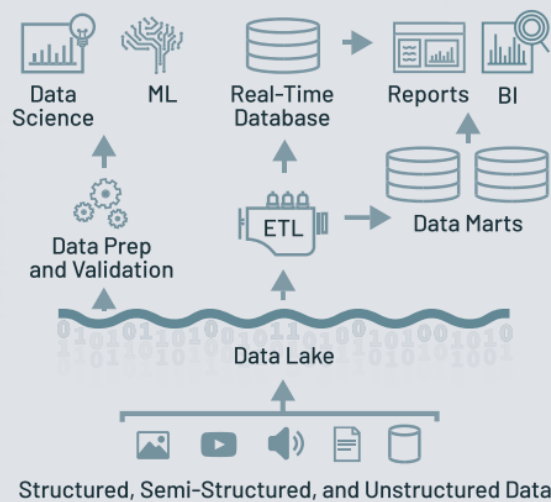
Late 1980s

DATA WAREHOUSE



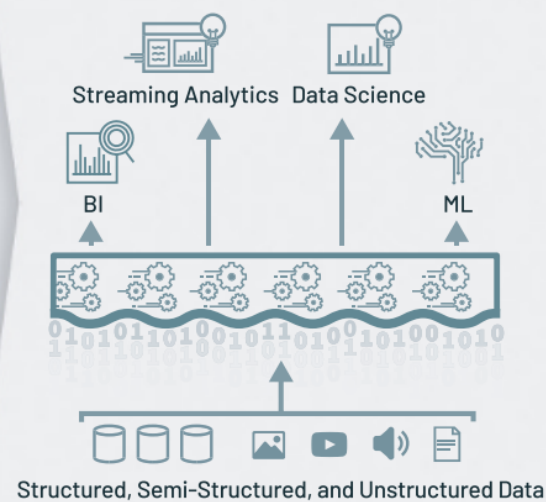
2011

DATA LAKE

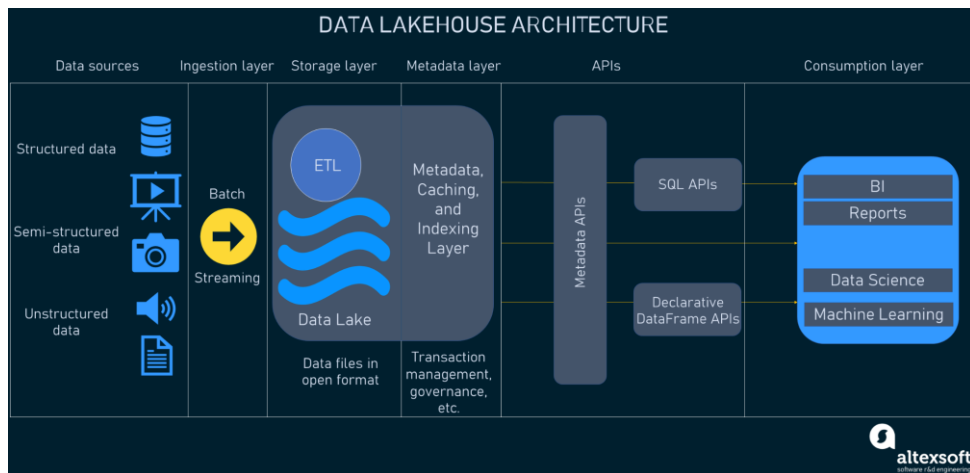


2020

LAKEHOUSE



SOURCE: DATABRICKS



Lakehouse is a concrete approach to creating data lakes

adding the attributes of data warehousing - reliability, performance and quality, while maintaining the openness and scale of data lakes.

ACID transactions

Handling large metadata

Lakehouse architecture treats metadata just like data, using the distributed computing power of Apache Spark to handle all metadata.

Indexing

Along with data partitioning, the lakehouse architecture includes various statistical techniques such as bloom filters

Schema validation

All data entered into a table must strictly conform to a defined schema.

If the data does not conform to the schema, it is moved to a quarantine where it can be examined and troubleshot later.



FIGURE 3-2: An example of an efficient data pipeline setup as part of a lakehouse approach.

The challenges that can be overcome with a data lakehouse approach:

Unifying data teams:

unifies all data teams - data engineers, data analysts and analysts - into a single architecture.

Breaking data silos:

Provide a complete and robust copy of all data in a centralized location.

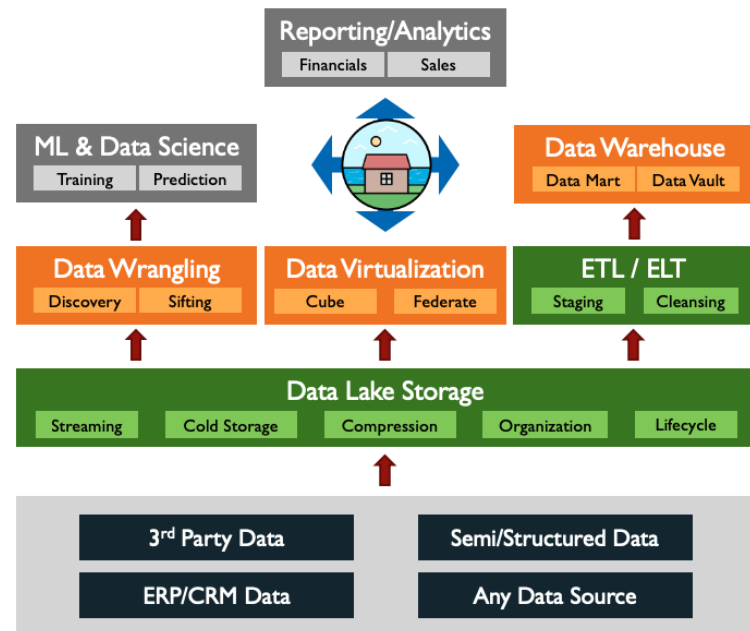
This allows everyone in the organization to access and manage structured and unstructured data.

Prevent data obsolescence:

Processes batch and streaming data, updating tables and dashboards in near real-time, so data always generates value, is updated and never becomes outdated.

Reduce the risk of single-vendor dependency:

open formats and open standards that allow data to be stored independently of the tools currently used to process it, making it easy to move data to another vendor or technology at any time.



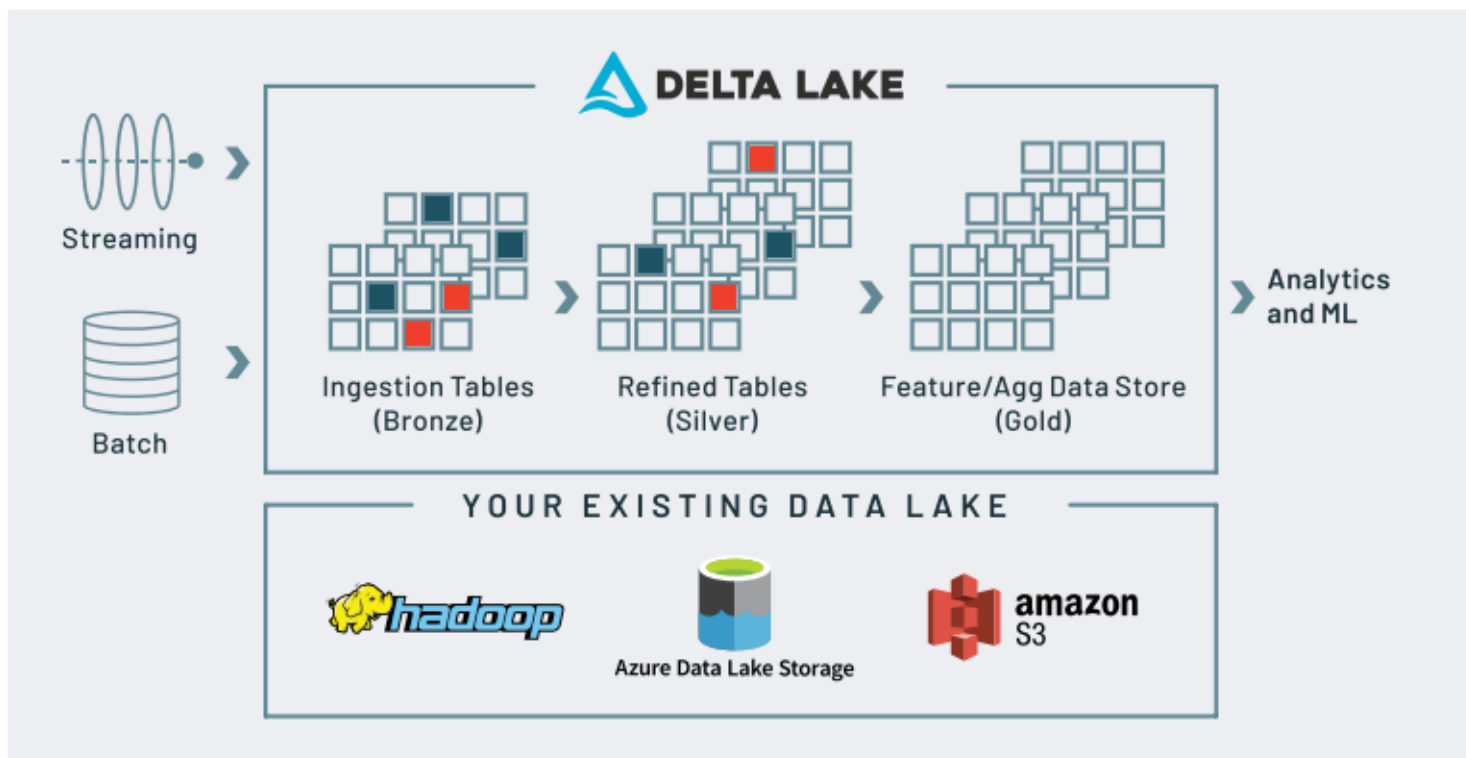


FIGURE 4-1: Delta Lake is an open-source storage layer that brings improved reliability to the lakehouse.



Microsoft Fabric

The data platform for the era of AI



Synapse Data
Warehousing



Synapse Data
Engineering



Data
Factory



Synapse Data
Science



Synapse Real
Time Analytics



Power BI

T-SQL

Spark

Serverless
compute

KQL

Analysis
Services

OneSecurity

Warehouse

Delta –
Parquet
Format

Lakehouse

Delta –
Parquet
Format



OneLake

Kusto DB

Delta –
Parquet
Format

Dataset

Delta –
Parquet
Format

„(..) how a developer at the fictional Wide World Importers company from the retail domain completes the following steps:

Sign into your Power BI account, or if you don't have one yet, sign up for a free trial.

Build and implement an end-to-end lakehouse for your organization:

Create a Fabric workspace

Create a lakehouse. It includes an optional section to implement the medallion architecture that is the bronze, silver, and gold layers.

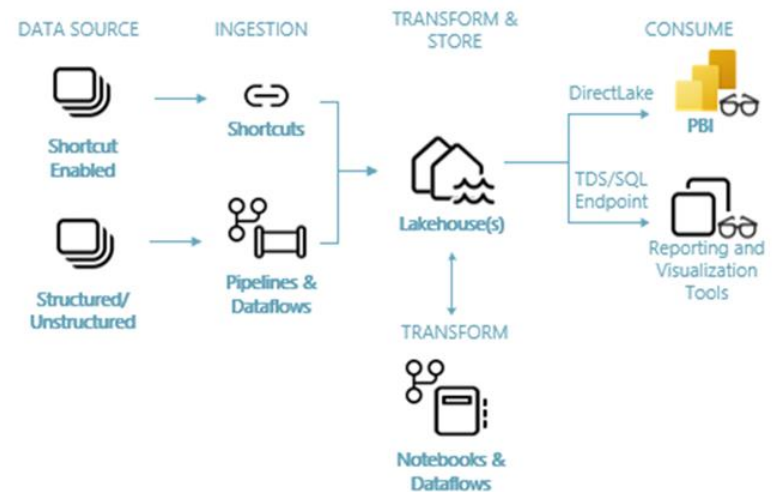
Ingest data, transform data, and load it into the lakehouse. Load data from the bronze, silver, and gold zones as delta lake tables. You can also explore the OneLake, OneCopy of your data across lake mode and warehouse mode.

Connect to your lakehouse using TDS/SQL endpoint and Create a Power BI report using DirectLake to analyze sales data across different dimensions.

Optionally, you can orchestrate and schedule data ingestion and transformation flow with a pipeline.

Clean up resources by deleting the workspace and other items.”

<https://learn.microsoft.com/en-us/fabric/data-engineering/tutorial-lakehouse-introduction>



Data Mesh

This decentralized approach treats data as a product, with domain-oriented teams responsible for their own data pipelines and governance.

It represents a shift from centralized, monolithic data architectures to a distributed model that aligns more closely with organizational structures.

Key Principles:

- Domain-driven data ownership and architecture

- Data as a product

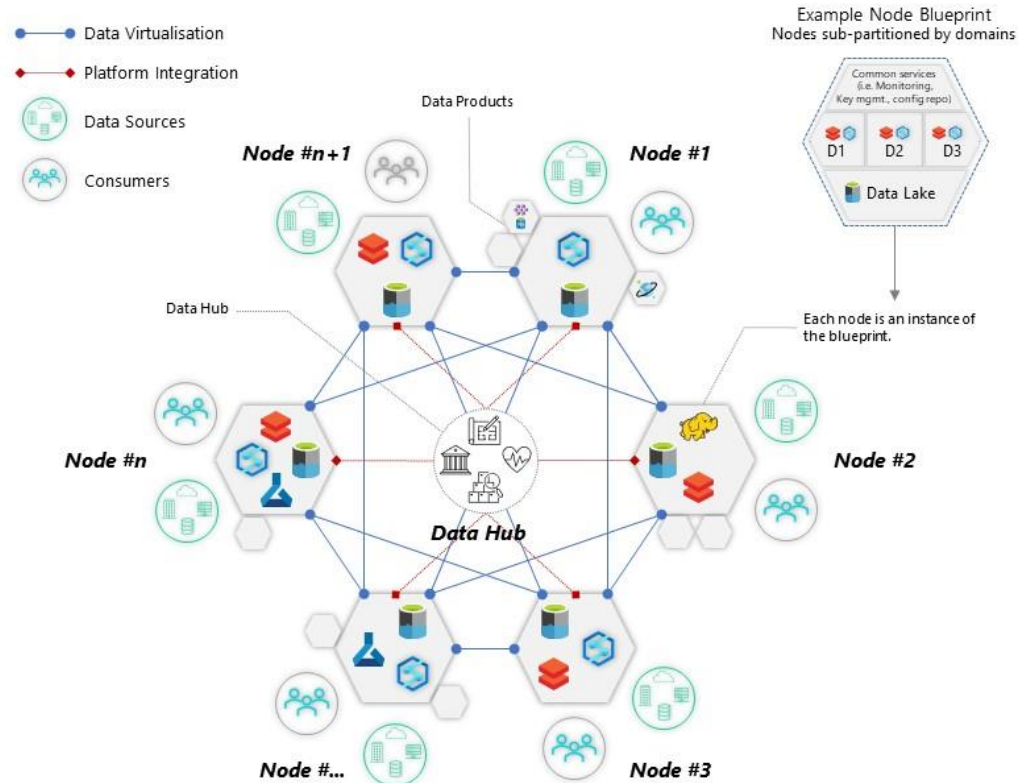
- Self-serve data infrastructure

- Federated computational governance

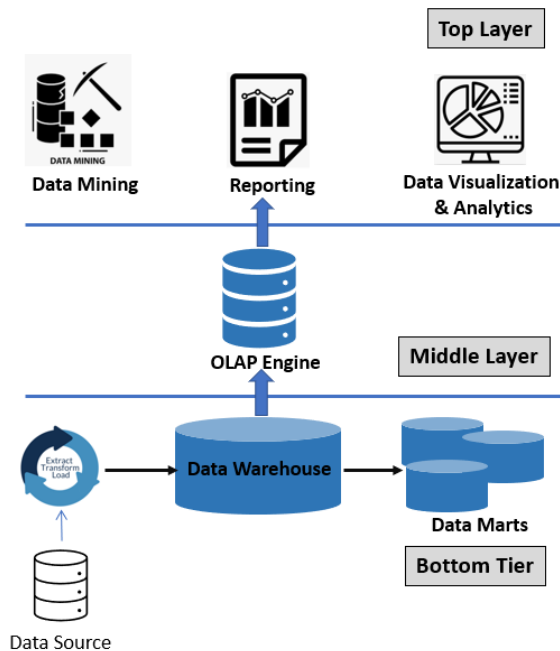
Data Mesh

Harmonised Mesh

- Azure Harmonised Mesh allows multiple groups within an organisation to operate their own analytics platform whilst adhering to common policies and standards.
- The central datahub hosts data catalogue, mesh wide audit capabilities, monitoring, and auxiliary services for automation.
- The central data platform group defines blueprints that encompass baseline security, policies, capabilities and standards.
- New nodes are instantiated based on these blueprints, which encompass key capabilities to enable enterprise analytics (i.e.. Storage, monitoring, key management, ELT, analytical engines, and automation)
- Node instances can be augmented to serve respective business requirements, i.e. deploying additional domains, customising domains and data products within the node.
- Nodes are typically split by either org-division, function, or region.



Data Warehouse Architecture



Cloud-Based Three-Tier Architecture

The most popular modern approach

Consists of:

Data Layer - handles data extraction, transformation, and storage:

Storage layer - Using distributed file systems and columnar formats

Processing layer - Leveraging MPP databases like Amazon Redshift, Google BigQuery, or Snowflake

Semantics Layer - Restructures data for analytics using OLAP and OLTP servers

Analytics Layer - Provides user interfaces and tools for data interaction:

Consumption layer - Featuring BI tools like Tableau, Power BI, or Looker

Key Features:

Elastic scaling of compute and storage resources

Pay-as-you-go pricing models

Built-in data sharing and collaboration tools

Automated maintenance and upgrades

MPP (Massively Parallel Processing) Architecture

This architecture distributes data processing across multiple servers or nodes:

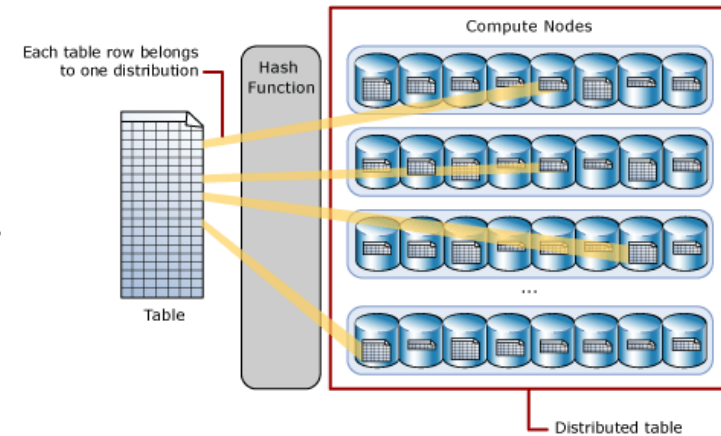
In-memory processing - Data loaded directly into memory for faster queries

Distributed storage - Data spread across multiple nodes

Shared-nothing architecture - Each node operates independently

Columnar storage - Data stored by columns rather than rows

Query optimization - Sophisticated techniques to distribute workloads efficiently



Multi-Structured Data Architecture

This approach accommodates diverse data formats and structures:

Data lake integration for storing raw, unprocessed data

NoSQL databases for handling various data formats

ELT approach for better handling of diverse data types

Schema-on-read methodology

Polyglot persistence using multiple storage technologies

Multi-Cloud and Hybrid Architectures

Organizations are increasingly adopting multi-cloud or hybrid approaches, combining on-premises infrastructure with multiple cloud providers. This strategy provides flexibility, avoids vendor lock-in, and allows for optimal resource allocation based on specific workload requirements.

Considerations:

Data integration and movement across environments

Consistent governance and security policies

Performance optimization across diverse infrastructures

Jensen C.S., Pedersen T.B., Thomsen C.,
Multidimensional Databases and Data Warehousing,
Morgan & Claypool Publishers series SYNTHESIS LECTURES ON
DATA MANAGEMENT, 2010

Inmon W.,
Building the Data Warehouse,
John Wiley & Sons, New York 2002

Sherman R.,
Business Intelligence Guidebook
Elsevier, 2015

Claudia Imhoff, Nicholas Gallemmo, Jonathan G. Geiger,
Mastering Data Warehouse Design - Relational and Dimensional
Techniques,
Wiley Publishing, Inc., 2003

Efraim Turban, Ramesh Sharda, Dursun Delen, David King,
Janine E. Aronson
Business Intelligence: A Managerial Approach
Prentice Hall, 2010

Sanjiv Purba (ed.)
Handbook of Data Management (1999 Edition)
Auerbach, 1999

Bibliography

SOURCES